Межгосударственный Авиационный Комитет

Авиационный Регистр

КВАЛИФИКАЦИОННЫЕ ТРЕБОВАНИЯ

ЧАСТЬ 178С

ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ БОРТОВОЙ АППАРАТУРЫ И СИСТЕМ ПРИ СЕРТИФИКАЦИИ АВИАЦИОННОЙ ТЕХНИКИ

-2016

Предисловие

Основой настоящих Квалификационных требований КТ-178С «Требования к программному обеспечению бортовой аппаратуры и систем при сертификации авиационной техники» является документ RTCA DO-178C «Software Considerations in Airborne Systems and Equipment Certification».

Документ КТ-178С развивает требования и методические подходы, приведенные в ранее выпущенном документе КТ-178В. Термины и определения, использованные в КТ-178В, по возможности сохранены. Заключение о соответствии ПО требованиям КТ-178С является обязательным при квалификации комплектующих изделий.

СОДЕРЖАНИЕ

1.0. ВВЕДЕНИЕ ... 1
1.1. Назначение документа ... 1
1.2. Область применения документа ... 1
1.3. Связь с другими документами ... 2
1.4. Рекомендации относительно работы с документом ... 2
1.5. Структура документа ... 4
1.6. Отличия данного документа от DO-178C ... 5
2.0. СИСТЕМНЫЕ АСПЕКТЫ, ОТНОСЯЩИЕСЯ К РАЗРАБОТКЕ ПО ... 7
2.1. Процесс отнесения системных требований к ПО ... 7
2.2. Информационные потоки между процессами ЖЦ системы и процессами ЖЦ ПО ... 10
2.2.1. Информационный поток от процессов системы к процессам ПО ... 10
2.2.2. Информационный поток из процессов ПО в процессы системы ... 10
2.2.3. Информационный поток между процессами ПО и процессами аппаратуры ... 11
2.3. Процесс оценки безопасности системы и уровень ПО ... 11
2.3.1. Взаимосвязь между ошибками в ПО и отказными состояниями ... 12
2.3.2. Категоризация отказных состояний ... 12
2.3.3. Определение уровня ПО ... 13
2.3.4. Назначение уровня ПО ... 14
2.4. Аспекты, связанные с архитектурой ... 15
2.4.1. Обособление ... 15
2.4.2. Многоверсионное разнородное ПО ... 16
2.4.3. Мониторинг безопасности ... 16
2.5. Аспекты, связанные с ПО, в процессах ЖЦ системы ... 16
2.5.1. Компоненты параметрических данных ... 17
2.5.2. Модифицируемое пользователем ПО ... 18
2.5.3. Готовое коммерческое ПО ... 18
2.5.4. Опционное ПО ... 19
2.5.5. ПО, загружаемое на месте эксплуатации ... 19
2.5.6. Аспекты, связанные с ПО, при верификации системы ... 20
2.6. Системные аспекты в процессах ЖЦ ПО ... 20
3.0. ЖИЗНЕННЫЙ ЦИКЛ ПО ... 20
3.1. Процессы жизненного цикла ПО ... 20
3.2. Определение жизненного цикла ПО ... 20
3.3. Критерии перехода между процессами ... 21
4.0. ПРОЦЕСС ПЛАНИРОВАНИЯ ПО ... 22

4.1. Цели процесса планирования ПО 22
4.2. Мероприятия процесса планировании ПО 22
4.3. Планы создания ПО 23
4.4. Планирование среды ЖЦ ПО 24
4.4.1. Среда разработки ПО 25
4.4.2. Аспекты, связанные с языками программирования и компиляторами 25
4.4.3. Среда тестирования ПО 26
4.5. Стандарты на разработку ПО 26
4.6. Рассмотрение процесса планирования ПО 27
5.0. ПРОЦЕССЫ РАЗРАБОТКИ ПО 27
5.1. Процесс разработки требований к ПО 28
5.1.1. Цели процесса разработки требований к ПО 28
5.1.2. Мероприятия процесса разработки требований к ПО 28
5.2. Процесс проектирования ПО 29
5.2.1. Цели процесса проектирования ПО 29
5.2.2. Мероприятия процесса проектирования ПО 29
5.2.3. Проектирование модифицируемого пользователем ПО 30
5.3.1. Цели процесса кодирования ПО 31
5.3.2. Мероприятия процесса кодирования ПО 31
5.4. Процесс интеграции 31
5.4.1. Цели процесса интеграции 31
5.4.2. Мероприятия процесса интеграции 32
5.5. Трассируемость процесса разработки ПО 32
6.0. ПРОЦЕСС ВЕРИФИКАЦИИ ПО 33
6.1. Назначение процесса верификации ПО 33
6.2. Обзор мероприятий процесса верификации ПО 34
6.3. Рассмотрения и анализы ПО 35
6.3.1. Рассмотрения и анализы требований высокого уровня 35
6.3.2. Рассмотрения и анализы требований низкого уровня 36
6.3.3. Рассмотрения и анализы архитектуры ПО 36
6.3.4. Рассмотрения и анализы Исходного Кода 37
6.3.5. Рассмотрения и анализы выходных данных процесса интеграции 37
6.4. Тестирование ПО 38
6.4.1. Среда тестирования 39
6.4.2. Выбор тестов на основе требований 40
6.4.2.1. Тестовые примеры в допустимом диапазоне 40

6.4.2.2. Тестовые примеры на робастность 40
6.4.3. Методы тестирования на основе требований 41
6.4.4. Анализ тестового покрытия 42
6.4.4.1. Анализ покрытия требований тестами 43
6.4.4.2. Анализ структурного покрытия 43
6.4.4.3. Устранение недостатков по результатам анализа структурного покрытия 43
6.4.5. Рассмотрения и анализы тестовых примеров, процедур и результатов тестов 44
6.5. Трассируемость процесса верификации ПО 44
6.6. Верификация компонентов параметрических данных 45
7.0. ПРОЦЕСС УПРАВЛЕНИЯ КОНФИГУРАЦИЕЙ ПО 45
7.1. Цели процесса управления конфигурацией ПО 46
7.2. Мероприятия процесса управления конфигурацией ПО 47
7.2.1. Идентификация конфигурации 47
7.2.2. Базовые версии и трассируемость 47
7.2.3. Регистрация и отслеживание проблем и корректирующие действия 48
7.2.4. Управление изменениями 48
7.2.5. Рассмотрение изменений 49
7.2.6. Учет состояния конфигурации 49
7.2.7. Хранение, воспроизведение и выпуск 49
7.3. Категории контроля 50
7.4. Контроль загрузки ПО 51
7.5. Контроль среды ЖЦ ПО 52
8.0. ПРОЦЕСС ГАРАНТИИ КАЧЕСТВА ПО 53
8.1. Цели процесса гарантии качества ПО 53
8.2. Мероприятия процесса гарантии качества ПО 53
8.3. Рассмотрение соответствия ПО 54
9.0. ПРОЦЕСС ВЗАИМОДЕЙСТВИЯ ЗАЯВИТЕЛЯ С СЕРТИФИЦИРУЮЩИМ ОРГАНОМ 55
9.1. Методы определения соответствия и планирование 55
9.2. Доказательство соответствия 55
9.3. Минимальный состав представляемых сертифицирующему органу данных ЖЦ ПО 56
9.4. Данные ЖЦ ПО, относящиеся к типовой конструкции 56
10.0. ОБЗОР ПРОЦЕССА СЕРТИФИКАЦИИ 57
10.1. Сертификационный базис 57
10.2. Учет ПО при сертификации 57
10.3. Определение соответствия 57
11.0. ДАННЫЕ ЖИЗНЕННОГО ЦИКЛА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 58

11.1. План Сертификации ПО 59
11.2. План Разработки ПО 60
11.3. План Верификации ПО 60
11.4. План Управления Конфигурацией ПО 61
11.5. План Гарантии Качества ПО 62
11.6. Стандарты на Разработку Требований к ПО 63
11.7. Стандарты на Проектирование ПО 63
11.8. Стандарты на Кодирование ПО 63
11.9. Требования к ПО 64
11.10. Описание Проекта ПО 64
11.11. Исходный Код 65
11.12. Исполняемый Объектный Код 65
11.13. Примеры и Процедуры Верификации ПО 65
11.14. Результаты Верификации ПО 65
11.15. Указатель Конфигурации Среды ЖЦ ПО 66
11.16. Указатель Конфигурации ПО 66
11.17. Сообщения о Проблемах 67
11.18. Записи Управления Конфигурацией ПО 67
11.19. Записи Гарантии Качества ПО 67
11.20. Итоговое Заключение о ПО 67
11.21. Данные Трассировки 68
11.22. Файл Компонента Параметрических Данных 69
12.0. ДОПОЛНИТЕЛЬНЫЕ АСПЕКТЫ 69
12.1. Использование ранее разработанного ПО 69
12.1.1. Модификации ранее разработанного ПО 69
12.1.2. Изменение установки на воздушном судне 70
12.1.3. Изменение условий применения или среды разработки ПО 70
12.1.4. Доработка базовой версии 71
12.1.5. Аспекты, связанные с управлением конфигурацией ПО 72
12.1.6. Аспекты, связанные с гарантией качества ПО 72
12.2. Квалификация инструмента 72
12.2.1. Определение необходимости квалификации инструмента 72
12.2.2. Определение уровня квалификации инструмента 72
12.2.3. Процесс квалификации инструмента 73
12.3. Альтернативные методы 73
12.3.1. Тестирование по методу полного перебора 74
12.3.2. Аспекты, связанные с верификацией многоверсионного разнородного ПО 74

12.3.2.1. Независимость многоверсионного разнородного ПО ... 75
12.3.2.2. Верификация ПО, реализуемого на нескольких процессорах ... 75
12.3.2.3. Верификация многоверсионного исходного кода ... 75
12.3.2.4. Квалификация инструментов для многоверсионного разнородного ПО ... 76
12.3.2.5. Применение нескольких симуляторов при верификации ... 76
12.3.3. Модели надежности ПО ... 76
12.3.4. История эксплуатации изделия ... 76
12.3.4.1. Релевантность истории эксплуатации ... 77
12.3.4.2. Достаточность собранной истории эксплуатации ... 78
12.3.4.3. Сбор, регистрация и анализ проблем, обнаруженных в течение истории эксплуатации 78
12.3.4.4. Информация периода эксплуатации, которая должна быть включена в План ... 79
Сертификации ПО ... 79
ПРИЛОЖЕНИЕ А ... 80
ЦЕЛИ И ВЫХОДНЫЕ ДАННЫЕ ПРОЦЕССОВ В ЗАВИСИМОСТИ ОТ УРОВНЯ ПО ... 80
ПРИЛОЖЕНИЕ В ... 91
СОКРАЩЕНИЯ И СЛОВАРЬ ТЕРМИНОВ ... 91

1.0. ВВЕДЕНИЕ

Быстрый рост использования программного обеспечения (ПО) в бортовых системах и оборудовании, применяемых на летательных аппаратах и двигателях, в начале 1980-х годов привел к необходимости разработки принятых в отрасли инструктивных материалов для удовлетворения требованиям норм летной годности. Документ КТ-178 Требования к Программному Обеспечению Бортовой Аппаратуры и Систем при Сертификации Авиационной Техники был разработан, чтобы удовлетворить эту потребность.

Настоящий документ, пересмотренный с учетом накопленного опыта, предоставляет авиационному сообществу инструктивные материалы для определения единообразно и с приемлемым уровнем доверия того, что ПО бортовых систем и оборудования удовлетворяет требованиям летной годности. По мере роста использования ПО, развития технологий и накопления опыта использования настоящего документа он будет пересматриваться и изменяться.

1.1. Назначение документа

Назначением данного документа является предоставление инструктивных материалов по созданию ПО бортовых систем и оборудования, которое выполняет предписанные ему функции с уровнем доверия к безопасности, удовлетворяющим требованиям норм летной годности. Инструктивные материалы включают:

- Цели процессов жизненного цикла (ЖЦ) ПО.
- Мероприятия, обеспечивающие средства для достижения этих целей.
- Описания свидетельств в форме данных ЖЦ ПО, которые показывают, что эти цели были достигнуты.
- Различия в целях, требованиях к независимости, данных ЖЦ ПО и категориях контроля в зависимости от уровня ПО.
- Дополнительные аспекты (например, по ранее разработанному ПО), которые принимаются во внимание в определенных условиях применения.
- Определения терминов, представленные в словаре терминов.

Чтобы помочь читателю в понимании представленных инструктивных материалов, в настоящий документ включены также и пояснительные материалы.

1.2. Область применения документа

В настоящем документе рассматриваются те аспекты сертификации, которые относятся к созданию ПО для бортовых систем и оборудования, используемых на воздушных судах, двигателях, винтах. Чтобы облегчить понимание процесса сертификации, при обсуждении этих вопросов описывается ЖЦ системы и его связь с ЖЦ ПО. Полное описание процессов ЖЦ системы, включая процесс оценки безопасности системы и процесс валидации, как и описание процесса сертификации, не является назначением данного документа.

Приведенные в настоящем документе инструктивные материалы не определяют и не подразумевают степень участия сертифицирующего органа в процессе сертификации. Чтобы понять степень участия сертифицирующего органа, Заявитель должен обратиться к соответствующим правилам и инструктивным материалам, выпущенным этим сертифицирующим органом.

Поскольку вопросы сертификации обсуждаются только в части ЖЦ ПО, эксплуатационные аспекты создаваемого ПО не рассматриваются. Например, аспекты сертификации данных, которые могут меняться пользователем, выходят за область применения настоящего документа.

Этот документ не делает попытку определить встроенное микропрограммное обеспечение. Микропрограммное обеспечение должно быть классифицировано как аппаратура или как ПО, и к нему должны быть применены соответствующие процессы. В настоящем документе предполагается, что при проектировании системы функции были распределены между ПО и аппаратурой. Существуют другие документы, которые предоставляют инструктивные материалы по гарантии разработки для функций, которые были отнесены к реализации в аппаратуре. В настоящем документе предоставляются инструктивные материалы для функций, которые были отнесены к ПО.

Примечание: Это позволяет определить эффективный метод реализации и гарантии разработки в момент, когда определяются характеристики системы и распределяются функции. Все заинтересованные стороны должны согласиться с этим решением в тот момент, когда такое распределение функций производится.

Вопросы относительно структуры организации Заявителя, коммерческих связей между Заявителем и его поставщиками и критерии аттестации персонала остаются вне области действия этого документа.

1.3. Связь с другими документами

Дополнительно к требованиям норм летной годности существуют различные национальные и международные стандарты на ПО. В некоторых сообществах может потребоваться их выполнение. Однако требования применения конкретных национальных или международных стандартов или предложение способов применения этих стандартов как альтернативных или в дополнение к настоящему документу выходят за рамки настоящего документа.

Общепризнано, что проекты могут быть обязаны, по контракту или иным способом, соответствовать дополнительным стандартам, которые задаются, например, производителем двигателя или воздушного судна. Такие стандарты могут быть выведены из общих стандартов, создаваемых или принимаемых производителями для своей деятельности, и их необходимо рассматривать в процессе планирования ПО и учитывать, по необходимости, при контроле поставщиков.

1.4. Рекомендации относительно работы с документом

При использовании документа необходимо иметь в виду следующее:

a. Настоящий документ предназначен для использования государствами-участниками Соглашения о гражданской авиации и об использовании воздушного пространства, утверждённого 30 декабря 1991 года в г.Минск (далее по тексту - Соглашение). Чтобы способствовать этому, ссылки на конкретные национальные нормы и правила сведены к минимуму. Вместо этого применяются общеупотребительные термины. Например, термин «Сертифицирующий орган» используется для обозначения организации или лица, выдающего удостоверение летной годности от имени страны, ответственной за сертификацию типа (например, воздушного судна, двигателя или комплектующего изделия). Если другая страна или группа стран санкционирует такую сертификацию или участвует в ней, настоящий документ может быть использован при должном признании двусторонних соглашений или меморандумов о взаимопонимании между участвующими странами.

b. В настоящем документе признается, что изложенные в нем инструктивные материалы не имеют обязательной юридической силы, но представляют единодушное мнение авиационного сообщества. Кроме того, признается, что Заявитель может использовать методы, альтернативные приведенным в настоящем документе.

c. Если Заявитель принимает настоящий документ в качестве метода определения соответствия, то он должен достичь всех применимых целей. Настоящий документ необходимо применять к Заявителю и к

любому из его поставщиков, привлекаемому к любому из процессов ЖЦ ПО или выходным данным процессов, описываемых в настоящем документе. Заявитель ответственен за контроль всех своих поставщиков.

d. Заявитель должен запланировать набор мероприятий, необходимых для достижения этих целей. В настоящем документе описываются мероприятия по достижению этих целей. Заявитель может планировать и, при условии утверждения сертификационным органом, выполнять мероприятия, альтернативные описываемым в настоящем документе. Заявитель может также планировать и выполнять дополнительные мероприятия, которые считаются необходимыми.

e. Заявитель должен указать в своих планах и стандартах создания ПО любые дополнительные аспекты.

f. Заявитель должен выполнить запланированные мероприятия и предоставить свидетельства, как указано в разделе 11, чтобы доказать, что указанные цели достигнуты.

g. Чтобы обеспечить читателю понимание обсуждаемого вопроса, в текст документа включены пояснительные материалы. Например, в разделе 2 представлена информация, необходимая для понимания взаимодействия между ЖЦ системы и ЖЦ ПО. Аналогично, в разделе 3 представлено описание ЖЦ ПО, а в разделе 10 – обзор процесса сертификации.

h. В разделе 11 содержится перечень данных, которые обычно создаются для обеспечения аспектов ПО при сертификации. В тексте документа наименования таких данных выделяются с помощью заглавных букв, например: Исходный Код.

i. В разделе 12 обсуждаются дополнительные аспекты, включая инструктивные материалы, касающиеся использования ранее разработанного ПО, квалификации инструментов и применения методов, альтернативных описанным в разделах 2 – 11. Указания раздела 12 могут быть не применимы для некоторых проектов.

j. В Приложении А представлена применимость целей, мероприятий и данных ЖЦ ПО для каждого уровня ПО, а также различия в требованиях к независимости и категориях контроля для каждого уровня ПО. Чтобы полностью понять инструктивные материалы, необходимо учитывать все содержимое настоящего документа.

k. В тех случаях, когда для иллюстрации возможного применения настоящих инструктивных материалов используются примеры в графической или повествовательной форме, они не должны рассматриваться в качестве предпочтительных методов. В таких случаях эти примеры должны рассматриваться в качестве пояснительных материалов.

l. Любой приведенный в настоящем документе перечень (мероприятий, данных, и т.п.) не должен считаться исчерпывающим.

m. В настоящем документе примечания используются для изложения пояснительных материалов, придания особого значения некоторому вопросу или для привлечения внимания к смежным вопросам, выходящим за рамки контекста.

n. Основные разделы пронумерованы как X.0 в сквозном порядке во всем документе. Следует отметить, что ссылки на весь раздел обозначаются как «раздел X», в то время как ссылки на содержимое между заголовками разделов X.0 и X.1 указываются как «раздел X.0».

o. Существуют дополнения к настоящему документу, которые расширяют инструктивный материал, представленный в настоящем документе, в отношении конкретного подхода. Дополнения используются вместе с настоящим документом и могут также быть использованы друг с другом. Если не используются альтернативные методы (см. 1.4.i), и если для конкретного подхода существует дополнение, это дополнение необходимо использовать для добавления, удаления или иного изменения целей, мероприятий, пояснительного текста и данных ЖЦ ПО, представленных в настоящем документе, в

отношении этого подхода, как соответствующим образом определено в каждом дополнении. Ответственность за то, что данное дополнение приемлемо для конкретного сертифицирующего органа, лежит на Заявителе. В рамках процесса планирования ПО Заявитель должен рассмотреть все потенциально применимые дополнения и определить те, которые он будет использовать. Информацию из дополнений необходимо использовать вместе с настоящим документом и таким же образом. Приложение А каждого дополнения определяет, как цели настоящего документа пересматриваются в отношении конкретного подхода, рассматриваемого в этом дополнении.

р. Соответствие достигается, когда все применимые цели достигнуты за счет выполнения всех запланированных мероприятий и получения соответствующих свидетельств.

1.5. Структура документа

Структура данного документа в виде перечня разделов и их взаимосвязи приведена на Рисунке 1-1.

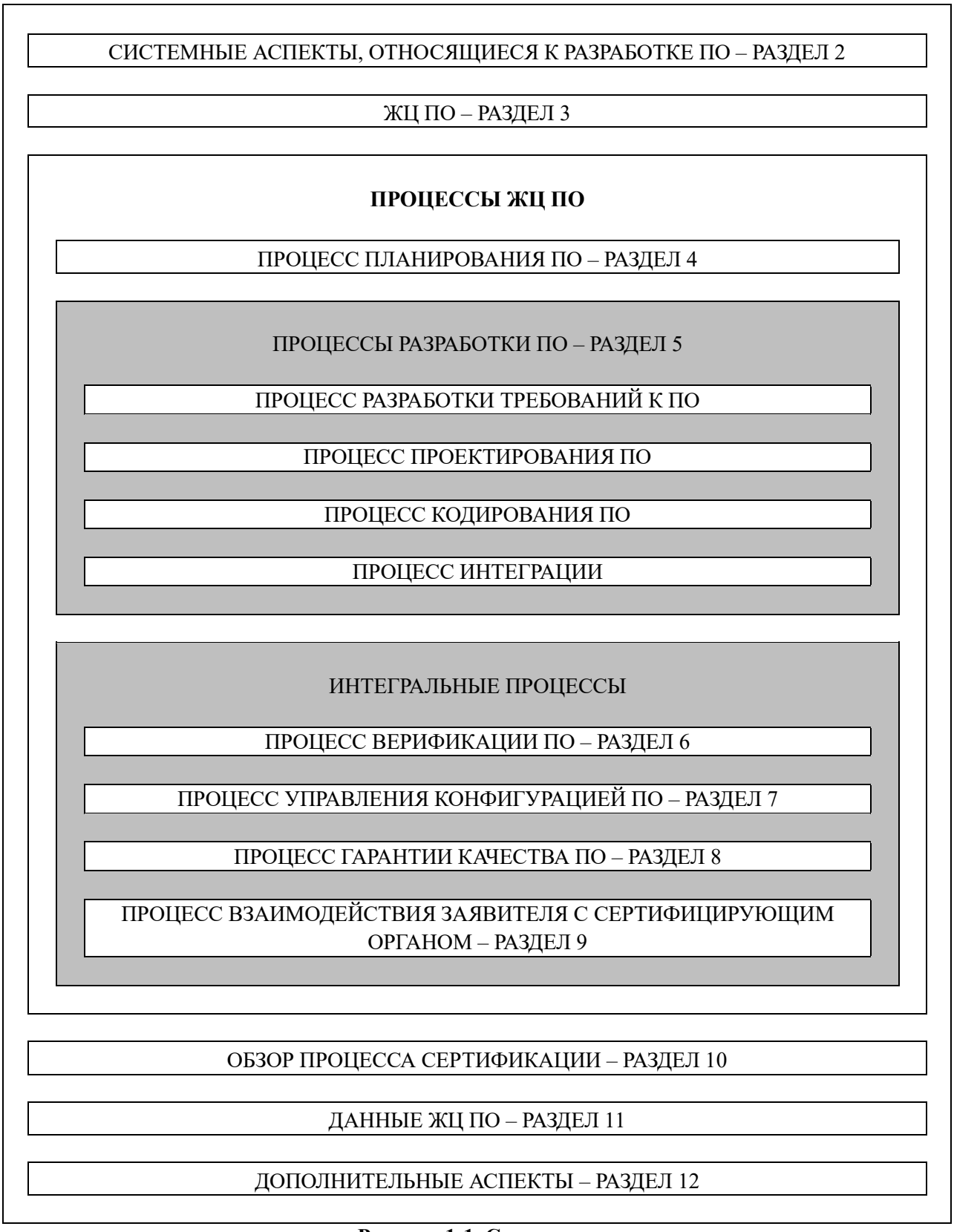


Рисунок 1-1. Структура документа

1.6. Отличия данного документа от DO-178C

Настоящий документ соответствует документу DO-178C. Отличия, описанные ниже, заключаются в специфике применения в государствах-участниках Соглашения.

a. В разделе 1.4.a указано, что документ предназначен для использования государствами-участниками Соглашения.

b. Удалены упоминания про сертификацию вспомогательной силовой установки (ВСУ).

c. Исправлены несоответствия с документом Р-4754 «Руководство по процессам сертификации высокоинтегрированных сложных бортовых систем воздушных судов гражданской авиации», касающиеся назначения уровня ПО в рамках процесса оценки безопасности системы.

2.0. СИСТЕМНЫЕ АСПЕКТЫ, ОТНОСЯЩИЕСЯ К РАЗРАБОТКЕ ПО

В данном разделе рассматриваются аспекты процессов ЖЦ систем, которые необходимы для понимания процессов ЖЦ ПО. Описание процессов ЖЦ системы можно найти в других отраслевых документах (например, Р-4754).

В настоящем разделе описываются:

- Процесс отнесения системных требований к ПО (см. 2.1). • Информационный поток между процессами ЖЦ системы и процессами ЖЦ ПО, а также между процессами ЖЦ ПО и процессами ЖЦ аппаратуры (см. 2.2).
- Процесс оценки безопасности системы, отказные состояния, определения уровней ПО и назначение уровней ПО (см. 2.3).
- Аспекты, связанные с архитектурой (см. 2.4).
- Аспекты, связанные с ПО в процессах ЖЦ системы (см. 2.5).
- Аспекты, связанные с системой в процессах ЖЦ ПО (см. 2.6).

Термин «система» в рамках настоящего документа относится только к бортовой системе и бортовому оборудованию, а не к более широкому определению системы, которое может включать в себя операторов, эксплуатационные процедуры и т.п.

2.1. Процесс отнесения системных требований к ПО

Требования к системе разрабатываются в рамках процессов ЖЦ системы на основе эксплуатационных требований к системе и иных соображений, таких как требования по обеспечению безопасности, к информационной безопасности и к техническим характеристикам. Требования, связанные с обеспечением безопасности, получаются из процесса оценки безопасности системы и могут включать в себя требования по функциональности, работоспособности и надежности, а также проектные ограничения.

Процесс оценки безопасности системы определяет и классифицирует отказные состояния системы. В рамках процесса оценки безопасности системы определяются требования, связанные с обеспечением безопасности, чтобы гарантировать работоспособность системы за счет указания требуемой невосприимчивости к таким отказным состояниям и реакции системы на эти отказы. Эти требования определяются для аппаратуры и ПО, чтобы предотвратить или ограничить отрицательное влияние отказов, и могут обеспечивать обнаружение отказов, устойчивость к отказам, устранение отказов и предотвращение отказов.

Процессы системы отвечают за уточнение требований к системе и их отнесение к аппаратуре и (или) к ПО в соответствии с архитектурой системы.

Требования к системе, отнесенные к ПО, включая требования, связанные с обеспечением безопасности, уточняются и перерабатываются в требования к ПО, которые верифицируются в ходе мероприятий процесса верификации ПО. Эти требования и связанные с ними результаты верификации должны подтвердить, что ПО выполняет заданные функции в любых ожидаемых условиях эксплуатации. Требования к системе, отнесенные к ПО, могут включать в себя: a. Функциональные и эксплуатационные требования.

b. Требования к интерфейсу.

c. Требования к характеристикам.

d. Требования, связанные с обеспечением безопасности, включая стратегии обеспечения безопасности, проектные ограничения и методы проектирования, такие как обособление, разнородность, резервирование и мониторинг безопасности. В случаях, когда система является частью другой системы,

требования и отказные состояния для этой другой системы могут также формировать часть требований к системе, отнесенных к ПО.

e. Требования к информационной безопасности.

f. Требования по техническому обслуживанию.

g. Сертификационные требования, включая любые соответствующие правила и инструкции, выпускаемые сертифицирующей организацией, и т. д.

h. Дополнительные требования, необходимые для оказания поддержки процессам ЖЦ системы.

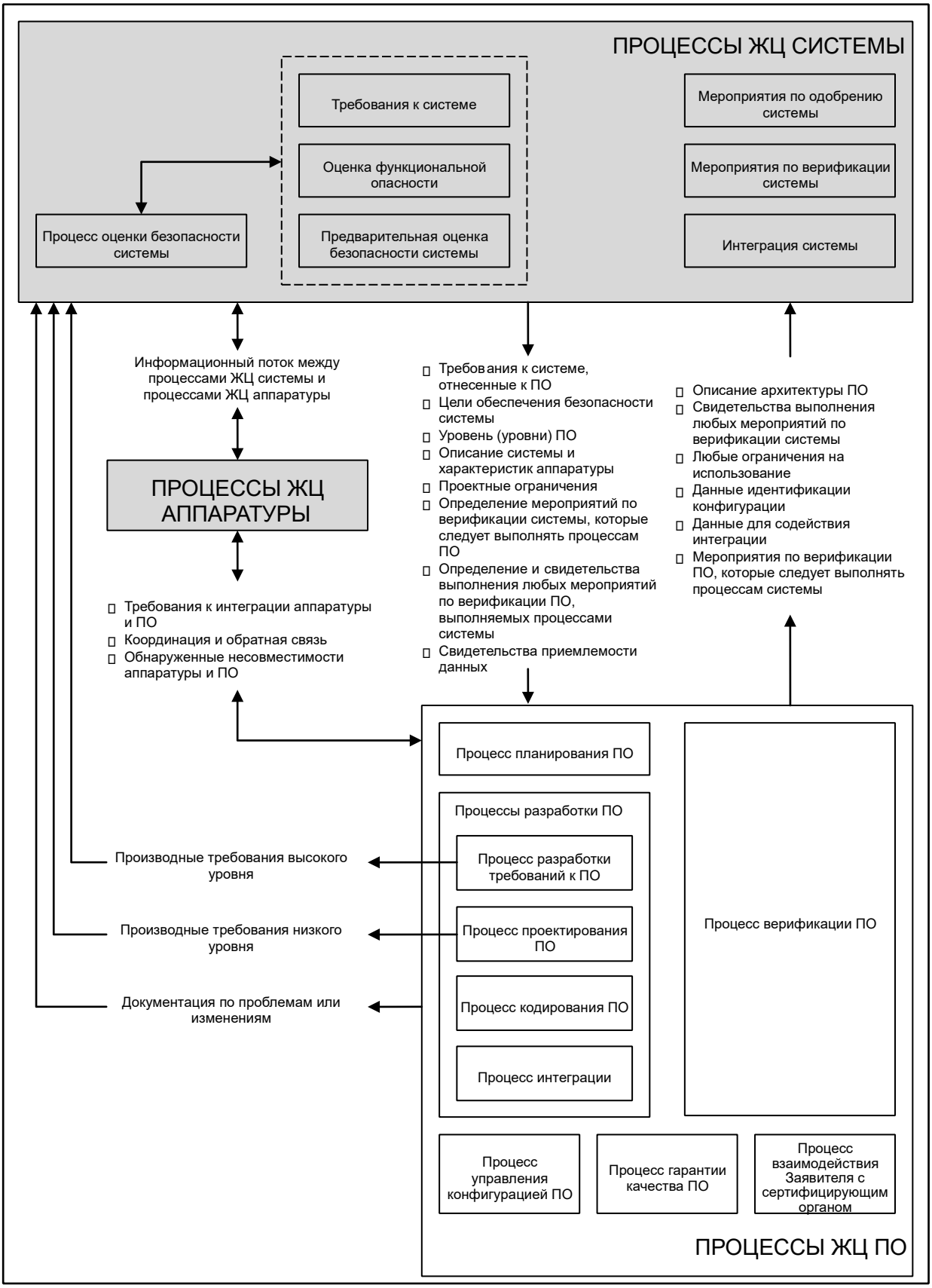


Рисунок 2-1. Информационные потоки между процессами ЖЦ системы и процессами ЖЦ ПО

2.2. Информационные потоки между процессами ЖЦ системы и процессами ЖЦ ПО

На Рисунке 2-1 представлен общий вид информационных потоков между процессами ЖЦ системы и процессами ЖЦ ПО. Эти информационные потоки включают в себя аспекты безопасности системы. В связи с взаимной зависимостью процесса оценки безопасности и процесса проектирования системы поток информации, представленный в настоящем разделе, является итеративным.

2.2.1. Информационный поток от процессов системы к процессам ПО

В рамках процесса отнесения требований или во время ЖЦ разработки процессы системы передают процессам ЖЦ ПО следующие данные:

a. Требования к системе, отнесенные к ПО.

b. Цели обеспечения безопасности системы.

c. Уровень ПО для компонентов ПО и описание связанного отказного состояния (состояний), если применимо.

d. Описание системы и характеристики аппаратуры.

e. Проектные ограничения, включая внешние интерфейсы, требования обособления и т.п.

f. Детальная информация о любых мероприятиях ЖЦ системы, которые предлагается выполнять в качестве части ЖЦ ПО. Следует отметить, что подтверждение требований к системе обычно не является частью процессов ЖЦ ПО. Процессы ЖЦ системы отвечают за гарантию любых мероприятий ЖЦ системы, которые предлагается выполнять в качестве части ЖЦ ПО.

g. Свидетельства приемлемости или неприемлемости любых данных, предоставленных процессами ПО процессам системы, на основании которых процессами системы было выполнено какое-либо мероприятие. Примерами таких мероприятий является оценка в процессах системы:

    1. Производных требований, создаваемых в процессах ПО, для определения того, имеется ли какоелибо влияние на оценку безопасности системы и на требования к системе.

    2. Проблем, поднятых процессами ПО в отношении прояснения или корректировки требований к системе, отнесенных к ПО.

h. Свидетельства выполнения мероприятий по верификации ПО, проведенных процессами ЖЦ системы, если таковые имеются.

Процессы ЖЦ ПО должны рассматривать любые свидетельства, предоставляемые процессами системы (см. 2.2.1.f и 2.2.1.g), как Результаты Верификации ПО (см. 11.14).

2.2.2. Информационный поток из процессов ПО в процессы системы

Анализ требований к системе, отнесенных к ПО, в процессах ЖЦ ПО осуществляется в рамках процесса разработки требований к ПО. Если такой анализ определяет, что какие-либо требования к системе являются неполными или некорректными, процессы ЖЦ ПО должны это фиксировать и предоставлять процессам системы для решения. Затем, по мере продвижения проектирования и реализации ПО добавляются детали и выполняются изменения, которые могут повлиять на оценку безопасности системы и на требования к системе.

Чтобы помочь оценить развивающийся проект и изменения проекта, процессы ЖЦ ПО должны предоставлять данные процессам системы, включая процесс оценки безопасности системы. Эти данные помогут в проведении анализов и экспертиз для установления влияния на оценку безопасности системы и на

требования к системе. Наиболее выгодным может оказаться совместное выполнение этих анализов и экспертиз процессами системы и процессами ПО. Такие данные включают в себя:

a. Детальную информацию по производным требованиям, созданным при выполнении процессов ЖЦ ПО.

b. Описание архитектуры ПО, включая обособление ПО.

c. Свидетельства выполнения процессами ЖЦ ПО мероприятий ЖЦ системы, если таковые имеются.

d. Документацию по проблемам или изменениям, включая проблемы, обнаруженные в требованиях к системе, отнесенных к ПО, а также обнаруженные несовместимости между аппаратурой и ПО. е. Любые ограничения на использование.

f. Идентификация конфигурации и любые ограничения состояния конфигурации.

g. Технические характеристики, временные характеристики и точность.

h. Данные для содействия интеграции ПО в систему.

i. Детальная информация о мероприятиях по верификации ПО, выполнение которых предполагается во время верификации системы, если таковые имеются.

2.2.3. Информационный поток между процессами ПО и процессами аппаратуры

Перечисленные ниже данные передаются между процессами ЖЦ ПО и процессами ЖЦ аппаратуры либо в ходе отнесения (к ПО и аппаратуре) требований к системе, либо в ходе других процессов разработки:

a. Все требования, включая производные требования, необходимые для интеграции аппаратуры и ПО, такие как определение протоколов, временные ограничения и схемы адресации для интерфейса между аппаратурой и ПО.

b. Случаи, в которых требуется координация мероприятий по верификации аппаратуры и ПО.

c. Обнаруженные несовместимости между аппаратурой и ПО.

2.3. Процесс оценки безопасности системы и уровень ПО

Настоящий раздел даёт краткое введение в то, как определяется уровень ПО для программных компонентов, и в то, как аспекты, связанные с архитектурой, могут повлиять на его назначение. Определение того, как это должно быть сделано, не входит в число задач настоящего документа; определение уровней должно быть выполнено как часть процессов ЖЦ системы.

Уровень ПО программного компонента основан на вкладе ПО в потенциальные отказные состояния. Это определяется процессом оценки безопасности системы, посредством определения того, как ошибка в программном компоненте связана с отказным состоянием (состояниями) системы и серьезности такого отказного состояния (состояний). Уровень ПО устанавливает степень строгости, необходимую для демонстрации соответствия настоящему документу.

Разработка ПО в соответствии с заданным уровнем ПО не подразумевает задания для этого ПО интенсивности отказов. Поэтому в процессе оценки безопасности системы нельзя использовать нормы надежности ПО на основе уровня ПО так же, как используются интенсивности отказов аппаратуры.

Процессы системы могут назначать индивидуальные уровни ПО только обособленным программным компонентам (см. 2.4.1). Если невозможно продемонстрировать обособление между компонентами ПО, то при назначении уровней ПО эти компоненты необходимо рассматривать как единый компонент (т.е. всем компонентам назначается уровень ПО, соответствующий наиболее серьезному отказному состоянию, вклад в которое может внести ПО).

Заявитель должен определить используемый процесс оценки безопасности системы на основании инструктивных материалов сертифицирующего органа. Затем, базируясь на этом процессе, каждому компоненту ПО необходимо назначить уровень ПО и согласовать его с сертифицирующим органом.

2.3.1. Взаимосвязь между ошибками в ПО и отказными состояниями

На Рисунке 2-2 представлена последовательность событий, при которых ошибка в ПО ведет к отказному состоянию на уровне ВС. Ошибка в ПО может быть скрытой и, таким образом, не сразу привести к отказу. Эта модель специально упрощена до линейного представления. В реальной эксплуатации последовательность событий, которые ведут от ошибки в ПО к отказному состоянию, может быть сложной, и ее трудно изобразить в виде последовательности событий, как показано на Рисунке 2-2.

Важно понимать, что вероятность того, что ПО содержит ошибку, не может быть оценена количественно таким же образом, как для случайных отказов аппаратуры.

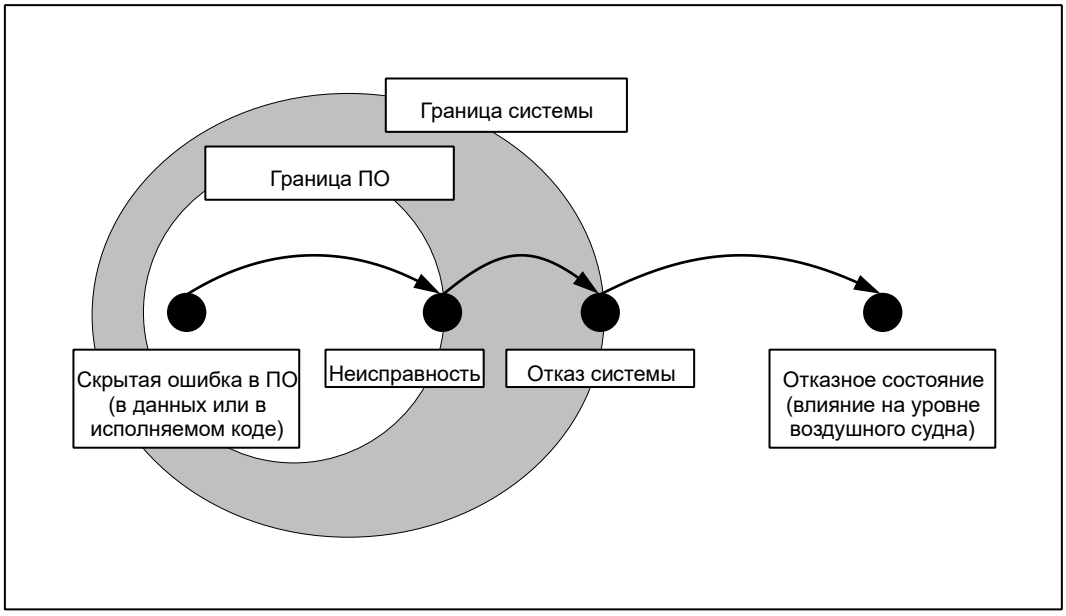


Рисунок 2-2. Последовательность событий для ошибки в ПО, ведущей к отказному состоянию

Аспекты, связанные с архитектурой, (см. 2.4) и (или) внешние для системы факторы также могут рассматриваться как часть процесса оценки безопасности системы при определении категорий отказных состояний и назначения уровня ПО каждому программному компоненту.

2.3.2. Категоризация отказных состояний

Для полного определения категорий отказных состояний Заявитель должен обратиться к соответствующим нормативам и инструктивным материалам, выпускаемым соответствующим сертифицирующим органом. Перечисленные в Таблице 2-1 категории отказных состояний действительны для больших транспортных ВС на основании Рекомендательного Циркуляра № РЦ-АП-25.1309 АР МАК и включены для облегчения использования настоящего документа.

Таблица 2-1. Описание категорий отказных состояний

Категория	Описание

Катастрофические	Отказные состояния, которые приводят к многочисленным жертвам, обычно с потерей ВС.
Аварийные	Отказные состояния, которые снижают способность ВС или возможности экипажа справиться с неблагоприятными условиями до такой степени, что имеет место: • значительное снижение запасов по безопасности или функциональных возможностей; или • ухудшение условий работы или чрезмерная рабочая нагрузка, такие, что нельзя полагаться на то, что летный экипаж выполнит свои задачи точно и полностью; или • серьезные или смертельные ранения относительно небольшого числа находящихся на борту людей, не входящих в летный экипаж.
Сложные (значительные)	Отказные состояния, которые снижают способность ВС или возможности экипажа справиться с неблагоприятными условиями до такой степени, что имеет место, например: • заметное снижение запасов по безопасности или функциональных возможностей; или • заметное увеличение рабочей нагрузки на экипаж или появление условий, понижающих эффективность работы экипажа; или • дискомфорт для летного экипажа или ухудшение условий для пассажиров или бортпроводников, возможно, с причинением травм.
Усложнение условий полета (незначительные)	Отказные состояния, которые незначительно снижают безопасность полета ВС и которые требуют от экипажа действий, заведомо находящихся в пределах их возможностей. Эти отказные состояния характеризуются, например: • незначительным снижением запасов по безопасности или функциональных возможностей; или • незначительным увеличением рабочей нагрузки на экипаж, таким как обычное изменение плана полета; или • некоторым физическим дискомфортом для пассажиров или бортпроводников.
Не влияющие на безопасность	Отказные состояния, которые не влияют на безопасность; например, отказные состояния, которые не оказывают неблагоприятного влияния на эксплуатационные возможности ВС или не увеличивают рабочую нагрузку на экипаж.

2.3.3. Определение уровня ПО

В настоящем документе определяются пять уровней ПО, от уровня A до уровня E. На примере категорий отказных состояний, перечисленных в разделе 2.3.2, связь между этими уровнями ПО и отказными состояниями следующая:

a. Уровень A: ПО, ненормальное поведение которого, согласно результатам процесса оценки безопасности системы, может вызвать отказ или способствовать отказу функции системы, который приводит к катастрофическому отказному состоянию для воздушного судна.

b. Уровень B: ПО, ненормальное поведение которого, согласно результатам процесса оценки безопасности системы, может вызвать отказ или способствовать отказу функции системы, который приводит к аварийному отказному состоянию для воздушного судна.

c. Уровень C: ПО, ненормальное поведение которого, согласно результатам процесса оценки безопасности системы, может вызвать отказ или способствовать отказу функции системы, который приводит к сложному отказному состоянию для воздушного судна.

d. Уровень D: ПО, ненормальное поведение которого, согласно результатам процесса оценки безопасности системы, может вызвать отказ или способствовать отказу функции системы, который приводит к отказному состоянию типа «усложнение условий полета» воздушного судна.

e. Уровень E: ПО, ненормальное поведение которого, согласно результатам процесса оценки безопасности системы, может вызвать отказ или способствовать отказу функции системы без влияния на эксплуатационные возможности воздушного судна или рабочую нагрузку на пилота. Получение от сертифицирующего органа подтверждения, что данное ПО относится к уровню E, означает, что к нему не применимы изложенные ниже положения данного документа.

При категоризации серьезности отказных состояний и уровня ПО Заявитель должен всегда учитывать соответствующие инструктивные материалы по сертификации и аспекты, связанные с разработкой системы.

2.3.4. Назначение уровня ПО

В процессе оценки безопасности системы назначается уровень (уровни) ПО, соответствующий компонентам ПО конкретной системы, на основании отказного состояния, к которому может привести ненормальное поведение ПО. Необходимо проанализировать влияние как невозможности выполнения функции, так и неправильного ее выполнения. При определении категорий отказных состояний и назначении уровня ПО могут учитываться внешние факторы, такие как неблагоприятные условия окружающей среды, а также архитектурные стратегии (как описано в разделе 2.4).

Примечание 1: Заявитель может пожелать рассмотреть запланированные функциональности, которые будут введены в дальнейшем, а также возможные изменения в требованиях к системе, отнесенных к ПО, что может привести к отказным состояниям более серьезной категории и более высокому уровню ПО. Поэтому может оказаться желательным разработать ПО более высокого уровня, чем тот, который был определен в процессе оценки безопасности системы для ее первоначального применения, так как последующая разработка данных ЖЦ ПО, устанавливающих применение с более высоким уровнем ПО, может оказаться затруднительной.

Примечание 2: Уровень ПО бортовых систем и оборудования, которые не влияют на летную годность воздушного судна, но установка которых на борту является обязательной согласно эксплуатационным правилам (примером является регистратор полетных данных), должен соответствовать выполняемой ими функции. В некоторых случаях уровень ПО может быть установлен в стандартах, определяющих минимальные требования к характеристикам оборудования.

Если ненормальное поведение некоторого компонента ПО способствует возникновению более чем одного отказного состояния, то уровень ПО этого компонента определяется его наиболее серьезным отказным состоянием, возникновению которого данное ПО может способствовать, включая комбинированные отказные состояния.

2.4. Аспекты, связанные с архитектурой

В настоящем разделе представлена информация по нескольким архитектурным стратегиям, которые могут ограничивать влияние отказов или обнаруживать отказы и обеспечивать приемлемые реакции системы по их предотвращению. Эти подходы в архитектуре обычно определяются во время проектирования системы и не должны восприниматься как предпочтительные или требуемые решения.

Последовательная реализация – такая, в которой для выполнения функции системы используется несколько программных компонентов таким образом, что ненормальное поведение любого из этих компонентов может создать отказное состояние. В такой реализации программные компоненты будут иметь уровень ПО, соответствующий наиболее серьезной категории отказных состояний данной функции системы.

В реализациях, в которых для получения отказного состояния требуется ненормальное поведение двух или более обособленных программных компонентов, этот факт может быть учтен процессами системы при назначении уровня ПО для этих программных компонентов.

Процесс оценки безопасности системы устанавливает существование достаточной независимости между компонентами ПО в отношении как их функций (то есть требований высокого уровня), так и проекта (например, общие проектные элементы, языки и инструменты).

Если невозможно продемонстрировать обособление и независимость между программными компонентами, то при назначении уровней ПО они должны быть рассмотрены как единый программный компонент (т.е. всем компонентам назначается уровень ПО, соответствующий наиболее серьезному отказному состоянию, появлению которого может способствовать данный программный компонент).

2.4.1. Обособление

Обособление – это подход, позволяющий изолировать компоненты ПО с целью ограничения влияния и (или) изоляции отказов и потенциального снижения объема работ, связанных с верификацией ПО. Обособление программных компонентов может быть достигнуто путем выделения каждому компоненту отдельных ресурсов аппаратуры (т.е. на каждой аппаратной платформе в системе выполняется только один программный компонент). В качестве альтернативы могут быть реализованы средства обособления, допускающие выполнение нескольких программных компонентов на одной и той же аппаратной платформе. Вне зависимости от выбранного метода, для обособленных программных компонентов необходимо гарантировать следующее:

a. Обособленный программный компонент не должен иметь возможности искажать код, ввод-вывод или области хранения данных другого обособленного компонента ПО.

b. Обособленный программный компонент должен иметь возможность потреблять совместно используемые ресурсы процессора только в назначенное ему в соответствии с расписанием временное окно.

c. Отказы аппаратуры, выделенной для обособленного компонента ПО, не должны оказывать отрицательное влияние на другие обособленные компоненты ПО.

d. Любое ПО, обеспечивающее обособление, должно иметь тот же самый или более высокий уровень ПО, что и самый высокий уровень ПО, назначенный любому из обособленных компонентов ПО.

e. Любая аппаратура, обеспечивающая обособление, должна быть оценена в процессе оценки безопасности системы, чтобы убедиться, что она не влияет отрицательно на безопасность.

Процессы ЖЦ ПО должны учитывать аспекты, связанные с проектированием обособления. Они включают в себя степень и объем взаимодействия, допустимого между обособляемыми компонентами, и способ реализации защиты: с помощью аппаратуры, ПО или сочетанием аппаратуры и ПО.

2.4.2. Многоверсионное разнородное ПО

Многоверсионное разнородное ПО представляет собой подход в проектировании систем, связанный с созданием двух или более компонентов ПО, обеспечивающих выполнение одной и той же функции таким образом, чтобы избежать некоторых источников общих ошибок для этих компонентов. Многоверсионное разнородное ПО называют также многоверсионным ПО, многоверсионным независимым ПО, разнородным ПО, N-версионным программированием.

Процессы ЖЦ ПО, выполненные или начатые до применения метода разнородности в разработке, остаются потенциальными источниками ошибок. В требованиях к системе может оговариваться конфигурация аппаратуры, обеспечивающая исполнение многоверсионного разнородного ПО.

Степень разнородности и, следовательно, степень защиты, обычно не поддается количественному измерению. Вероятность потери функции системы не превышает величины, при которой возникшие ошибки обнаруживаются мониторингом безопасности, рассматривающим результат сравнения, превышающий пороговые значения. Версии разнородного ПО, таким образом, обычно используются в качестве средства дополнительной защиты, применяемой после того, как достигнуты цели процесса верификации для рассматриваемого уровня ПО, описанные в разделе 6.

Если в процессе оценки безопасности системы можно показать приемлемость потенциальной утраты функции системы, то методы верификации многоверсионного разнородного ПО могут быть сокращены по сравнению с методами верификации одноверсионного ПО.

Верификация многоверсионного разнородного ПО рассмотрена в п. 12.3.2.

2.4.3. Мониторинг безопасности

Мониторинг безопасности представляет собой средство защиты от конкретных отказных состояний путем непосредственного мониторинга функции на предмет отказов, которые могли бы привести к отказному состоянию. Функции мониторинга могут быть осуществлены в аппаратуре, в ПО или сочетанием аппаратуры и ПО.

Использование подходов мониторинга безопасности может позволить назначить ПО, подлежащему мониторингу, уровень, соответствующий потере связанной с ним функции системы. Чтобы разрешить такое назначение, следует определить три важные характеристики монитора:

a. Уровень ПО: Программному обеспечению мониторинга безопасности назначается уровень, соответствующий наиболее серьезной категории отказного состояния для контролируемой функции.

b. Полнота контроля неисправностей системы. Оценка полноты контроля неисправностей системы монитором гарантирует, что его проект и реализация таковы, что неисправности, подлежащие обнаружению, будут обнаружены при всех необходимых условиях.

c. Независимость функции и монитора. Монитор и механизм защиты не должны терять работоспособность в результате возникновения того же отказа, который вызывает отказное состояние.

2.5. Аспекты, связанные с ПО, в процессах ЖЦ системы

В этом разделе представлен обзор вопросов (не обязательно взаимоисключающих), связанных с ПО, которые следует учитывать, при необходимости, в процессах ЖЦ системы:

a. Компоненты параметрических данных.

b. Модифицируемое пользователем ПО.

c. Готовое коммерческое ПО.

d. Опционное ПО.

e. Загружаемое на месте эксплуатации ПО.

f. Аспекты, связанные с ПО, при верификации системы.

2.5.1. Компоненты параметрических данных

ПО состоит из Исполняемого Объектного Кода и (или) данных, и может включать одну или несколько единиц конфигурации. Набор данных, которые влияют на поведение ПО без изменения Исполняемого Объектного Кода, и управляемый как отдельная единица конфигурации, называется компонентом параметрических данных. Таким образом, когда речь идет о компонентах параметрических данных и Исполняемом Объектном Коде, подразумевается, что компоненты параметрических данных не являются частью Исполняемого Объектного Кода.

Компонент параметрических данных включает в себя структуру из отдельных элементов, каждому из которых может быть назначено конкретное значение. Каждый элемент обладает атрибутами, такими как тип, диапазон или набор допустимых значений.

Примерами компонентов параметрических данных являются таблицы и базы данных конфигурации, но не аэронавигационные данные, поскольку они выходят за пределы рассмотрения настоящего документа. Компоненты параметрических данных могут содержать данные, которые могут: a. Влиять на пути выполнения Исполняемого Объектного Кода.

b. Подключать или отключать компоненты ПО и функции.

c. Адаптировать вычисления, выполняемые ПО, к конкретной конфигурации системы.

d. Использоваться в качестве данных для вычислений.

e. Определять распределение времени и памяти при обособлении.

f. Обеспечивать начальные значения для компонентов ПО.

В зависимости от того, как компонент параметрических данных будет использоваться в бортовой системе, необходимо учесть следующее:

- Инструктивный материал по модифицируемому пользователем ПО.
- Инструктивный материал по опционному ПО. В случаях, когда компонент параметрических данных подключает или отключает функции, необходимо также обратиться к инструктивному материалу для отключенного кода.
- Инструктивный материал по ПО, загружаемому на месте эксплуатации. Необходимо уделить особое внимание обнаружению искаженных компонентов параметрических данных, а также несовместимости между Исполняемым Объектным Кодом и компонентами параметрических данных.

Компоненту параметрических данных необходимо назначать такой же уровень ПО, как и программному компоненту, использующему его.

Дополнительную информацию по верификации компонентов параметрических данных см. в разделе 6.6.

2.5.2. Модифицируемое пользователем ПО

Модифицируемый пользователем компонент является частью ПО, которая может изменяться пользователем в пределах ограничений модификации без рассмотрения сертифицирующим органом, если требования к системе предусматривают модификацию ПО пользователем. Немодифицируемый компонент – это такой компонент, изменение которого пользователем не предполагается. Потенциальное влияние модификации пользователем определяется процессом оценки безопасности системы и используется для разработки требований к ПО, а затем – мероприятий процесса верификации ПО. Проектирование модифицируемого пользователем ПО рассматривается далее в разделе 5.2.3. Изменение, которое влияет на немодифицируемое ПО, его защиту или границы модифицируемого ПО, представляет собой модификацию ПО и рассматривается в разделе 12.1.1.

Инструктивный материал по модифицируемому пользователем ПО включает в себя следующее:

a. Модифицируемое пользователем ПО не должно оказывать отрицательное влияние на безопасность, эксплуатационные возможности, рабочую нагрузку летного экипажа, любые немодифицируемые компоненты ПО или на любые используемые механизмы защиты ПО. Если это не может быть подтверждено, то ПО не может быть классифицировано как модифицируемое пользователем. Также необходимо учитывать влияние на безопасность отображаемой информации на основании модифицируемого пользователем ПО.

b. Если требования к системе предусматривают модификацию пользователем, то пользователи могут модифицировать ПО в пределах ограничений модификации без рассмотрения сертифицирующим органом.

c. В требованиях к системе необходимо указывать методы, предотвращающие влияние внесенных пользователем изменений на безопасность системы, вне зависимости от того, корректно или нет они реализуются. Компоненту, который обеспечивает защиту от ошибок при внесении пользователем изменений, необходимо назначать тот же уровень ПО, что и функции, которую он защищает от ошибок в модифицируемом компоненте.

d. Если в требования к системе не включено положение о модификации пользователем, то ПО не должно модифицироваться пользователем, если только для модификации не будет продемонстрировано соответствие настоящему документу.

e. На время внесения изменений пользователь берет на себя ответственность за все, что связано с модификацией ПО, например: управление конфигурацией ПО, гарантия качества ПО и верификация ПО.

f. Заявитель должен предоставить необходимую информацию, чтобы позволить пользователю управлять ПО таким образом, чтобы это не оказывало негативного влияния на безопасность ВС.

2.5.3. Готовое коммерческое ПО

Готовое коммерческое ПО, включаемое в бортовые системы или оборудование, должно соответствовать целям настоящего документа.

Если данных ЖЦ, подготовленных в процессе создания готового коммерческого ПО, недостаточно, то эти данные должны быть дополнены, чтобы соответствовать целям настоящего документа. В таком случае могут быть применимы инструктивные материалы из раздела 12.1.4 «Доработка базовой версии» и из раздела 12.3.4 «История эксплуатации изделия».

2.5.4. Опционное ПО

Некоторые бортовые системы и оборудование могут включать в себя опциональные функции, которые можно выбирать, используя возможности ПО, а не контакты разъемов аппаратуры. Функции опционного ПО используются для выбора конкретной конфигурации в целевом вычислителе. Инструктивный материал по отключенному коду см. в разделах 4.2.h, 5.2.4 и 6.4.4.3.d.2.

В тех случаях, когда в состав ПО включены дополнительные функции, выбираемые программным способом, необходимо предусмотреть средства, гарантирующие невозможность случайного выбора конфигурации, не утвержденной для данного целевого вычислителя и данных условий его установки.

2.5.5. ПО, загружаемое на месте эксплуатации

Термин «бортовое ПО, загружаемое на месте эксплуатации» относится к ПО, которое может быть загружено без демонтажа системы или оборудования. Требования к обеспечению безопасности, связанные с функцией загрузки ПО, являются частью требований к системе. Если ненамеренное включение функции загрузки ПО может вызвать отказное состояние системы, то в требованиях к системе необходимо указать требования в части обеспечения безопасности для функции загрузки ПО.

Аспекты, связанные с безопасностью системы, касающиеся загружаемого на месте эксплуатации ПО, включают в себя:

- Обнаружение искаженного или частично загруженного ПО,
- Определение последствий загрузки неподходящего ПО,
- Совместимость аппаратуры и ПО,
- Совместимость компонентов ПО,
- Совместимость ВС и ПО,
- Возможность непреднамеренного включения загрузки на месте эксплуатации,
- Потеря или искажение отображения идентификации конфигурации ПО.

Инструктивные материалы по загружаемому на месте эксплуатации ПО включают в себя:

a. Если иное не подтверждено процессом оценки безопасности системы, механизмам обнаружения частичных или искаженных загрузок ПО необходимо назначить то же самое отказное состояние или уровень ПО, что и самое серьезное отказное состояние или уровень ПО, связанные с функцией, которая использует загрузку ПО.

b. Если после обнаружения загрузки искаженного или несоответствующего ПО происходит восстановление системы до режима по умолчанию или до безопасного состояния, то каждый обособленный компонент системы должен иметь требования в части обеспечения безопасности, указанные для восстановления и работы в этом режиме. Также может потребоваться пересмотреть работу взаимодействующих систем в части правильной работы в режиме по умолчанию.

c. Функции загрузки ПО, включая системы и процедуры поддержки, должны содержать в себе средства обнаружения неправильных комбинаций ПО и (или) аппаратуры и (или) ВС и должны обеспечивать защиту, соответствующую отказному состоянию функции. Если ПО состоит из нескольких единиц конфигурации, необходимо гарантировать их совместимость.

d. Если ПО является частью бортового устройства отображения, которое позволяет подтвердить, что воздушное судно соответствует сертифицированной конфигурации, то либо данное ПО необходимо разрабатывать по уровню ПО, соответствующему самому высокому уровню загружаемого ПО, либо

процесс оценки безопасности системы должен подтверждать целостность полной проверки идентификации конфигурации ПО.

2.5.6. Аспекты, связанные с ПО, при верификации системы

Инструктивные материалы по верификации системы выходят за рамки настоящего документа. Однако процессы ЖЦ ПО помогают и взаимодействуют с процессом верификации системы и могут достичь некоторых целей процесса верификации системы. Для помощи в верификации системы необходимо предоставить детали проекта ПО, связанные с функциональностью системы.

2.6. Системные аспекты в процессах ЖЦ ПО

Можно получить зачет от процессов ЖЦ системы в части достижения или частичного достижения целей ПО, определенных в настоящем документе. В таких случаях необходимо показать, что мероприятия процессов ЖЦ системы, от которых требуется получить зачет, достигают применимых целей настоящего документа с подтверждением завершения запланированных мероприятий, а их выходные данные необходимо идентифицировать как часть данных ЖЦ ПО.

3.0. ЖИЗНЕННЫЙ ЦИКЛ ПО

В настоящем разделе рассматриваются процессы ЖЦ ПО, определение ЖЦ ПО и критерии перехода между процессами ЖЦ ПО. Настоящий документ не предписывает предпочтительные модели ЖЦ ПО и взаимодействие между ними. Разделение процессов не означает, что оно отражает структуру организации (организаций), выполняющей их. Для каждого программного продукта планируется (планируются) ЖЦ ПО, включающий (включающие) эти процессы.

3.1. Процессы жизненного цикла ПО

Процессами ЖЦ ПО являются:

a. Процесс планирования ПО, который определяет и координирует мероприятия процессов разработки ПО и интегральных процессов в рамках проекта. Процесс планирования ПО описан в разделе 4.

b. Процессы разработки ПО, в результате выполнения которых получается программный продукт. Эти процессы включают в себя процесс разработки требований к ПО, процесс проектирования ПО, процесс кодирования ПО, процесс интеграции. Процессы разработки ПО описаны в разделе 5.

c. Интегральные процессы, которые гарантируют корректность, управляемость и доверие к процессам ЖЦ ПО и их выходным данным. Интегральные процессы включают в себя процесс верификации ПО, процесс управления конфигурацией ПО, процесс гарантии качества ПО, процесс взаимодействия заявителя с сертифицирующим органом. Важно понимать, что эти интегральные процессы выполняются одновременно с процессами планирования и разработки ПО в течение всего ЖЦ ПО. Описание интегральных процессов дается в разделах 6–9.

3.2. Определение жизненного цикла ПО

Проект определяет один или более ЖЦ ПО посредством выбора мероприятий для каждого из процессов, установления последовательности мероприятий и назначения ответственных за эти мероприятия.

В конкретном проекте последовательность этих процессов определяется свойствами проекта, такими как функциональность и сложность системы, объем и сложность ПО, стабильность требований, использование ранее разработанного ПО, стратегии разработки и доступность аппаратуры. Обычная последовательность

процессов разработки ПО следующая: разработка требований, проектирование, кодирование и интеграция. На Рисунке 3-1 представлена последовательность процессов разработки ПО для нескольких компонентов одного программного продукта, имеющих различные ЖЦ. Компонент W реализует набор требований к системе посредством разработки требований к ПО, определения на основе этих требований проекта ПО, воплощения этого проекта в Исходном Коде и затем интеграции этого компонента с аппаратурой. Компонент X иллюстрирует применение ранее разработанного ПО, использованного в сертифицированном изделии. Компонент Y иллюстрирует применение простой обособленной функции, для которой Исходный Код можно написать прямо на основании требований к ПО. Компонент Z иллюстрирует стратегию прототипирования. Обычно прототип создается для лучшего понимания требований к ПО и для смягчения рисков разработки и технических рисков. Исходные требования используются как базис для реализации прототипа. Этот прототип оценивается в типовом окружении предполагаемого применения разрабатываемой системы. Результаты оценки используются для уточнения требований.

Процессы ЖЦ ПО могут иметь итерационный характер, то есть, могут неоднократно повторяться. Время выполнения и количество итераций изменяются в зависимости от степени наращивания функций в процессе разработки, сложности, степени уточнения требований, наличия аппаратуры, обратной связи в предшествующие процессы и от других особенностей проекта.

Различные части выбранного ЖЦ ПО связаны между собой сочетанием мероприятий инкрементного процесса интеграции и процесса верификации ПО.

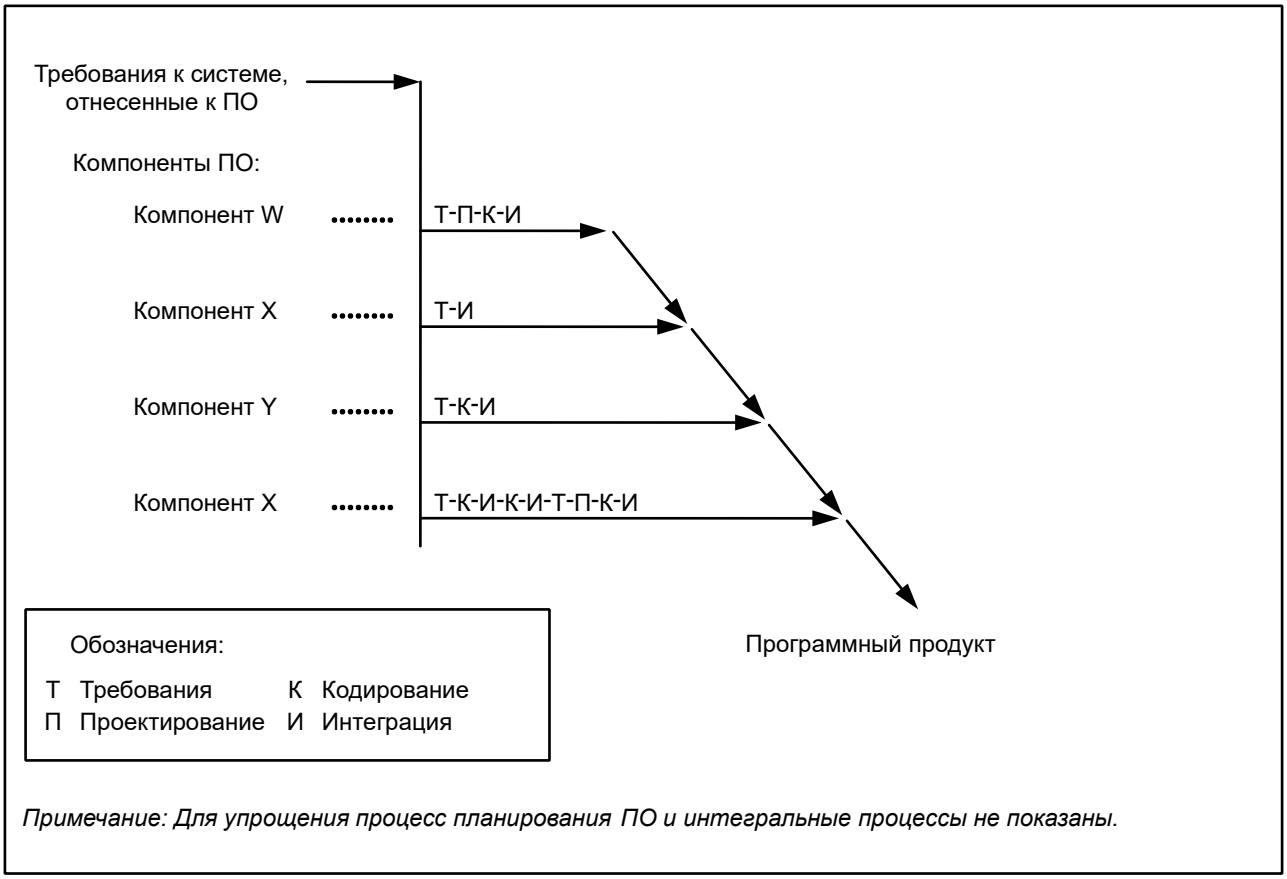


Рисунок 3-1. Пример проекта, использующего четыре различные последовательности разработки

3.3. Критерии перехода между процессами

Критерии перехода используются для того, чтобы определить, можно ли начинать или повторно начинать процесс. Каждый процесс ЖЦ ПО выполняет мероприятия над входными данными для того, чтобы получить

выходные данные. Процесс может порождать обратную связь для других процессов, а также получать обратные связи из других процессов. Определение обратной связи включает в себя решение вопросов, каким образом информация распознается, контролируется и используется для принятия решения принимающим процессом. Примером обратной связи является регистрация проблем.

Критерии перехода зависят от запланированной последовательности процессов разработки ПО и интегральных процессов, на них также может влиять уровень ПО. Примерами выбираемых критериев перехода могут быть: рассмотрения процесса верификации завершены, входные данные являются идентифицированными единицами конфигурации и анализ трассируемости для входного объекта завершен.

Если удовлетворен установленный для процесса критерий перехода, то для начала процесса не требуется, чтобы были готовы все входные данные.

Если для некоторого процесса входные данные поступают по частям, то эти данные необходимо проверять, чтобы убедиться, что они удовлетворяют критериям перехода. Кроме того, при поступлении новых входных данных их необходимо проверить, чтобы убедиться, остались ли в силе ранее полученные выходные данные процессов разработки и верификации ПО.

4.0. ПРОЦЕСС ПЛАНИРОВАНИЯ ПО

В данном разделе описываются цели и мероприятия процесса планирования ПО. Результатом процесса планирования являются планы и стандарты создания ПО, в соответствии с которыми выполняются процессы разработки ПО и интегральные процессы. В Таблице А-1 Приложения А представлена сводка целей и выходных данных процесса планирования ПО для разных уровней ПО.

4.1. Цели процесса планирования ПО

Назначением процесса планирования является определение средств создания ПО, с тем чтобы оно удовлетворяло предъявляемым к нему требованиям и обеспечивало уровень доверия, соответствующий уровню ПО. Цели процесса планирования ПО следующие:

a. Определены мероприятия процессов разработки ПО и интегральных процессов ЖЦ ПО, которые учитывают требования к системе и уровень (-ни) ПО (см. раздел 4.2).

b. Определены ЖЦ ПО с указанием взаимосвязей между процессами, последовательности их выполнения, механизмов обратных связей и критериев перехода (см. раздел 3).

c. Выбрана и определена среда ЖЦ ПО, включая методы и инструменты, которые будут использоваться при выполнении мероприятий каждого процесса ЖЦ ПО (см. раздел 4.4).

d. Учтены, при необходимости, дополнительные аспекты, подобные приведенным в разделе 12.

e. Определены стандарты на разработку ПО, соответствующие целям обеспечения безопасности системы для создаваемого ПО (см. раздел 4.5).

f. Разработаны планы создания ПО, соответствующие разделам 4.3 и 11.

g. Разработка и пересмотр планов создания ПО скоординированы (см. раздел 4.3).

4.2. Мероприятия процесса планировании ПО

Для создания ПО, соответствующего инструктивным материалам настоящего документа, определяющим фактором является эффективное планирование. Мероприятия для процесса планирования ПО включают:

a. Планы создания ПО должны быть разработаны таким образом, чтобы они служили руководством для персонала, выполняющего процессы ЖЦ ПО. См. также раздел 9.1.

b. Должны быть определены или выбраны используемые в проекте стандарты на разработку ПО.

c. Должны быть выбраны методы и инструменты, помогающие предотвращать ошибки и обеспечивающие обнаружение ошибок в процессах разработки ПО.

d. Процесс планирования ПО должен обеспечить координацию процессов разработки ПО и интегральных процессов так, чтобы достичь согласованности стратегий в планах создания ПО.

e. Должны быть указаны средства пересмотра планов создания ПО по мере продвижения проекта.

f. Если в системе используется многоверсионное разнородное ПО, то в процессе планирования ПО должны быть определены методы и инструменты, позволяющие достичь разнородности, которая необходима для достижения целей обеспечения безопасности системы.

g. Чтобы процесс планирования ПО считать завершенным, планы и стандарты создания ПО должны находиться под управлением изменениями, и должны быть завершены их рассмотрения (раздел 4.6).

h. Если предусматривается наличие отключенного кода, то процесс планирования ПО должен описывать, как механизм отключения и отключенный код будут определяться и верифицироваться для достижения целей обеспечения безопасности системы.

i. Если предусматривается разработка модифицируемого пользователем ПО, то в планах и в стандартах создания ПО должны быть указаны процессы, инструменты, среда и данные, обосновывающие такое проектное решение (раздел 5.2.3).

j. Если планируется создание компонентов параметрических данных, необходимо учитывать следующее:

    1. Способ использования компонентов параметрических данных.

    2. Уровень ПО для компонентов параметрических данных.

    3. Процессы, необходимые для разработки, верификации и изменения компонентов параметрических данных, и квалификация соответствующих инструментов.

    4. Контроль загрузки ПО и совместимость.

k. Процесс планирования ПО должен учитывать любые применимые дополнительные аспекты.

l. Если мероприятия по разработке ПО будут выполняться поставщиком, процесс планирования должен предусматривать надзор за поставщиком.

Другие процессы ЖЦ ПО могут начинаться до завершения процесса планирования ПО, если удовлетворены критерии перехода для конкретного мероприятия процесса.

4.3. Планы создания ПО

Планы создания ПО определяют средства для достижения целей настоящего документа. Планы определяют также организации, которые будут выполнять указанные в планах мероприятия. Планами создания ПО являются:

- План Сертификации ПО (см. раздел 11.1) служит в качестве основного средства, с помощью которого предлагаемые методы разработки ПО представляются сертифицирующему органу для согласования, и определяет методы подтверждения соответствия настоящему документу;
- План Разработки ПО (см. раздел 11.2) определяет ЖЦ ПО, среду разработки ПО, а также средства, с помощью которых будут достигаться цели процесса разработки ПО.
- План Верификации ПО (см. раздел 11.3) определяет средства, с помощью которых будут достигаться цели процесса верификации ПО.

- План Управления Конфигурацией ПО (см. раздел 11.4) определяет средства, с помощью которых будут достигаться цели процесса управления конфигурацией ПО.
- План Гарантии Качества ПО (см. раздел 11.5) определяет средства, с помощью которых будут достигаться цели процесса гарантии качества ПО.

Мероприятия для планов создания ПО включают в себя:

a. Планы должны соответствовать настоящему документу.

b. В планах создания ПО должны быть определены критерии перехода между процессами ЖЦ ПО, указывая:

    1. Входные данные процесса, включая обратные связи от других процессов.

    2. Любые мероприятия интегральных процессов, которые могут потребоваться для работы с этими входными данными.

    3. Доступность инструментов, методов, планов и процедур.

c. В планах создания ПО должны быть установлены процедуры внесения изменений в ПО до его применения на сертифицированном изделии. Такие изменения могут быть результатами обратных связей от других процессов и они могут приводить к необходимости внесения изменений в планы создания ПО.

4.4. Планирование среды ЖЦ ПО

Планирование среды ЖЦ ПО определяет методы, инструменты, процедуры, языки программирования и аппаратуру, которые будут использоваться для разработки, верификации, контроля и создания данных ЖЦ ПО (см. раздел 11) и программного продукта. Примерами того, как выбранная среда может положительно повлиять на ПО, являются обеспечение применения стандартов, обнаружение ошибок, а также реализация методов предотвращения ошибок и обеспечения отказоустойчивости. Среда ЖЦ ПО является потенциальным источником ошибок, которые могут привести к отказным состояниям. На состав данной среды ЖЦ ПО могут оказывать влияние требования к безопасности, установленные процессом оценки безопасности системы, например требование применения разнородных резервированных компонентов.

Цель методов предотвращения ошибок – избежать ошибок в процессах разработки ПО, которые могут способствовать возникновению отказных состояний. Основным принципом при этом является выбор методов разработки требований и методов проектирования, инструментов и языков программирования, ограничивающих возможность внесения ошибок, а также выбор методов верификации, гарантирующих обнаружение внесенных ошибок. Целью методов обеспечения отказоустойчивости является внесение в проект ПО или в Исходный Код таких функций обеспечения безопасности, которые гарантируют, что ПО будет правильно реагировать на ошибки входных данных и предотвращать ошибки на выходе и в управлении. Необходимость применения методов предотвращения ошибок или методов обеспечения отказоустойчивости определяется требованиями к системе и процессом оценки безопасности системы.

Изложенные выше аспекты могут оказать влияние на:

a. Методы и системы обозначений, используемые в процессе разработки требований к ПО и в процессе проектирования ПО.

b. Язык(и) программирования и методы, используемые в процессе кодирования ПО.

c. Инструменты среды разработки ПО.

d. Инструменты верификации ПО и управления конфигурацией ПО.

e. Необходимость квалификации инструментов (см. раздел 12.2).

4.4.1. Среда разработки ПО

Среда разработки ПО является важным фактором создания ПО высокого качества. Но среда разработки ПО может оказывать и неблагоприятное влияние на изготовление ПО. Например, компилятор может вносить ошибки, создавая искаженные выходные данные, или линкер может не обнаружить ошибку, допущенную при распределении памяти. Мероприятия по выбору методов и инструментов среды разработки ПО включают в себя:

a. В процессе планирования ПО среда разработки ПО должна быть выбрана таким образом, чтобы уменьшить риск ее неблагоприятного влияния на разрабатываемое ПО.

b. Должен быть выбран такой порядок использования инструментов или комбинации инструментов и частей среды разработки ПО, чтобы достигался необходимый уровень уверенности в том, что ошибка, внесенная в одной части среды, будет обнаружена другой ее части. Приемлемая среда формируется тогда, когда обе части используются совместно и согласованно. Такой выбор включает в себя оценку необходимости квалификации инструментов.

c. Чтобы уменьшить количество ошибок, связанных со средой разработки ПО, необходимо определить мероприятия процесса верификации ПО или стандарты на разработку ПО, учитывающие уровень ПО.

d. Если сертификационный зачет запрашивается для сочетания инструментов, то последовательность их применения должна быть определена в соответствующем плане.

e. Если в проекте предусматривается использование опциональных функций программных инструментов, то необходимо изучить и описать в соответствующем плане влияние этих опций. Это особенно важно для компиляторов и генераторов автокода.

f. Должны быть оценены известные проблемы и ограничения инструментов, и должны быть учтены те из них, которые могут отрицательно повлиять на бортовое ПО.

4.4.2. Аспекты, связанные с языками программирования и компиляторами

После успешного завершения верификации программного продукта компилятор считается пригодным для создания этого продукта. Чтобы это утверждение было справедливым, в процессе верификации ПО должны рассматриваться характерные особенности языков программирования и компилятора. Процесс планирования ПО рассматривает эти особенности при выборе языка программирования и планировании верификации. Мероприятия включают в себя:

a. Некоторые компиляторы способны оптимизировать характеристики объектного кода. Если тестовые примеры дают покрытие, соответствующее уровню ПО, то нет необходимости проверять корректность оптимизации. В противном случае необходимо оценить влияние оптимизации на анализ структурного покрытия. Дополнительную информацию можно найти в Разделе 6.4.4.2.

b. Для реализации определенных возможностей компиляторы ряда языков могут создавать объектный код, не трассируемый непосредственно на Исходный Код, включающий, например, инициализацию, встроенные средства обнаружения ошибок, обработчики исключений (см. раздел 6.4.4.2.b). В процессе планирования ПО должны быть предусмотрены средства обнаружения этого объектного кода и обеспечения верификационного покрытия; эти средства должны быть определены в соответствующем плане.

c. Если в течение ЖЦ ПО вводятся новые версии компилятора, линкера или загрузчика, или изменяются опции компилятора, то полученные ранее тесты и анализы покрытия больше не могут считаться

достоверными. При планировании верификации должны быть предусмотрены средства повторной верификации в соответствии с рекомендациями, изложенными в разделах 6 и 12.1.3.

Примечание: Хотя компилятор считается приемлемым после того, как будут достигнуты все цели верификации, такой компилятор считается приемлемым только по отношению к данному продукту и не обязательно для других продуктов.

4.4.3. Среда тестирования ПО

Планирование среды тестирования определяет методы, инструменты, процедуры и аппаратуру, которые будут использоваться для тестирования выходных данных процесса интеграции. Тестирование может проводиться с использованием целевого вычислителя, эмулятора целевого вычислителя или симулятора, работающего на инструментальном компьютере. Мероприятия включают в себя:

a. Может потребоваться квалификация эмулятора или симулятора в соответствии с рекомендациями раздела 12.2.

b. Должны быть оценены различия между целевым вычислителем и эмулятором или симулятором, а также влияние этих различий на способность обнаруживать ошибки и выполнять проверку функциональности. Обнаружение таких ошибок должен обеспечивать процесс верификации ПО, и это необходимо указывать в Плане Верификации ПО.

4.5. Стандарты на разработку ПО

Стандарты на разработку ПО устанавливают правила и ограничения для процессов разработки ПО. В состав этих стандартов входят Стандарты на Разработку Требований к ПО, Стандарты на Проектирование ПО и Стандарты на Кодирование ПО. В процессе верификации ПО указанные стандарты используются в качестве основы для оценки соответствия полученных выходных данных некоторого процесса ожидаемым выходным данным. Мероприятия по разработке стандартов на разработку ПО включают в себя:

a. Стандарты на разработку ПО должны удовлетворять требованиям раздела 11.

b. Стандарты на разработку ПО должны обеспечивать единообразие проектирования и реализации компонентов данного программного продукта или семейства продуктов.

c. Стандарты на разработку ПО должны запрещать использование конструкций или методов, результаты которых не могут быть верифицированы или которые несовместимы с требованиями безопасности.

d. В стандартах на разработку ПО необходимо рассмотреть вопросы, связанные с обеспечением устойчивости к ошибкам (робастности).

Примечание 1: При разработке стандартов может быть учтен предыдущий опыт. Для контроля сложности ПО в них могут быть включены ограничения и правила, относящиеся к методам разработки, проектирования и кодирования. Для улучшения робастности могут быть рассмотрены приемы «защитного программирования».

Примечание 2: Для компенсации последствий одиночного сбоя могут использоваться приемы обнаружения и контроля ошибок в сохраняемых данных, обновление и непрерывный контроль состояния и конфигурации аппаратуры, если эти требования к системе отнесены к ПО.

4.6. Рассмотрение процесса планирования ПО

Рассмотрения процесса планирования ПО выполняются для того, чтобы гарантировать, что планы создания ПО и стандарты на разработку ПО соответствуют инструктивным материалам настоящего документа и что представлены средства для их выполнения. Мероприятия включают в себя:

a. Выбор методов, которые обеспечивают достижение целей настоящего документа.

b. Процессы ЖЦ ПО могут применяться согласованно.

c. Каждый процесс представляет свидетельства того, что его выходные данные могут быть трассированы на его мероприятия и входные данные, показывая степень независимости мероприятия, используемые среду и методы.

d. Выходные данные процесса планирования ПО являются непротиворечивыми и соответствуют требованиям раздела 11.

5.0. ПРОЦЕССЫ РАЗРАБОТКИ ПО

В настоящем разделе рассматриваются цели и мероприятия процессов разработки ПО. Эти процессы применяются так, как это определено процессом планирования ПО (см. раздел 4) и Планом Разработки ПО (см. раздел 11.2). В Таблице А-2 Приложения А представлена итоговая сводка целей и выходных данных процессов разработки ПО для разных уровней ПО. Процессами разработки ПО являются:

- Процесс разработки требований к ПО
- Процесс проектирования ПО
- Процесс кодирования ПО
- Процесс интеграции

В процессах разработки ПО формулируются один или более уровней требований к ПО. Требования высокого уровня получаются непосредственно из анализа требований к системе и архитектуры системы. Обычно эти требования высокого уровня перерабатываются в процессе проектирования ПО, порождая таким образом требования одного или более последовательных низких уровней. Однако если Исходный Код создается непосредственно по требованиям высокого уровня, то эти требования высокого уровня рассматриваются так же, как требования низкого уровня, и к ним применим инструктивный материал, относящийся к требованиям низкого уровня.

Примечание: При разработке ПО по требованиям только одного уровня от Заявителя может потребоваться обоснование процессов разработки.

Разработка архитектуры ПО связана с решениями, принятыми относительно структуры ПО. В процессе проектирования ПО определяется архитектура ПО и формулируются требования низкого уровня. Требования низкого уровня представляют собой требования к ПО, по которым непосредственно, без дополнительной информации, можно написать Исходный Код.

Каждый процесс разработки ПО может порождать производные требования. Примерами требований, которые могут быть определены, как производные, являются:

- Необходимость разработки программы обработки прерываний для выбранного целевого вычислителя.
- Задание частоты периодического мониторинга, когда она не определена требованиями к системе, отнесенными к ПО.
- Дополнительные ограничения на масштаб при использовании арифметики с фиксированной точкой.

Производные требования могут присутствовать как в требованиях высокого уровня, так и в требованиях низкого уровня. Чтобы определить влияние производных требований на оценку безопасности системы и на требования к системе, все производные требования должны быть доступными для процессов системы, включая процесс оценки безопасности системы.

5.1. Процесс разработки требований к ПО

Для составления требований высокого уровня процесс разработки требований к ПО использует выходные данные процессов ЖЦ системы. Эти требования высокого уровня включают требования к функциональности, техническим характеристикам, интерфейсу и безопасности.

5.1.1. Цели процесса разработки требований к ПО

a. Цели процесса разработки требований к ПО следующие: a. Требования высокого уровня разработаны.

b. Производные требования высокого уровня определены и переданы в процессы системы, включая процесс оценки безопасности системы.

5.1.2. Мероприятия процесса разработки требований к ПО

Входные данные для процесса разработки требований к ПО включают: требования к системе, интерфейс с аппаратурой и архитектуру системы (если не включено в требования к системе) из процессов ЖЦ системы, а также План Разработки ПО и Стандарты на Разработку Требований к ПО из процесса планирования ПО. После выполнения запланированных критериев перехода эти входные данные используются для разработки требований высокого уровня.

Основным результатом этого процесса являются Требования к ПО (см. раздел 11.9).

Процесс разработки требований к ПО считается завершенным, когда его цели достигнуты и достигнуты цели связанных с ним интегральных процессов. Мероприятия этого процесса следующие:

a. Должны быть проанализированы отнесенные к ПО требования к функциональности и интерфейсу системы на отсутствие неясностей, противоречий и неопределенных условий.

b. Входные данные для процесса разработки требований к ПО, признанные недостаточными или неверными, должны быть переданы по обратной связи на входы соответствующих процессовисточников входных данных для уточнения или поправок.

c. Каждое требование к системе, отнесенное к ПО, должно быть включено в требования высокого уровня.

d. Должны быть определены те требования высокого уровня, которые связаны с требованиями к системе, отнесенными к ПО и введенными для предотвращения опасных состояний в системе.

e. Требования высокого уровня должны соответствовать Стандартам на Разработку Требований к ПО, быть верифицируемыми и непротиворечивыми.

f. Требования высокого уровня должны быть установлены в количественной форме с указанием допусков там, где это применимо.

g. В требованиях высокого уровня не должно быть описания деталей реализации и верификации, за исключением установленных и обоснованных проектных ограничений.

h. Должны быть определены производные требования высокого уровня и причины их создания.

i. Производные требования высокого уровня должны быть переданы в процессы системы, включая процесс оценки безопасности системы.

j. Если планируется использование компонентов параметрических данных, то в требованиях высокого уровня должно быть описано, как в ПО используется каждый компонент параметрических данных. В требованиях высокого уровня должна быть также указана их структура, атрибуты для каждого элемента данных, а также, когда это применимо, значение каждого элемента данных. Значения элементов компонентов параметрических данных должны быть совместимы со структурой компонента параметрических данных и атрибутами элементов этих данных.

5.2. Процесс проектирования ПО

Требования высокого уровня уточняются за одну или более итераций в процессе проектирования ПО, чтобы разработать архитектуру ПО и требования низкого уровня, которые можно использовать для создания Исходного Кода.

5.2.1. Цели процесса проектирования ПО

Целями процесса проектирования ПО являются:

a. Архитектура ПО и требования низкого уровня разработаны на основе требований высокого уровня.

b. Производные требования низкого уровня определены и переданы в процессы системы, включая процесс оценки безопасности системы.

5.2.2. Мероприятия процесса проектирования ПО

Входными данными для процесса проектирования ПО являются Требования к ПО, План Разработки ПО и Стандарты на Проектирование ПО. После выполнения запланированных критериев перехода требования высокого уровня используются в процессе проектирования для разработки архитектуры ПО и требований низкого уровня. При этом могут быть сформулированы требования одного или нескольких более низких уровней.

Главным результатом этого процесса является Описание Проекта ПО (см. раздел 11.10), в котором дается описание архитектуры ПО и формулируются требования низкого уровня.

Процесс проектирования ПО считается завершенным, когда его цели достигнуты и достигнуты связанные с ним цели интегральных процессов. Мероприятия этого процесса включают в себя:

a. Требования низкого уровня и архитектура ПО, разработанные в процессе проектирования ПО, должны соответствовать Стандартам на Проектирование ПО, быть трассируемыми, верифицируемыми и непротиворечивыми.

b. Необходимо определить производные требования низкого уровня и причины их создания и проанализировать их, чтобы убедиться в том, что требования более высокого уровня не нарушены.

c. Мероприятия процесса проектирования ПО могут привнести возможные виды отказов в ПО или, наоборот, устранить другие виды. Использование обособления или другого архитектурного решения в проекте ПО может изменить присвоенный уровень для некоторых компонентов ПО. В таких случаях необходимо определить дополнительные данные в виде производных требований и передать их в процессы системы, включая процесс оценки безопасности системы.

d. Должны быть определены интерфейсы между компонентами ПО в форме потока данных и потока управления таким образом, чтобы они были согласованными между компонентами.

e. Должен быть предусмотрен контроль потоков управления и данных, например, с помощью сторожевых таймеров, проверок на допустимость и перекрестного сравнения каналов, когда это диктуется требованиями в части обеспечения безопасности.

f. Реакции на отказные состояния должны удовлетворять требованиям в части обеспечения безопасности.

g. Недостаточные или некорректные входные данные, выявленные в процессе проектирования ПО, должны быть переданы в процессы ЖЦ системы, в процесс разработки требований к ПО или в процесс планирования ПО по обратной связи для уточнения или поправок.

Примечание: Современный уровень технологии программирования не позволяет установить количественную связь между сложностью ПО и достижением целей безопасности системы. И, хотя нет возможности дать объективный инструктивный материал по этому вопросу, в процессе проектирования ПО необходимо избегать его усложнения, поскольку по мере возрастания сложности становится труднее верифицировать проект ПО и демонстрировать, что удовлетворены требования в части обеспечения безопасности.

5.2.3. Проектирование модифицируемого пользователем ПО

Модифицируемое пользователем ПО разрабатывается таким образом, чтобы его мог изменять пользователь. Модифицируемый компонент – это такая часть ПО, которая предназначена для внесения в нее изменений пользователем, а немодифицируемый – та, которая не предназначена для модификации пользователем. Сложность модифицируемого пользователем ПО может быть разной. Примерами являются: отдельный бит памяти, используемый для выбора одной из двух опций оборудования, таблица сообщений или область памяти, которую можно запрограммировать, откомпилировать и слинковать для реализации функций технического обслуживания. Модифицируемые компоненты могут содержаться в ПО любого уровня.

Мероприятия для модифицируемого пользователем ПО включают в себя:

a. Немодифицируемые компоненты должны быть защищенными от влияния модифицируемых для предотвращения неблагоприятного воздействия на безопасную работу немодифицируемых компонентов. Эта защита может быть реализована аппаратурой, ПО, инструментами, используемыми для внесения изменений, или комбинацией трех этих средств. Если такая защита обеспечивается ПО, она должна быть спроектирована и верифицирована на том же уровне ПО, что и немодифицируемое ПО. Если такая защита обеспечивается инструментом, то ему должен быть присвоен соответствующий уровень квалификации, и он должен быть квалифицирован так, как определено в разделе 12.2.

b. Необходимо показать, что средства, предусмотренные для изменения модифицируемого компонента, являются единственными средствами, с помощью которых можно изменять модифицируемый компонент.

5.2.4. Проектирование при наличии отключенного кода

Система или оборудование могут быть спроектированы так, чтобы иметь несколько конфигураций, не все из которых предназначены для использования при каждом применении. Это может привести к появлению отключенного кода, который не может быть выполнен, например, невыбранная функциональность или

неиспользуемые библиотечные функции или неиспользуемые данные. Отключенный код отличается от мертвого кода. Мероприятия для отключенного кода включают в себя:

a. Необходимо спроектировать и реализовать механизм, гарантирующий, что отключаемые функции или компоненты не оказывают негативного влияния на активные функции или компоненты.

b. Необходимо предоставить свидетельства того, что отключенный код блокируется для условий эксплуатации, в которых его использование не предусмотрено. Непреднамеренное исполнение отключенного кода из-за ненормальных состояний системы представляет собой то же самое, что и непреднамеренное исполнение активированного кода.

c. Разработка отключенного кода, как и активного кода, должна соответствовать целям настоящего документа.

5.3. Процесс кодирования ПО

В процессе кодирования ПО Исходный код реализуется на основе архитектуры ПО и требований низкого уровня.

Примечание: В рамках настоящего документа компиляция, линковка и загрузка рассматриваются в Процессе интеграции (см. раздел 5.4).

5.3.1. Цели процесса кодирования ПО

Цель процесса кодирования следующая:

a. Исходный Код разработан на основании требований низкого уровня.

5.3.2. Мероприятия процесса кодирования ПО

Входными данными процесса кодирования ПО являются требования низкого уровня и архитектура ПО, полученные из процесса проектирования ПО, а также План Разработки ПО и Стандарты на Кодирование ПО. К процессу кодирования можно перейти или повторно перейти после того, как удовлетворены запланированные критерии перехода. Исходный Код создается этим процессом на основе архитектуры ПО и требований низкого уровня.

Основным результатом этого процесса является Исходный Код (см. раздел 11.11).

Процесс кодирования считается завершенным, когда его цели достигнуты и достигнуты связанные с ним цели интегральных процессов. Мероприятия этого процесса включают в себя:

a. Исходный Код должен реализовывать требования низкого уровня и соответствовать архитектуре ПО.

b. Исходный Код должен соответствовать Стандартам на Кодирование ПО.

c. Недостаточные или некорректные входные данные, выявленные в процессе кодирования ПО, должны быть переданы в процесс разработки требований к ПО, процесс проектирования ПО или в процесс планирования ПО по обратной связи для уточнения или поправок.

d. Генераторы автокода должны использоваться в соответствии с ограничениями, определенными в процессе планирования.

5.4. Процесс интеграции

В процессе интеграции используются целевой вычислитель и Исходный Код из процесса кодирования ПО, а также данные для компиляции, линковки и загрузки (см. раздел 11.16) для получения интегрированной системы или оборудования.

5.4.1. Цели процесса интеграции

Цель процесса интеграции следующая:

a. Исполняемый Объектный Код и соответствующие ему Файлы Компонентов Параметрических Данных, если таковые имеются, созданы и загружены в целевой вычислитель для интеграции ПО и аппаратуры.

5.4.2. Мероприятия процесса интеграции

Процесс интеграции состоит из интеграции ПО и интеграции аппаратуры и ПО.

К процессу интеграции можно перейти или повторно перейти после того, как удовлетворены запланированные критерии перехода. Входными данными процесса интеграции являются архитектура ПО, полученная из процесса проектирования ПО, а также Исходный Код из процесса кодирования ПО.

Выходными данными процесса интеграции являются: объектный код, Исполняемый Объектный Код (см. раздел 11.12), Файл Компонента Параметрических Данных (см. раздел 11.22) и данные для компиляции, линковки и загрузки. Процесс интеграции считается завершенным, когда его цели достигнуты и достигнуты связанные с ним цели интегральных процессов. Мероприятия этого процесса включают в себя:

a. Объектный код и Исполняемый Объектный Код должны быть получены из исходного кода и данных для компиляции, линковки и загрузки. Должны быть созданы все Файлы Компонентов Параметрических Данных.

b. Интеграция ПО должна быть выполнена на инструментальном компьютере, эмуляторе целевого вычислителя или на целевом вычислителе.

c. ПО должно быть загружено в целевой вычислитель для интеграции аппаратуры и ПО.

d. Недостаточные или некорректные входные данные, выявленные в процессе интеграции, должны быть переданы в процесс разработки требований к ПО, в процесс проектирования ПО, в процесс кодирования или в процесс планирования ПО по обратной связи для уточнения или поправок.

e. В ПО, заявляемом для применения в сертифицированном изделии, не следует использовать «заплаты» для реализации изменений требований или архитектуры, или изменений, необходимость которых выявлена в результате проведения мероприятий процесса верификации ПО. «Заплаты» могут использоваться ограниченно, для каждого отдельного случая, например, для устранения известных недостатков в среде разработки ПО, таких как известная проблема компилятора.

f. При использовании «заплаты» должно быть предоставлено:

    1. Подтверждение того, что процесс управления конфигурацией ПО может эффективно отследить данную «заплату».

    2. Анализ, обеспечивающий подтверждение того, что ПО с «заплатой» достигает всех применимых целей.

    3. Обоснование использования «заплаты» в Итоговом Заключении о ПО.

5.5. Трассируемость процесса разработки ПО

Мероприятия трассируемости процесса разработки ПО включают:

a. Разработку Данных Трассировки, демонстрирующих двустороннюю связь между требованиями к системе, отнесенными к ПО, и требованиями высокого уровня. Назначением этих Данных Трассировки является:

    1. Обеспечить возможность верификации полноты реализации требований к системе, отнесенных к ПО.
    2. Наглядно показать те производные требования высокого уровня, которые непосредственно не трассируются на системные требования.

b. Разработку Данных Трассировки, демонстрирующих двустороннюю связь между требованиями высокого уровня и требованиями низкого уровня. Назначением этих Данных Трассировки является:

   1. Обеспечить возможность верификации полноты реализации требований высокого уровня.
   2. Наглядно показать те производные требования низкого уровня, которые непосредственно не трассируются на требования высокого уровня, и архитектурные проектные решения, принятые в процессе проектирования ПО.

c. Разработку Данных Трассировки, демонстрирующих двустороннюю связь между требованиями низкого уровня и Исходным Кодом. Назначением этих Данных Трассировки является:

   1. Обеспечить возможность верификации того, что нигде в Исходном коде не реализованы недокументированные функции.
   2. Обеспечить возможность верификации полноты реализации требований низкого уровня.

6.0. ПРОЦЕСС ВЕРИФИКАЦИИ ПО

В настоящем разделе рассматриваются цели и мероприятия процесса верификации ПО. Верификация – это техническая оценка выходных данных процесса планирования ПО, процессов разработки ПО и процесса верификации ПО. Процесс верификации ПО выполняется согласно тому, как это было определено процессом планирования ПО (см. раздел 4), и согласно Плану Верификации ПО (см. раздел 11.3). Верификация выходных данных процесса планирования представлена в разделе 4.6.

Верификация – это не просто тестирование. Тестирование, вообще говоря, не может продемонстрировать отсутствие ошибок. Вследствие этого, при рассмотрении мероприятий процесса верификации, которые обычно представляют собой сочетание рассмотрений, анализов и тестов, в последующих подразделах вместо термина «тестировать» используется термин «верифицировать».

В Таблицах с А-3 по А-7 Приложения А приведена итоговая сводка целей и выходных данных процесса верификации ПО с учетом уровней ПО.

Примечание: Для ПО более низкого уровня меньшее внимание уделяется:

- *верификации Исходного Кода;*
- *верификации требований низкого уровня;*
- *верификации архитектуры ПО;*
- *степени покрытия тестами;*
- *контролю процедур верификации;*
- *независимости мероприятий процесса верификации ПО;*
- *перекрывающимся мероприятиям верификации ПО, то есть проведению нескольких верификационных мероприятий, каждое из которых может быть способно выявлять ошибки одного и того же класса;*
- *тестированию робастности;*
- *мероприятиям верификации, косвенно влияющим на обнаружение или предотвращение ошибок, например, подтверждению соответствия стандартам на разработку ПО.*

6.1. Назначение процесса верификации ПО

Назначением процесса верификации ПО является обнаружение и регистрация ошибок, которые, возможно, были привнесены в процессах разработки ПО. Устранение ошибок – мероприятие процессов разработки ПО. Процесс верификации ПО проверяет, что:

a. Требования к системе, отнесенные к ПО, переработаны в требования высокого уровня, которые удовлетворяют этим требованиям к системе.

b. Требования высокого уровня переработаны в архитектуру ПО и требования низкого уровня, которые удовлетворяют требованиям высокого уровня. Если один или более уровней требований к ПО разрабатываются между требованиями высокого уровня и требованиями низкого уровня, последующие уровни требований разработаны так, что каждый последующий более низкий уровень удовлетворяет соответствующим требованиям более высокого уровня. Если код сгенерирован непосредственно из требований высокого уровня, то этот пункт не применяется.

c. Архитектура ПО и требования низкого уровня переработаны в Исходный Код, который удовлетворяет требованиям низкого уровня и архитектуре ПО.

d. Исполняемый Объектный Код удовлетворяет требованиям к ПО (то есть, заданной функции) и обеспечивает уверенность в отсутствии непредусмотренной функциональности.

e. Исполняемый Объектный Код является робастным в отношении требований к ПО так, что он может правильно реагировать на ненормальные входные данные и условия.

f. Средства, использованные для выполнения этой верификации, технически корректны и достаточны для данного уровня ПО.

6.2. Обзор мероприятий процесса верификации ПО

Цели процесса верификации ПО удовлетворяются посредством сочетания рассмотрений, анализов, разработки тестовых примеров и процедур и последующего выполнения этих процедур. Рассмотрения и анализы позволяют оценить точность, полноту и верифицируемость требований к ПО, архитектуры ПО и Исходного Кода. Разработка тестовых примеров и процедур может обеспечить более полную оценку внутренней непротиворечивости и полноты требований. Выполнение тестовых процедур позволяет продемонстрировать соответствие требованиям.

Входные данные процесса верификации ПО включают в себя требования к системе, требования к ПО, архитектуру ПО, Данные Трассировки, Исходный Код, Исполняемый Объектный Код и План Верификации ПО.

Выходные данные процесса верификации ПО фиксируются в виде Примеров и Процедур Верификации ПО (см. раздел 11.13), Результатов Верификации ПО (см. раздел 11.14) и соответствующих Данных Трассировки (см. раздел 11.21).

Необходимость обеспечить верифицируемость требований, реализованных в ПО, может сама по себе наложить дополнительные требования или ограничения на процессы разработки ПО.

Аспекты, связанные с верификацией ПО, включают в себя:

a. Если протестированный код не идентичен ПО, установленному на борт, должны быть определены и обоснованы эти различия.

b. Если некоторые требования к ПО невозможно верифицировать посредством выполнения ПО в реалистичной среде тестирования, должны быть применены другие средства, и обоснование того, что эти средства обеспечивают достижение целей процесса верификации, должно быть приведено в Плане Верификации ПО или Результатах Верификации ПО.

c. Информация о недостатках и ошибках, выявленных в процессе верификации ПО, должна быть передана в соответствующие процессы ЖЦ ПО для внесения уточнений и исправлений.

d. После выполнения корректирующих действий и (или) изменений, которые могли повлиять на ранее верифицированную функциональность, должна быть выполнена повторная верификация. Повторная верификация должна подтвердить, что изменение было реализовано корректно.

e. Независимость верификации достигается, когда мероприятие верификации выполняется лицом (лицами), не являющимися разработчиками проверяемого объекта. Для достижения эквивалентности с верификацией, выполняемой человеком, может быть использован инструмент. Для обеспечения независимости лицо, создающее набор тестовых примеров, основанных на требованиях низкого уровня, не должно быть тем же самым лицом, которое разрабатывает соответствующий Исходный Код по этим требованиям низкого уровня.

6.3. Рассмотрения и анализы ПО

Рассмотрения и анализы проводятся применительно к выходным данным процессов разработки ПО. Одно различие между рассмотрениями и анализами заключается в том, что анализы дают повторно воспроизводимое свидетельство правильности, а рассмотрения – качественную оценку правильности. Рассмотрение может включать в себя проверку выходных данных процесса с помощью проверочного перечня или аналогичного средства. Анализ может заключаться в подробном изучении функциональности, рабочих характеристик, трассируемости и безопасности применения компонента ПО, а также его взаимосвязей с другими компонентами в пределах системы или оборудования.

Могут быть случаи, когда описываемые в настоящем разделе цели верификации не могут быть полностью достигнуты посредством только рассмотрений и анализов. В таких случаях эти цели верификации могут быть достигнуты дополнительным тестированием программного продукта. Например, для определения наихудшего времени исполнения или проверки использования стека можно разработать комбинацию рассмотрений, анализов и тестов.

6.3.1. Рассмотрения и анализы требований высокого уровня

Мероприятия рассмотрений и анализов обнаруживают и регистрируют ошибки в требованиях, которые могли быть допущены в процессе разработки требований к ПО. Эти мероприятия подтверждают, что требования высокого уровня достигают следующих целей:

a. Соответствие требованиям к системе. Цель – подтвердить, что функции системы, выполняемые ПО, определены, требования к функциональности, техническим характеристикам и к обеспечению безопасности системы удовлетворяются требованиями высокого уровня, и что производные требования и причины их создания определены правильно.

b. Точность и непротиворечивость. Цель – подтвердить, что каждое требование высокого уровня точно сформулировано, имеет однозначное толкование, достаточно детализировано, и что требования не противоречат друг другу.

c. Совместимость с целевым вычислителем. Цель – подтвердить отсутствие конфликтов между требованиями высокого уровня и аппаратно-программными характеристиками целевого вычислителя, в особенности в отношении времени реакции системы и аппаратуры ввода-вывода.

d. Верифицируемость. Цель – подтвердить, что каждое требование высокого уровня может быть верифицировано.

e. Соответствие стандартам. Цель – подтвердить, что в процессе разработки требований к ПО соблюдались Стандарты на Разработку Требований к ПО, и что отступления от стандартов обоснованы.

f. Трассируемость. Цель – подтвердить, что системные требования к техническим характеристикам, к функциональности и к обеспечению безопасности, отнесенные к ПО, переработаны в требования высокого уровня.

g. Алгоритмические аспекты. Цель – подтвердить точность и правильное поведение предложенных алгоритмов, особенно вблизи точек разрыва.

6.3.2. Рассмотрения и анализы требований низкого уровня

Мероприятия рассмотрений и анализов обнаруживают и регистрируют ошибки в требованиях, которые могли быть допущены в процессе проектирования ПО. Эти мероприятия подтверждают, что требования низкого уровня достигают следующих целей:

a. Соответствие требованиям высокого уровня. Цель – подтвердить, что требования низкого уровня соответствуют требованиям высокого уровня и что производные требования вместе с проектным обоснованием их создания определены правильно.

b. Точность и непротиворечивость. Цель – подтвердить, что каждое требование низкого уровня точно сформулировано, имеет однозначное толкование и не противоречит другим требованиям низкого уровня.

c. Совместимость с целевым вычислителем. Цель – подтвердить отсутствие конфликтов между требованиями низкого уровня и аппаратно-программными характеристиками целевого вычислителя, в особенности в отношении использования ресурсов, таких, как загрузка шины, время реакции системы и аппаратуры ввода-вывода.

d. Верифицируемость. Цель – подтвердить, что каждое требование низкого уровня может быть верифицировано.

e. Соответствие стандартам. Цель – подтвердить, что в процессе проектирования ПО соблюдались Стандарты на Проектирование ПО, и что отступления от стандартов обоснованы.

f. Трассируемость. Цель – подтвердить, что требования высокого уровня и производные требования переработаны в требования низкого уровня.

g. Алгоритмические аспекты. Цель – подтвердить точность и правильное поведение предложенных алгоритмов, особенно вблизи точек разрыва.

6.3.3. Рассмотрения и анализы архитектуры ПО

Мероприятия рассмотрений и анализов обнаруживают и регистрируют ошибки, которые могли быть допущены в процессе разработки архитектуры ПО. Эти мероприятия подтверждают, что архитектура ПО достигает следующих целей:

a. Совместимость с требованиями высокого уровня. Цель – подтвердить, что архитектура ПО, особенно функции, обеспечивающие целостность системы (например, схемы обособления), не противоречит требованиям высокого уровня.

b. Непротиворечивость. Цель – подтвердить, что между компонентами архитектуры ПО установлена правильная взаимосвязь. Эта взаимосвязь реализуется через поток данных и поток управления. Если имеется интерфейс с компонентом более низкого уровня ПО, должно быть также подтверждено, что компонент более высокого уровня имеет соответствующие встроенные механизмы защиты от поступления потенциальных ошибочных входных данных от компонента с более низким уровнем ПО.

c. Совместимость с целевым вычислителем. Цель – подтвердить отсутствие конфликтов между архитектурой ПО и характеристиками аппаратуры и ПО целевого вычислителя, в особенности, в части инициализации, асинхронной работы, синхронизации и прерываний.

d. Верифицируемость. Цель – подтвердить, что архитектура ПО может быть верифицирована (например, что в ней отсутствуют неограниченные рекурсивные алгоритмы).

e. Соответствие стандартам. Цель – подтвердить, что в процессе проектирования ПО соблюдались положения Стандартов на Проектирование ПО, и что отступления от стандартов обоснованы, например, в части отклонений от ограничений на сложность и от принципов построения структуры проекта.

f. Целостность обособления. Цель – подтвердить, что нарушения обособления предотвращены.

6.3.4. Рассмотрения и анализы Исходного Кода

Мероприятия рассмотрений и анализов обнаруживают и регистрируют ошибки, которые могли быть допущены в процессе кодирования ПО. Основное внимание уделяется корректности кода по отношению к требованиям к ПО и архитектуре ПО, а также соответствию Стандартам на Кодирование ПО. Такие рассмотрения и анализы обычно ограничены Исходным Кодом и подтверждают, что Исходный Код достигают следующих целей:

a. Соответствие требованиям низкого уровня. Цель – подтвердить, что Исходный Код точен и полон по отношению к требованиям низкого уровня, а Исходный Код, реализующий недокументированные функции, отсутствует.

b. Соответствие архитектуре ПО. Цель – подтвердить, что Исходный Код соответствует потоку данных и потоку управления, которые определены в архитектуре ПО.

c. Верифицируемость. Цель – подтвердить, что Исходный Код не содержит операторов и структур, которые не могут быть верифицированы, и для проведения тестирования в него не требуется вносить изменения.

d. Соответствие стандартам. Цель – подтвердить, что разработка кода велась в соответствии со Стандартами на Кодирование ПО, например, в части ограничений на сложность и на конструкции кода. Сложность включает в себя степень связности между компонентами ПО, уровни вложенности управляющих структур, а также сложность логических или численных выражений. Этот анализ также подтверждает, что отступления от стандартов обоснованы.

e. Трассируемость. Цель – подтвердить, что требования низкого уровня переработаны в Исходный Код.

f. Точность и непротиворечивость. Цель – оценить правильность и непротиворечивость Исходного Кода, включая оценку использования стека, использования памяти, переполнения и разрешающей способности арифметики с фиксированной точкой, арифметики с плавающей точкой, конкуренции и ограничений в использовании ресурсов, наихудшего времени исполнения, обработки исключений, использования неинициализированных переменных, управления кэшем, неиспользуемых переменных и искажения данных из-за конфликтов задач или прерываний. Наихудшее время исполнения может зависеть от компилятора (и его опций), линкера (и его опций) и некоторых особенностей аппаратуры, и это влияние должно быть оценено.

6.3.5. Рассмотрения и анализы выходных данных процесса интеграции

Мероприятия рассмотрений и анализов обнаруживают и регистрируют ошибки, которые могли быть допущены в процессе интеграции. Цель следующая:

a. Подтвердить, что результаты процесса интеграции являются полными и корректными.

Мероприятия включают в себя проведение детальной проверки данных компиляции, линковки и загрузки, а также карт распределения памяти. Типичные примеры возможных ошибок включают:

- предупреждающие сообщения компилятора;
- неправильные аппаратные адреса;

- перекрытия областей памяти;
- потерянные компоненты ПО.

6.4. Тестирование ПО

Тестирование ПО используется, чтобы продемонстрировать, что ПО удовлетворяет предъявляемым к нему требованиям, и продемонстрировать с высокой степенью уверенности, что были устранены ошибки, которые могли привести к недопустимым отказным состояниям, как это определено процессом оценки безопасности системы.

Целями тестирования ПО является выполнение ПО для подтверждения того, что:

a. Исполняемый Объектный Код соответствует требованиям высокого уровня.

b. Исполняемый Объектный Код является робастным по отношению к требованиям высокого уровня.

c. Исполняемый Объектный Код соответствует требованиям низкого уровня.

d. Исполняемый Объектный Код является робастным по отношению к требованиям низкого уровня.

e. Исполняемый Объектный Код является совместимым с целевым вычислителем.

На Рисунке 6-1 показана схема мероприятий тестирования ПО, которые могут быть использованы для достижения целей тестирования ПО. На схеме представлены три вида тестирования:

- Тестирование интеграции аппаратуры и ПО: для верификации корректной работы ПО в среде целевого вычислителя.
- Тестирование интеграции ПО: для верификации взаимосвязей между требованиями к ПО и компонентами ПО, а также верификации реализации требований к ПО и компонентов ПО в архитектуре ПО.
- Тестирование низкого уровня: для верификации реализации требований низкого уровня.

Примечание: Если тестовый пример и соответствующая тестовая процедура разработаны и выполнены для тестирования интеграции аппаратуры и ПО или интеграции ПО, и при этом были обеспечены покрытие требований и структурное покрытие, то нет необходимости дублировать эти тесты тестами низкого уровня. Замена номинально эквивалентными тестами низкого уровня тестов высокого уровня может быть менее эффективной вследствие уменьшения полностью протестированной функциональности.

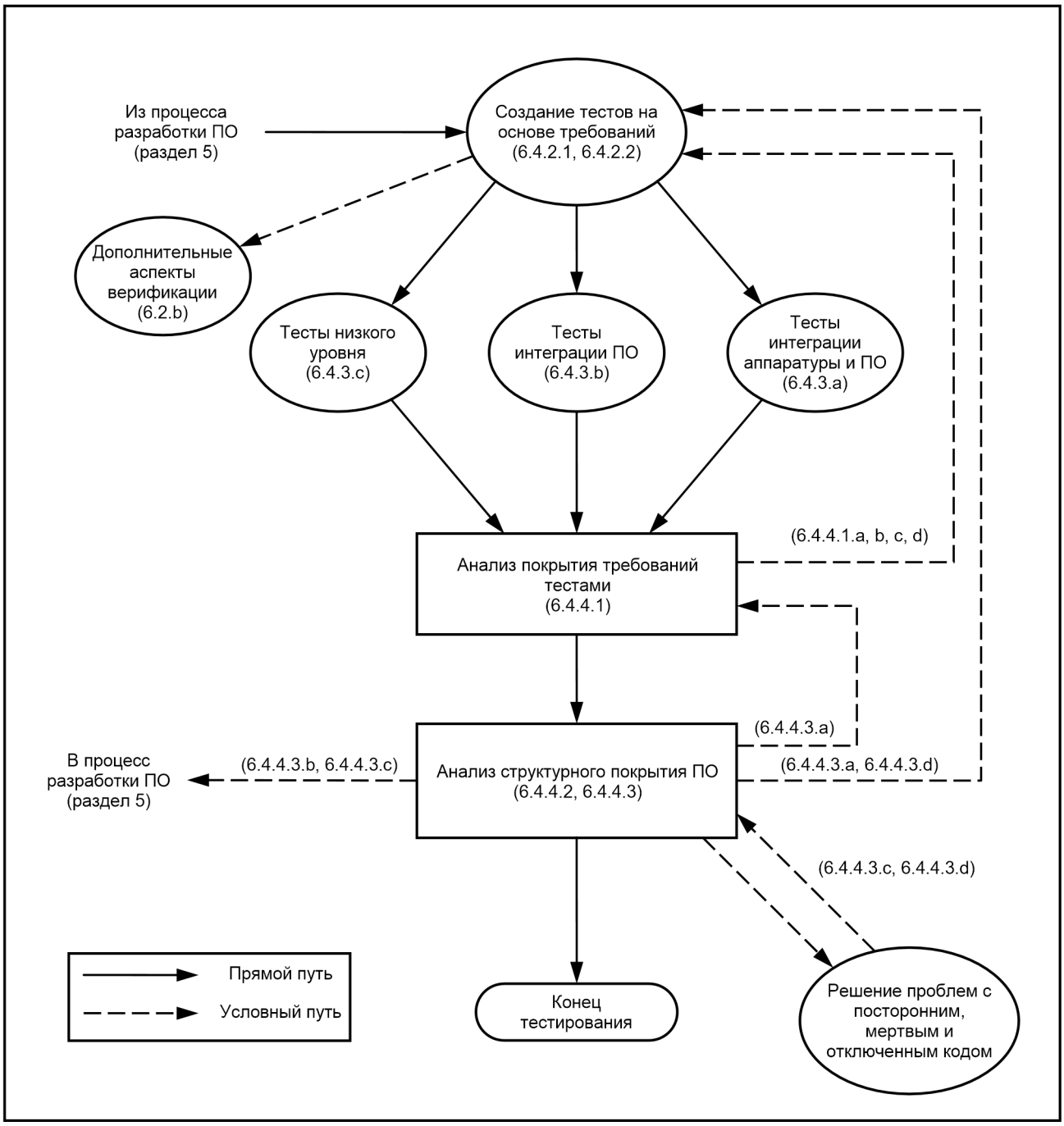


Рисунок 6-1. Мероприятия тестирования ПО

6.4.1. Среда тестирования

Для достижения целей тестирования ПО может потребоваться более, чем одна среда тестирования. Предпочтительная среда тестирования включает в себя ПО, загружаемое в целевой вычислитель и тестируемое в среде, которая максимально воспроизводит поведение среды целевого вычислителя.

Примечание: Во многих случаях необходимое покрытие требований и структурное покрытие могут быть достигнуты только при более точном управлении и мониторинге тестовых входных данных и выполнении кода, чем это обычно возможно в полностью интегрированной среде. Может оказаться необходимым проводить такое тестирование на небольшом компоненте ПО, функционально изолированном от других компонентов ПО.

Сертификационный зачет может быть получен на основании тестирования, проведенного с использованием эмулятора целевого вычислителя или симулятора на инструментальном компьютере. Мероприятия, связанные со средой тестирования, включают в себя:

a. Отдельные тесты должны быть выполнены в интегрированной среде целевого вычислителя, поскольку некоторые ошибки могут быть обнаружены только в этой среде.

6.4.2. Выбор тестов на основе требований

Тестированию, основанному на требованиях, придается повышенное значение, потому что эта стратегия признана наиболее эффективной для выявления ошибок. Мероприятия по выбору тестов на основе требований включают:

1. Должны быть разработаны тестовые примеры, включая тестовые примеры в допустимом диапазоне и тестовые примеры на робастность (вне допустимого диапазона).

2. Эти тестовые примеры должны быть разработаны на основе требований к ПО, принимая во внимание источники ошибок, присущие процессам разработки ПО.

Примечание: Тестовые примеры на робастность основаны на требованиях. Критерии тестов на робастность не могут быть полностью удовлетворены, если в требованиях к ПО не указана корректная реакция ПО на ненормальные состояния и входные данные. Тестовые примеры могут выявить неполноту в требованиях к ПО, в таких случаях требования к ПО должны быть изменены. Наоборот, если существует полный набор требований, который покрывает все ненормальные состояния и входные данные, тестовые примеры на робастность будут основаны на этих требованиях к ПО.

3. Тестовые процедуры сформированы из тестовых примеров.

6.4.2.1. Тестовые примеры в допустимом диапазоне

Тестовые примеры в допустимом диапазоне демонстрируют способность ПО реагировать на нормальные входные данные и условия. Мероприятия включают:

a. Действительные и целые входные переменные должны проверяться с использованием допустимых классов эквивалентности и граничных значений.

b. Для функций, зависящих от времени, таких как фильтры, интегрирующие звенья и задержки, должны быть выполнены многократные итерации кода для проверки характеристик таких функций в реалистичных условиях.

c. Для переходов между состояниями должны быть разработаны тестовые примеры для проверки переходов, возможных при нормальной работе.

d. Для требований к ПО, сформулированных с помощью логических выражений, тестовые примеры в допустимом диапазоне должны верифицировать использование переменных и логические операции.

6.4.2.2. Тестовые примеры на робастность

Тестовые примеры на робастность демонстрируют способность ПО реагировать на ненормальные входные данные и условия. Мероприятия включают:

a. Действительные и целые переменные должны проверяться с использованием выбора классов эквивалентности недопустимых значений.
b. Инициализация системы должна проверяться при ненормальных условиях.

c. Должны быть определены возможные виды отказов входных данных, особенно в сложных последовательностях цифровых данных, поступающих от внешних систем.

d. Для циклов число повторений которых является вычисляемой величиной, должны быть разработаны тестовые примеры, в которых бы делалась попытка вычислить число повторений, выходящее за пределы допустимого диапазона, и таким образом продемонстрировать робастность программы, содержащей циклы.
e. Должна быть выполнена проверка, подтверждающая, что механизм защиты от превышения установленного времени вычислений работает корректно.
f. Для функций, зависящих от времени, таких как фильтры, интегрирующие звенья и задержки, должны быть разработаны тестовые примеры для механизмов защиты от арифметического переполнения.
g. Для переходов между состояниями должны быть разработаны тестовые примеры, осуществляющие попытки переходов, недопустимых в соответствии с требованиями к ПО.

6.4.3. Методы тестирования на основе требований

Обсуждаемые в настоящем документе методы тестирования на основе требований представляют собой тестирование интеграции аппаратуры и ПО на основе требований, тестирование интеграции ПО на основе требований, и тестирование низкого уровня на основе требований. За исключением тестирования интеграции аппаратуры и ПО, эти методы не предписывают конкретную тестовую среду или стратегию тестирования. Мероприятия включают:

a. Тестирование интеграции аппаратуры и ПО на основе требований. Этот метод тестирования должен быть сконцентрирован на источниках ошибок, связанных с выполнением ПО в среде целевого вычислителя и на функциональности высокого уровня. Основанное на требованиях тестирование интеграции аппаратуры и ПО подтверждает, что ПО в целевом вычислителе удовлетворяет требованиям высокого уровня. Типичными ошибками, выявляемыми этим методом тестирования, являются:

   - Некорректная обработка прерываний.
   - Неудовлетворение требований к времени выполнения.
   - Неправильная реакция ПО на аппаратные переходные процессы или отказы аппаратуры, например последовательность запуска, переходные процессы, вызванные перепадами входных нагрузок, и переходные процессы питания.
   - Проблемы конкуренции при использовании шины данных или других ресурсов, например, при распределении памяти.
   - Неспособность встроенного контроля обнаружить отказы.
   - Ошибки в интерфейсах аппаратуры и ПО.
   - Некорректное поведение контуров управления.
   - Некорректное управление памятью или другой аппаратурой, управляемой программно.
   - Переполнение стека.
   - Некорректная работа механизмов, используемых для подтверждения правильности и совместимости ПО, загружаемого на месте эксплуатации.
   - Нарушения обособления ПО.

b. Тестирование интеграции ПО на основе требований. Этот метод тестирования должен быть сконцентрирован на взаимосвязях между требованиями к ПО и на реализации требований в архитектуре ПО. Тестирование интеграции ПО на основе требований подтверждает, что компоненты ПО правильно взаимодействуют друг с другом и удовлетворяют требованиям к ПО и архитектуре ПО. Этот метод может быть реализован путем расширения охвата требований за счет последовательной интеграции

компонентов программы с соответствующим расширением охвата тестовых примеров. Типичными ошибками, выявляемыми с помощью данного метода тестирования, являются:

- Некорректная инициализация переменных и констант.
- Ошибки в передаче параметров.
- Нарушение целостности данных, особенно глобальных данных.
- Недостаточная сквозная разрешающая способность вычислений.
- Неправильная последовательность событий и операций.

c. Тестирование на основе требований низкого уровня. Этот метод тестирования должен быть сконцентрирован на демонстрации того, что каждый компонент ПО соответствует относящимся к нему требованиям низкого уровня. Тестирование на основе требований низкого уровня подтверждает, что компоненты ПО соответствуют предъявляемым к ним требованиям низкого уровня. Типичными ошибками, выявляемыми с помощью этого метода тестирования, являются:

- Неспособность алгоритма удовлетворить требование к ПО.
- Некорректная работа циклов.
- Некорректные логические решения.
- Неспособность правильно обработать допустимые комбинации входных условий.
- Некорректные реакции на потерю или повреждение входных данных.
- Некорректная обработка исключительных ситуаций, таких как арифметические сбои или нарушения границ массивов.
- Некорректная последовательность вычислений.
- Недостаточная разрешающая способность, точность или производительность алгоритма.

6.4.4. Анализ тестового покрытия

Анализ тестового покрытия представляет собой двухэтапный процесс, состоящий из анализа покрытия требований и анализа структурного покрытия. На первом этапе проводится анализ тестовых примеров в отношении требований к ПО, чтобы подтвердить, что выбранные тестовые примеры удовлетворяют установленным критериям. На втором этапе подтверждается, что выполнение тестовых процедур на основе требований позволяет покрыть структуру кода в соответствии с применяемыми критериями покрытия. Если анализ структурного покрытия показал, что надлежащее покрытие не достигается, для устранения проблем, связанных, например, с мертвым кодом, должны быть указаны дополнительные мероприятия (см. раздел 6.4.4.3).

Целями анализа тестового покрытия является:

a. Покрытие требований высокого уровня достигнуто.

b. Покрытие требований низкого уровня достигнуто.

c. Покрытие структуры ПО по соответствующему критерию покрытия достигнуто.

d. Покрытие структуры ПО в отношении связности по данным и связности по управлению достигнуто.

6.4.4.1. Анализ покрытия требований тестами

Данный анализ определяет, насколько хорошо тестирование на основе требований верифицировало реализацию требований к ПО. Этот анализ может выявить необходимость создания дополнительных тестовых примеров на основе требований к ПО. Мероприятия включают:

a. Анализ, использующий соответствующие Данные Трассировки, для подтверждения, что для каждого требования к ПО имеются тестовые примеры.

b. Анализ, подтверждающий, что тестовые примеры удовлетворяют критериям тестов в допустимом диапазоне и тестов на робастность, определенным в разделе 6.4.2.

c. Устранение любых недостатков, выявленных анализом. Возможными решениями являются добавление или усовершенствование тестовых примеров.

d. Анализ, подтверждающий, что все тестовые примеры и, таким образом, все тестовые процедуры, используемые для обеспечения структурного покрытия, трассируются на требования.

6.4.4.2. Анализ структурного покрытия

Анализ структурного покрытия определяет, какая структура кода, включая интерфейсы между компонентами, не была выполнена тестовыми процедурами, основанными на требованиях. Тестовые примеры, основанные на требованиях, могут не полностью задействовать структуру кода, включая интерфейсы, поэтому выполняется анализ структурного покрытия и производится дополнительная верификация для обеспечения структурного покрытия. Мероприятия включают в себя:

a. Анализ информации о структурном покрытии, собранной во время тестирования на основе требований, для подтверждения, что степень структурного покрытия соответствует уровню ПО.

b. Анализ структурного покрытия может быть выполнен по Исходному Коду, объектному коду или Исполняемому Объектному Коду. Независимо от формы кода, на котором выполняется анализ структурного покрытия, если уровень ПО «A», а компилятор, линкер или иные средства генерируют дополнительный код, который напрямую не трассируется на операторы Исходного Кода, должна быть проведена дополнительная верификация, чтобы установить корректность таких сгенерированных последовательностей кода.

Примечание: *«Дополнительный код, который не трассируется напрямую на операторы исходного кода» представляет собой код, содержащий ветвления или побочные эффекты, которые не видны непосредственно на уровне Исходного Кода. Это означает, что генерируемые компилятором проверки границ массивов, например, не считаются напрямую трассируемыми относительно операторов Исходного Кода в рамках анализа структурного покрытия, и они должны быть подвергнуты дополнительной верификации.*

c. Анализ, подтверждающий, что при тестировании на основе требований была задействована связность по данным и по управлению между компонентами кода.

d. Устранение недостатков по результатам анализа структурного покрытия (см. раздел 6.4.4.3).

6.4.4.3. Устранение недостатков по результатам анализа структурного покрытия

Анализ структурного покрытия может обнаружить структуры кода, включая интерфейсы, которые не были выполнены во время тестирования. Устранение недостатков потребует дополнительных мероприятий процесса верификации ПО. Причины непокрытой структуры кода, включая интерфейсы, и соответствующие мероприятия для их устранения включают:

a. Недостатки в тестовых примерах или процедурах, основанных на требованиях. Для достижения полноты покрытия должны быть дополнены тестовые примеры или изменены тестовые процедуры. Может потребоваться пересмотр метода (методов), используемого для анализа покрытия на основе требований.

b. Недостаточность требований к ПО. Должны быть изменены требования к ПО, разработаны дополнительные тестовые примеры и выполнены дополнительные тестовые процедуры.

c. Наличие постороннего кода, включая мертвый код. Такой код должен быть удален, и должен быть выполнен анализ, чтобы оценить последствия и необходимость повторной верификации. При обнаружении постороннего кода на уровне Исходного или объектного кода его можно оставить, только если анализ показывает, что его нет в Исполняемом Объектном Коде (например, за счет интеллектуальной компиляции, линковки или за счет иного механизма) и имеются процедуры для предотвращения его включения в последующие сборки.

d. Отключенный код. Отключенный код должен быть обработан одним из двух способов, в зависимости от определенной для него категории:

   1. Первая категория. Отключенный код, который не предназначен для исполнения в любой имеющейся конфигурации, используемой в любом сертифицируемом изделии. Для этой категории сочетание анализа и тестирования должно показать, что средства, с помощью которых отключенный код мог бы быть выполнен непреднамеренно, ограничены, изолированы или устранены. Любое переназначение уровня ПО для отключенного кода должно быть обосновано процессом оценки безопасности системы и должно быть документировано в Плане Сертификации ПО. Аналогично, любые ослабления процесса верификации ПО для отключенного кода должны быть обоснованы процессами разработки ПО и должны быть документированы в Плане Сертификации ПО.

   2. Вторая категория. Отключенный код, который исполняется только в определенных утвержденных конфигурациях среды целевого вычислителя. Чтобы достичь требуемых целей покрытия, должна быть установлена эксплуатационная конфигурация, необходимая для нормального исполнения данного кода, и разработаны дополнительные тестовые примеры и процедуры.

6.4.5. Рассмотрения и анализы тестовых примеров, процедур и результатов тестов

Мероприятия рассмотрений и анализов подтверждают, что тестирование ПО достигает следующих целей:

a. Тестовые примеры. Цели, касающиеся верификации тестовых примеров, представлены в разделах

   6.4.4.a и 6.4.4.b.

b. Тестовые процедуры. Цель - подтвердить, что тестовые примеры, включая ожидаемые результаты, правильно переработаны в тестовые процедуры.

c. Результаты тестирования. Цель - подтвердить, что результаты тестирования корректны и что расхождения между фактическими и ожидаемыми результатами объяснены.

6.5. Трассируемость процесса верификации ПО

Мероприятия по обеспечению трассируемости процесса верификации ПО включают:

a. Разработаны Данные Трассировки, демонстрирующие двунаправленную связь между требованиями к ПО и тестовыми примерами. Эти Данные Трассировки поддерживают анализ покрытия требований тестами.

b. Разработаны Данные Трассировки, демонстрирующие двунаправленную связь между тестовыми примерами и тестовыми процедурами. Эти Данные Трассировки дают возможность подтвердить, что полный набор тестовых примеров был переработан в тестовые процедуры.

c. Разработаны Данные Трассировки, демонстрирующие двунаправленную связь между тестовыми процедурами и результатами тестирования. Эти Данные Трассировки дают возможность подтвердить, что был выполнен полный набор тестовых процедур.

6.6. Верификация компонентов параметрических данных

Верификация компонента параметрических данных может быть выполнена отдельно от верификации Исполняемого Объектного Кода, если применимы все следующие условия:

- Исполняемый Объектный Код был разработан и верифицирован тестированием в допустимом диапазоне на корректное обращение ко всем Файлам Компонентов Параметрических Данных, которые соответствуют заданной структуре и набору атрибутов.
- Исполняемый Объектный Код является робастным в отношении структуры и атрибутов Файлов Компонентов Параметрических Данных.
- Могут быть верифицированы все варианты поведения Исполняемого Объектного Кода, зависящие от содержания Файла Компонента Параметрических Данных.
- Структура данных ЖЦ позволяет управлять отдельно компонентом параметрических данных.

Если не выполнены условия, указанные выше, компоненты параметрических данных не должны верифицироваться отдельно от остальной части ПО. В этом случае Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных должны быть верифицированы совместно.

Для компонентов параметрических данных, которые могут быть верифицированы отдельно от верификации Исполняемого Объектного Кода, применяются цели, указанные ниже. Эти цели могут быть достигнуты с помощью сочетания тестирования, анализов и рассмотрений.

a. Файл Компонента Параметрических Данных должен быть верифицирован на соответствие его структуры требованиям высокого уровня; эта верификация включает подтверждение того, что Файл Компонента Параметрических Данных не содержит элементов, не определенных требованиями высокого уровня. Также должно быть продемонстрировано, что каждый элемент данных в Файле Компонента Параметрических Данных имеет корректное значение, не противоречит другим элементам данных, и соответствует его атрибутам, как определено требованиями высокого уровня.

Примечание: Для некоторых элементов данных единственным аспектом, который должен быть верифицирован, могут быть их атрибуты. В других случаях может быть нужна верификация значения элемента данных.

b. Все элементы Файла Компонента Параметрических Данных были покрыты при верификации.

Если произведены изменения в структуре или атрибутах компонента параметрических данных, то должна быть проанализирована необходимость изменения и повторной верификации Исполняемого Объектного Кода.

7.0. ПРОЦЕСС УПРАВЛЕНИЯ КОНФИГУРАЦИЕЙ ПО

В настоящем разделе рассматриваются цели и мероприятия процесса управления конфигурацией ПО (УКПО). Процесс УКПО осуществляется так, как это определено в процессе планирования ПО (см. раздел 4) и в Плане Управления Конфигурацией ПО (см. раздел 11.4). Выходные данные процесса УКПО сохраняются в Записях Управления Конфигурацией ПО (см. раздел 11.18) или в других данных ЖЦ ПО.

Процесс УКПО, работая совместно с другими процессами ЖЦ ПО, помогает в:

a. Обеспечении определенной и управляемой конфигурации ПО в течение всего ЖЦ ПО.

b. Обеспечении возможности корректного тиражирования Исполняемого Объектного Кода и Файлов Компонентов Параметрических Данных, если таковые имеются, при изготовлении ПО или повторного его получения в случае необходимости расследования или внесения изменений.

c. Обеспечении управления входными и выходными данными процессов в течение ЖЦ ПО, что гарантирует согласованность и повторяемость мероприятий процессов.

d. Обеспечении известной точки для рассмотрения, оценки состояния и управления изменениями посредством контроля единиц конфигурации и установлением базовых версий.

e. Обеспечении процедур управления, которые гарантируют, что проблемы принимаются во внимание, а изменения зарегистрированы, утверждены и реализованы.

f. Обеспечении свидетельства утверждения ПО за счет контроля выходных данных процессов ЖЦ ПО.

g. Оценке соответствия программного продукта требованиям.

h. Гарантии того, что для единиц конфигурации обеспечивается надежное физическое хранение, восстановление и управление.

7.1. Цели процесса управления конфигурацией ПО

Цели процесса УКПО следующие:

a. Каждая единица конфигурации и ее последовательные версии идентифицируются однозначным образом, чтобы сформировать основу для контроля и ссылок на единицы конфигурации.

b. Базовые версии определены для последующих мероприятий процессов ЖЦ ПО и позволяют ссылаться на единицы конфигурации, управлять ими и трассировать их друг на друга.

c. Процесс регистрации проблем фиксирует несоответствия процессов планам и стандартам создания ПО, фиксирует недостатки выходных данных процессов ЖЦ ПО, фиксирует ненормальное поведение программных продуктов и гарантирует решение этих проблем.

d. Управление изменениями обеспечивает регистрацию, оценку, анализ и утверждение изменений в течение всего ЖЦ ПО.

e. Рассмотрение изменений гарантирует, что с помощью методов регистрации проблем и управления изменениями, определенных в процессе планирования ПО, проблемы и изменения будут оценены, утверждены или отклонены, утвержденные изменения реализованы, а также предоставлена обратная связь к процессам, на которые оказывается влияние.

f. Учет состояния предоставляет данные для управления конфигурацией процессов ЖЦ ПО в отношении идентификации конфигурации, базовых версий, Сообщений о Проблемах и управления изменениями.

g. Хранение и воспроизведение гарантируют, что связанные с программным продуктом данные ЖЦ ПО можно воспроизвести в случае необходимости дублирования, повторного получения, повторного тестирования или изменения программного продукта. Цель мероприятия выпуска – гарантировать, что используется только разрешенное ПО, особенно для изготовления ПО, в дополнение к тому, что обеспечено его хранение и воспроизведение.

h. Контроль загрузки ПО гарантирует, что Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются, загружаются в систему или оборудование с соответствующими защитными мерами.

i. Контроль среды ЖЦ ПО гарантирует, что инструменты, используемые для создания ПО, идентифицируются, контролируются и воспроизводимы.

Цели УКПО не зависят от уровня ПО. Однако могут существовать две категории данных ЖЦ ПО, основанные на применяемых к этим данным процедурах УКПО (см. раздел 7.3).

В Таблице А-8 Приложения А представлена сводка целей и выходных данных процесса УКПО.

7.2. Мероприятия процесса управления конфигурацией ПО

Процесс УКПО включает в себя мероприятия по идентификации конфигурации, управлению изменениями, установлению базовых версий и хранению программного продукта вместе с относящимися к нему данными ЖЦ ПО. После утверждения программного продукта сертифицирующим органом процесс УКПО не завершается, а продолжается в течение всего срока эксплуатации системы или оборудования. Если мероприятия ЖЦ ПО будут выполняться поставщиком, тогда мероприятия по управлению конфигурацией применяются к поставщику.

7.2.1. Идентификация конфигурации

Мероприятия включают:

a. Должна быть введена идентификация конфигурации для данных ЖЦ ПО.

b. Должна быть введена идентификация конфигурации для каждой единицы конфигурации, для каждого отдельного контролируемого компонента единицы конфигурации и для комбинаций единиц конфигурации, образующих программный продукт.

c. Единицы конфигурации следует идентифицировать до реализации управления изменениями и анализа трассируемости.

d. Единица конфигурации должна быть идентифицирована до того, как эта единица используется другими процессами ЖЦ ПО, на нее ссылаются другие данные ЖЦ ПО, или она используется для изготовления или загрузки ПО.

e. Если идентификация программного продукта не может быть определена с помощью физической проверки (например, проверки маркировочной таблички на блоке), то в этом случае идентификация конфигурации должна содержаться в Исполняемом Объектном Коде и в Файлах Компонентов Параметрических Данных, если таковые имеются, и быть доступной для других частей системы или оборудования. Это может быть применимо к ПО, загружаемому на месте эксплуатации.

7.2.2. Базовые версии и трассируемость

Мероприятия включают:

a. Должны быть установлены базовые версии для единиц конфигурации, используемых для получения сертификационного зачета. Для поддержки управления мероприятиями процессов ЖЦ ПО могут вводиться промежуточные базовые версии.

b. Должна быть установлена базовая версия программного продукта и дано ее определение в Указателе Конфигурации ПО (см. раздел 11.16).

Примечание: Модифицируемое пользователем ПО не включается в базовую версию программного продукта, за исключением компонентов, обеспечивающих его защиту, и граничных компонентов. Таким образом, модифицируемое пользователем ПО может подвергаться модификации без изменения идентификации конфигурации базовой версии программного продукта.

c. Базовые версии должны быть установлены в контролируемых библиотеках ПО (на физических, электронных или других носителях) для гарантии их целостности. Как только базовая версия установлена, она должна быть защищена от изменений.

d. Разработка производной базовой версии на основе установленной базовой версии должна происходить в соответствии с мероприятиями управления изменениями.

e. Должна быть обеспечена трассируемость базовой версии к той базовой версии, из которой она была получена, если требуется получить сертификационный зачет для мероприятий процессов ЖЦ ПО или данных, связанных с разработкой предыдущей базовой версии.

f. Должна быть обеспечена трассируемость единицы конфигурации к той единице конфигурации, из которой она была получена, если требуется получить сертификационный зачет мероприятий процессов ЖЦ ПО или данных, связанных с разработкой предыдущей единицы конфигурации.

g. Должна быть обеспечена трассируемость базовой версии или единицы конфигурации либо на выходные данные, которые она определяет, либо на процесс, с которым она связана.

7.2.3. Регистрация и отслеживание проблем и корректирующие действия

Мероприятия включают:

a. Должно быть подготовлено Сообщение о Проблеме, которое описывает несоответствие процесса планам, дефект выходных данных или ненормальное поведение ПО, а также предпринятые корректирующие действия, как это определено в разделе 11.17.

Примечание: Проблемы процессов ЖЦ ПО и программного продукта могут регистрироваться в отдельных системах регистрации проблем.

b. Регистрация проблем должна предусматривать идентификацию конфигурации затронутой единицы (единиц) конфигурации или указание затронутых мероприятий процесса, регистрацию состояния Сообщений о Проблемах, а также утверждение и закрытие Сообщений о Проблемах.

c. Сообщения о Проблемах, которые требуют корректирующих действий в отношении программного продукта или выходных данных процессов ЖЦ ПО, должны инициировать мероприятия по управлению изменениями.

7.2.4. Управление изменениями

Мероприятия включают:

a. Управление изменениями должно сохранять целостность единиц конфигурации и базовых версий посредством защиты их от изменений.

b. Управление изменениями должно гарантировать, что любое изменение в единице конфигурации повлечет изменение ее идентификации.

c. Изменения в находящихся под управлением изменениями базовых версиях и единицах конфигурации для создания производных базовых версий необходимо регистрировать, утверждать и отслеживать. Регистрация проблем связана с управлением изменениями, поскольку решение зарегистрированной проблемы может привести к изменениям единиц конфигурации или базовых версий.

Примечание: Общепризнанно, что ранняя реализация управления изменениями улучшает контроль и управление мероприятиями процессов ЖЦ ПО.

d. Изменения ПО должны трассироваться до их источников, а процессы ЖЦ ПО должны повторяться, начиная с той точки, в которой изменение начинает влиять на выходные данные этих процессов. Например, ошибка, обнаруженная при интеграции аппаратуры и ПО и оказавшаяся ошибкой проектирования, потребует коррекции проекта и кода, а также повторения соответствующих мероприятий интегральных процессов.

e. В ходе мероприятий по внесению изменений данные ЖЦ ПО, затрагиваемые этими изменениями, должны обновляться, а записи должны сохраняться в ходе мероприятия управления изменениями. Мероприятия управления изменениями поддерживаются мероприятиями рассмотрения изменений.

7.2.5. Рассмотрение изменений

Мероприятия включают:

a. Оценка влияния проблемы или предлагаемого изменения на требования к системе. Должна быть обеспечена обратная связь к процессам системы, включая процесс оценки безопасности системы, а также должны быть оценены все ответы от процессов системы.

b. Оценка влияния проблемы или предлагаемого изменения на данные ЖЦ ПО с указанием изменений, которые необходимо провести, и действий, которые необходимо выполнить.

c. Подтверждение того, что затрагиваемые единицы конфигурации идентифицированы.

d. Передача по обратной связи Сообщения о Проблеме или информации о влиянии изменения и решений в затрагиваемые процессы.

7.2.6. Учет состояния конфигурации

Мероприятия включают:

a. Регистрацию идентификации единиц конфигурации, идентификации базовых версий, состояния Сообщений о Проблемах, истории изменений и состояния выпуска.

b. Определение хранимых данных и средств регистрации и предоставления отчетов о состоянии этих данных.

7.2.7. Хранение, воспроизведение и выпуск

Мероприятия включают:

a. Должна быть обеспечена возможность воспроизведения данных ЖЦ ПО, связанных с программным продуктом, из утвержденного источника (например, из архива в разрабатывающей организации или компании).

b. Должны быть установлены процедуры, гарантирующие целостность хранимых данных вне зависимости от носителя, на котором они хранятся, посредством:

    1. Гарантии невозможности внесения несанкционированных изменений.

    2. Выбора для хранения носителя, минимизирующего ошибки или искажения при перезаписи. 3. Предотвращение потери или искажения данных со временем. В зависимости от носителя для хранения данных это может включать в себя периодическую контрольную проверку носителя или обновления хранимых данных. 4. Хранения копий-дубликатов в физически отдельных архивах для минимизации риска их утраты в случае стихийного бедствия.

c. Процесс получения дубликатов должен быть проверен на предмет создания точных копий, а также должны существовать процедуры, гарантирующие безошибочное копирование Исполняемого Объектного Кода и Файлов Компонентов Параметрических Данных, если таковые существуют.

d. Единицы конфигурации должны быть идентифицированы и выпущены до начала изготовления ПО, а также должны быть установлены полномочия для их выпуска. Как минимум, должны быть выпущены

компоненты программного продукта, загружаемые в бортовую систему или оборудование. Это включает в себя Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются, а также может включать в себя соответствующий физический носитель для загрузки ПО.

Примечание: Выпуск обычно требуется также и для данных, которые определяют утвержденное ПО для загрузки в бортовую систему или оборудование. Определение этих данных выходит за рамки настоящего документа, но может включать в себя Указатель Конфигурации ПО.

e. Чтобы удовлетворить требования норм летной годности и позволить вносить изменения в ПО, должны быть установлены процедуры сохранения данных.

Примечание: Дополнительные аспекты, связанные с сохранением данных, могут учитывать коммерческие нужды и будущие экспертизы, проводимые сертифицирующим органом, но эти аспекты выходят за рамки данного документа.

7.3. Категории контроля

Данные ЖЦ ПО могут быть отнесены к одной из двух категорий контроля управления конфигурацией: категории контроля 1 (КК1) и категории контроля 2 (КК2). В Таблице 7-1 определена группа мероприятий процесса УКПО, связанная с каждой из категорий контроля, символ «●» указывает минимальные мероприятия, относящиеся к данным ЖЦ ПО конкретной категории. Мероприятия КК2 представляют собой подмножество мероприятий КК1.

В таблицах Приложения А указана категория контроля в соответствии с уровнем ПО для единиц данных ЖЦ ПО.

Таблица 7-1. Мероприятия процесса УКПО, назначенные данным категорий контроля 1 и 2

Мероприятия процесса УКПО	Ссылка	КК1	КК2
Идентификация конфигурации	7.2.1	●	●
Базовые версии	7.2.2.a 7.2.2.b 7.2.2.c 7.2.2.d 7.2.2.e	●	
Трассируемость	7.2.2.f 7.2.2.g	●	●
Регистрация проблем	7.2.3	●	
Управление изменениями – целостность и идентификация	7.2.4.a 7.2.4.b	●	●
Управление изменениями – отслеживание	7.2.4.c 7.2.4.d 7.2.4.e	●	
Рассмотрение изменений	7.2.5	●	
Учет состояния конфигурации	7.2.6	●	
Воспроизведение	7.2.7.a	●	●
Защита от несанкционированных изменений	7.2.7.b.1	●	●
Выбор физического носителя, обновление, дублирование	7.2.7.b.2 7.2.7.b.3 7.2.7.b.4 7.2.7.c	●	
Выпуск	7.2.7.d	●	
Сохранение данных	7.2.7.e	●	●

7.4. Контроль загрузки ПО

Понятие контроля загрузки ПО относится к процессу, посредством которого запрограммированные команды и данные пересылаются из эталонного запоминающего устройства в систему или оборудование. Например, методы могут включать в себя (при обязательном утверждении сертифицирующим органом) установку предварительно запрограммированных на заводе-изготовителе запоминающих устройств или перепрограммирование системы или оборудования без демонтажа с помощью устройства для обеспечения загрузки на месте эксплуатации. Вне зависимости от используемого метода в контроль загрузки ПО должны быть включены:

a. Процедуры присвоения идентификационных номеров и идентификации носителей, идентифицирующих конфигурации ПО, которые должны быть утверждены для загрузки в бортовую систему или оборудование.

b. Вне зависимости от того, поставляется ли ПО как отдельная единица или оно поставляется установленным в бортовую систему или оборудование, должны сохраняться записи, подтверждающие совместимость данного ПО с аппаратурой бортовой системы или оборудования.

7.5. Контроль среды ЖЦ ПО

Инструменты среды ЖЦ ПО определяются в процессе планирования ПО и идентифицируются в Указателе Конфигурации Среды ЖЦ ПО (см. раздел 11.15). Мероприятия включают в себя следующее:

a. Должна быть установлена идентификация конфигурации Исполняемого Объектного Кода (или его эквивалента) инструментов, используемых для разработки, контроля, сборки, верификации и загрузки ПО.

b. Процесс УКПО в части управления квалифицированными инструментами должен соответствовать целям, связанным с данными категории контроля 1 или 2 (см. раздел 7.3) в соответствии с инструктивными материалами, представленными в разделе 12.2.3.

c. За исключением случаев, когда применим раздел 7.5.b, процесс УКПО в части контроля Исполняемого Объектного Кода (или его эквивалента) для инструментов, используемых для сборки и загрузки ПО (например, компиляторов, ассемблеров и линкеров) должен соответствовать, как минимум, целям, связанным с данными категории контроля 2.

8.0. ПРОЦЕСС ГАРАНТИИ КАЧЕСТВА ПО

В настоящем разделе рассматриваются цели и мероприятия процесса гарантии качества ПО (ГКПО). Процесс ГКПО применяется так, как это определено в процессе планирования ПО (см. раздел 4) и в Плане Гарантии Качества ПО (см. раздел 11.5). Выходные данные мероприятий процесса ГКПО сохраняются в Записях ГКПО (см. раздел 11.19) или в других данных ЖЦ ПО.

Процесс ГКПО оценивает процессы ЖЦ ПО и их выходные данные для получения гарантии того, что цели достигаются, недостатки выявляются, оцениваются, прослеживаются и устраняются, а программный продукт и данные ЖЦ ПО удовлетворяют сертификационным требованиям.

8.1. Цели процесса гарантии качества ПО

Цели процесса ГКПО – обеспечить уверенность в том, что процессы ЖЦ ПО создают ПО, удовлетворяющее предъявляемым к нему требованиям, за счет гарантии того, что эти процессы выполняются в соответствии с утвержденными планами и стандартами создания ПО.

Целями процесса ГКПО является получение гарантии того, что:

a. Планы и стандарты создания ПО разработаны и рассмотрены на предмет непротиворечивости и соответствия настоящему документу.

b. Процессы ЖЦ ПО, включая процессы поставщиков, соответствуют утвержденным планам и стандартам создания ПО.

c. Удовлетворены критерии перехода для процессов ЖЦ ПО.

d. Выполнено рассмотрение соответствия программного продукта.

В Таблице А-9 Приложения А представлена сводка целей и выходных данных процесса ГКПО.

8.2. Мероприятия процесса гарантии качества ПО

Мероприятия для достижения целей процесса ГКПО включают в себя:

a. Процесс ГКПО должен играть активную роль в мероприятиях процессов ЖЦ ПО, а персонал, задействованный в процессе ГКПО, должен быть наделен полномочиями, ответственностью и независимостью, чтобы обеспечить достижение целей этого процесса.

b. Процесс ГКПО должен гарантировать, что планы и стандарты создания ПО составляются и проверяются на соответствие настоящему документу и на согласованность между собой.

c. Процесс ГКПО должен гарантировать, что процессы ЖЦ ПО соответствуют утвержденным планам и стандартам создания ПО.

d. Процесс ГКПО должен включать в себя аудиты процессов ЖЦ ПО в течение ЖЦ ПО, чтобы получить гарантию того, что:

    1. Планы создания ПО доступны, как указано в разделе 4.2.

    2. Отклонения от планов и стандартов создания ПО выявляются, регистрируются, оцениваются, отслеживаются и устраняются.

    Примечание: Общепризнанно, что раннее выявление отклонений в процессах способствует успешному достижению целей процессов ЖЦ ПО.

    3. Утвержденные отклонения регистрируются.

4. Среда разработки ПО соответствует указанной в планах создания ПО.

5. Мероприятия по регистрации и отслеживанию проблем и выполнению корректирующих действий соответствуют Плану Управления Конфигурацией ПО.

6. Данные, поступившие от процессов системы, включая процесс оценки безопасности системы, учтены в процессах ЖЦ ПО.

Примечание: Для обеспечения гарантии того, что мероприятия ЖЦ ПО находятся под контролем, они могут подвергаться мониторингу.

e. Процесс ГКПО должен гарантировать, что критерии перехода для процессов ЖЦ ПО были удовлетворены в соответствии с утвержденными планами создания ПО.

f. Процесс ГКПО должен гарантировать, что данные ЖЦ ПО управляются в соответствии с категориями контроля, как это определено в разделе 7.3 и в таблицах Приложения А.

g. Перед поставкой программных продуктов, представляемых как часть сертификационной заявки, должно быть выполнено рассмотрение соответствия ПО.

h. Процесс ГКПО должен оставлять записи о мероприятиях (см. раздел 11.19), включающие результаты аудитов и свидетельства завершения рассмотрения соответствия ПО для каждого программного продукта, входящего в сертификационную заявку.

i. Процесс ГКПО должен гарантировать, что процессы и выходные данные поставщика соответствуют утвержденным планам и стандартам создания ПО.

8.3. Рассмотрение соответствия ПО

Рассмотрение соответствия ПО проводится для получения гарантий в отношении программного продукта, представляемого как часть сертификационной заявки, что процессы ЖЦ ПО завершены, данные ЖЦ ПО полны, Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются, контролируются и могут быть сгенерированы повторно.

Это рассмотрение должно установить, что:

a. Запланированные для получения сертификационного зачета мероприятия процессов ЖЦ ПО, включая создание данных ЖЦ ПО, завершены и записи об их завершении имеются.

b. Данные ЖЦ ПО, разработанные на основании конкретных требований к системе, требований по обеспечению безопасности или требований к ПО, трассируемы на эти требования.

c. Существуют свидетельства того, что данные ЖЦ ПО получены в соответствии с планами и стандартами создания ПО и управляются в соответствии с Планом УКПО.

d. Существуют свидетельства того, что Сообщения о Проблемах оценены и их статус зарегистрирован.

e. Отступления от требований к ПО зарегистрированы и утверждены.

f. Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются, могут быть повторно получены из хранимого Исходного Кода.

g. Утвержденное ПО может быть успешно загружено с использованием выпущенных инструкций.

h. Сообщения о Проблемах, отложенные по результатам предыдущих рассмотрений соответствия ПО, оценены повторно для определения их состояния.

i. Если запрашивается сертификационный зачет на использование ранее разработанного ПО, то текущая базовая версия программного продукта трассируема на предыдущую базовую версию и к утвержденным изменениям той базовой версии.

Примечание: Для изменений, вносимых после сертификации, может выполняться не весь набор мероприятий рассмотрения соответствия ПО согласно тому, как это обосновывается значимостью изменений.

9.0. ПРОЦЕСС ВЗАИМОДЕЙСТВИЯ ЗАЯВИТЕЛЯ С СЕРТИФИЦИРУЮЩИМ ОРГАНОМ

Цели процесса взаимодействия Заявителя с сертифицирующим органом следующие:

a. Установить связь и взаимопонимание между Заявителем и сертифицирующим органом в течение всего ЖЦ ПО для содействия процессу сертификации.

b. Достичь согласия по методам определения соответствия путем утверждения Плана Сертификации ПО.

c. Привести доказательство соответствия.

Процесс взаимодействия Заявителя с сертифицирующим органом применяется так, как это определено процессом планирования ПО (см. раздел 4) и Планом Сертификации ПО (см. раздел 11.1). В Таблице А-10 Приложения А представлена сводка целей и выходных данных этого процесса.

9.1. Методы определения соответствия и планирование

Заявитель предлагает методы определения соответствия, которые определяют, каким образом разработка бортовой системы или оборудования будет соответствовать сертификационному базису. План Сертификации ПО (см. раздел 11.1) определяет аспекты, связанные с ПО бортовой системы или оборудования, в контексте предлагаемых методов определения соответствия. Этот план также указывает уровень (уровни) ПО, как это определено процессом оценки безопасности системы.

Мероприятия включают в себя следующее:

a. Предоставление на рассмотрение сертифицирующему органу Плана Сертификации ПО и других запрашиваемых данных.

b. Решение обнаруженных сертификационным органом проблем, касающихся планирования аспектов сертификации, связанных с ПО.

c. Согласование Плана Сертификации ПО с сертифицирующим органом.

9.2. Доказательство соответствия

Заявитель представляет свидетельства того, что процессы ЖЦ ПО удовлетворяют планам создания ПО, предоставляя сертифицирующему органу доступ к данным ЖЦ ПО для проверки. Аудиторские проверки сертифицирующего органа могут проводиться в различных производственных помещениях. Например, проверки могут проводиться в производственных помещениях Заявителя, в производственных помещениях поставщика Заявителя или в помещениях сертифицирующего органа. Это может включать консультации с Заявителем или его поставщиками. Эти проверки мероприятий процессов ЖЦ ПО организуются Заявителем, а при необходимости он же представляет доступ к данным ЖЦ ПО.

Мероприятия включают в себя следующее:

a. Решение вопросов, поставленных сертифицирующим органом по результатам проводимых им проверок;

b. Представление сертифицирующему органу Итогового Заключения о ПО (см. раздел 11.20) и Указателя Конфигурации ПО (см. раздел 11.16);

c. По запросу сертифицирующего органа представлять или открывать доступ к другим данным или свидетельствам, подтверждающим соответствие.

9.3. Минимальный состав представляемых сертифицирующему органу данных ЖЦ ПО

Минимальный состав представляемых сертифицирующему органу данных ЖЦ ПО включает в себя:

a. План Сертификации ПО.

b. Указатель Конфигурации ПО.

c. Итоговое Заключение о ПО.

9.4. Данные ЖЦ ПО, относящиеся к типовой конструкции

Если с сертифицирующим органом не согласовано иное, то правила, касающиеся порядка воспроизведения и утверждения данных ЖЦ ПО, относящихся к типовой конструкции сертифицируемого изделия, применяются к следующим данным: a. Требования к ПО.

b. Описание Проекта ПО.

c. Исходный Код.

d. Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются.

e. Указатель Конфигурации ПО.

f. Итоговое Заключение о ПО.

10.0. ОБЗОР ПРОЦЕССА СЕРТИФИКАЦИИ

В настоящем разделе представлен обзор процесса сертификации в отношении аспектов, связанных с ПО бортовых систем и оборудования, который приводится только в информационных целях.

Авиационное сообщество и сертифицирующие органы используют несколько терминов, касающихся выдачи разрешения на полеты воздушных судов с соответствующим оборудованием. Используются такие термины, как «сертификация», «одобрение» и, в отношении инструментов, «квалификация».

Термин «сертификация» относится к воздушным судам, двигателям или винтам. Сертифицирующие органы рассматривают ПО как часть бортовой системы или оборудования, устанавливаемых на сертифицируемом изделии, то есть сертифицирующие органы не сертифицируют ПО как уникальный самостоятельный продукт.

Системы и оборудование, включая встроенное ПО, должны получить «одобрение», чтобы быть принятыми как часть сертификации. Одобрение со стороны сертифицирующих органов выдается на основании успешной демонстрации или проверки результатов ЖЦ ПО. В настоящее время любое такое одобрение имеет значение только в контексте конкретной сертификации.

«Квалификация» инструментов рассматривается в разделе 12.2.

10.1. Сертификационный базис

После консультаций с Заявителем, сертифицирующий орган определяет сертификационный (квалификационный) базис. В сертификационном (квалификационном) базисе определяются конкретные нормативные требования и любые специальные технические условия, которые могут дополнять опубликованные нормативные документы.

При модификации сертифицированного изделия сертифицирующий орган рассматривает влияние модификации на исходный сертификационный базис. В некоторых случаях сертификационный базис для модификации может не измениться по сравнению с исходным сертификационным базисом, однако, исходные методы определения соответствия могут оказаться не применимыми для демонстрации соответствия модификации сертификационному базису и может потребоваться их изменение.

10.2. Учет ПО при сертификации

Сертифицирующий орган оценивает План Сертификации ПО с точки зрения его полноты и согласованности с методами определения соответствия, которые, согласно достигнутой договоренности, соответствуют сертификационному базису. Сертифицирующий орган убеждается в том, что предложенный Заявителем уровень (уровни) ПО согласуется с выходными данными процесса оценки безопасности системы и другими данными ЖЦ системы. Сертифицирующий орган информирует Заявителя о проблемах в предложенных планах создания ПО, которые Заявитель должен устранить до согласования с сертифицирующим органом.

10.3. Определение соответствия

До сертификации образца АТ сертифицирующий орган устанавливает, что аспекты, связанные с ПО его систем или оборудования, соответствует сертификационному базису. Для ПО это достигается с помощью рассмотрения Итогового Заключения о ПО и свидетельств соответствия. Сертифицирующий орган использует Итоговое Заключение о ПО как обзорный документ по аспектам сертификации, связанным с ПО. В течение ЖЦ ПО сертифицирующий орган по своему усмотрению может проводить проверку процессов ЖЦ ПО и их выходных данных, как это описано в разделе 9.2.

11.0. ДАННЫЕ ЖИЗНЕННОГО ЦИКЛА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

В течение ЖЦ ПО создаются данные для планирования, организации, объяснения, определения, регистрации мероприятий или представления свидетельств проведения этих мероприятий. Эти данные позволяют выполнять процессы ЖЦ ПО, выполнять сертификацию системы или оборудования, а также вносить изменения в программный продукт после его сертификации. В данном разделе рассматриваются характеристики, форма, средства управления конфигурацией и содержание данных ЖЦ ПО.

a. Характеристики. Данные ЖЦ ПО должны быть:

   1. Однозначными. Информация является однозначной, если она записана в терминах, допускающих единственное толкование, и с добавлением, при необходимости, определений.

   2. Полными. Информация является полной, если она включает необходимые и релевантные требования и (или) описательный материал, определены реакции в диапазоне допустимых входных данных, используемые рисунки снабжены надписями, определены термины и единицы измерения.

   3. Верифицируемыми. Информация является верифицируемой, если ее правильность может быть проверена человеком или инструментом.

   4. Непротиворечивыми. Информация является непротиворечивой, если она не содержит внутренних конфликтов.

   5. Модифицируемыми. Информация является модифицируемой, если она имеет такие структуру и стиль, что изменения можно выполнить полностью, непротиворечиво и корректно при сохранении структуры.

   6. Трассируемыми. Информация является трассируемой, если можно определить происхождение ее составных частей.

b. Форма. Форма данных ЖЦ ПО должна обеспечивать эффективное воспроизведение и рассмотрение данных ЖЦ ПО в течение всего срока эксплуатации бортовой системы или оборудования. Данные и их конкретная форма должны быть определены в Плане Сертификации ПО.

Примечание 1: Данные ЖЦ ПО могут храниться в различных формах (например, храниться в электронном виде или в отпечатанном виде).

Примечание 2: Заявитель может группировать данные ЖЦ ПО так, как ему удобно (например, в виде отдельных или сгруппированных элементов данных).

Примечание 3: Некоторые сертифицирующие органы могут потребовать План Сертификации ПО и Итоговое Заключение о ПО в виде отдельных документов.

Примечание 4: Термин «данные» относится к свидетельствам и к другой информации и не подразумевает конкретной формы, которую они должны принимать.

c. Средства управления конфигурацией. Данные ЖЦ ПО можно отнести к одной из двух категорий, связанных с применяемыми средствами управления конфигурацией ПО: КК1 и КК2 (см. раздел 7.3). Минимальная категория контроля, присваиваемая каждой единице данных, и ее изменение в зависимости от уровня ПО указаны в таблицах Приложения А. Если кроме описанных в настоящем документе единиц данных производятся дополнительные единицы данных в качестве свидетельства для поддержки процесса сертификации, они должны управляться, как минимум, по категории КК2.

d. Содержимое. Описания данных ЖЦ ПО, представленные в приведенных ниже разделах, определяют данные, которые обычно создаются в ЖЦ ПО. Эти описания не предназначены для описания всех данных, которые могут потребоваться для разработки программного продукта, а также не предназначены

для указания конкретного метода группировки данных или структуры сгруппированных данных. Представленные в настоящем разделе описания содержания единиц данных ЖЦ ПО не являются исчерпывающими, и их следует читать совместно с другими разделами настоящего документа и адаптировать к нуждам Заявителя.

11.1. План Сертификации ПО

План Сертификации ПО является основным средством, используемым сертифицирующим органом для определения, предлагает ли Заявитель ЖЦ ПО, соответствующий строгости, требуемой уровнем разрабатываемого ПО. Этот план должен содержать:

a. Обзорное описание системы: В этом разделе дается обзорное описание системы, включающее описание ее функций и их распределение между аппаратурой и ПО, описание архитектуры, используемого процессора (процессоров), интерфейсов аппаратуры и ПО, а также описание мер обеспечения безопасности.

b. Обзорное описание ПО: В данном разделе кратко описываются функции ПО, уделяя особое внимание предлагаемым концепциям обеспечения безопасности и обособления. Например, разделение ресурсов, резервирование, отказоустойчивость, компенсация единичного сбоя, стратегии синхронизации и диспетчеризации.

c. Аспекты, связанные с сертификацией: В данном разделе приводится обобщение сертификационного базиса, включая методы определения соответствия, имеющие отношение к аспектам сертификации, связанным с ПО. Кроме того, в этом разделе устанавливается предлагаемый уровень (уровни) ПО и обобщается обоснование, получаемое в процессе оценки безопасности системы, включая потенциальный вклад ПО в отказные состояния.

d. Описание ЖЦ ПО: В данном разделе определяется ЖЦ ПО, который предполагается использовать, с кратким описанием каждого из процессов ЖЦ ПО. Более подробная информация по этим вопросам дается в соответствующих планах создания ПО. Краткое описание показывает, каким образом будут достигнуты цели каждого процесса ЖЦ ПО, определяет привлекаемые организации, определяет ответственных за организацию работ, за процессы ЖЦ системы, а также за процесс взаимодействия между Заявителем и сертифицирующим органом.

e. Данные ЖЦ ПО: В данном разделе определяются данные ЖЦ ПО, которые будут создаваться и контролироваться процессами ЖЦ ПО. В этом разделе описывается также взаимосвязь данных между собой и с другими данными, определяющими систему, описываются данные ЖЦ ПО, представляемые на проверку сертифицирующему органу, указывается форма этих данных и средства, с помощью которых эти данные будут доступны сертифицирующему органу.

f. План-график: В этом разделе описывается, каким образом Заявитель будет информировать сертифицирующий орган о мероприятиях процессов ЖЦ ПО для того, чтобы можно было запланировать соответствующие экспертизы.

g. Дополнительные аспекты: В данном разделе описываются специфические особенности, которые могут повлиять на процесс сертификации, например:

   - альтернативные методы определения соответствия,
   - квалификация инструментов,
   - ранее разработанное ПО,
   - опционное ПО,
   - ПО, модифицируемое пользователем,
   - отключенный код,

- готовое коммерческое ПО,
- ПО, загружаемое на месте эксплуатации,
- компоненты параметрических данных, - многоверсионное разнородное ПО, - история эксплуатации изделия.

h. Контроль поставщика: В данном разделе описываются средства обеспечения того, что процессы и выходные данные поставщика будут соответствовать утвержденным планам и стандартам создания ПО.

11.2. План Разработки ПО

План Разработки ПО представляет собой описание процедур разработки и ЖЦ ПО, которые будут использоваться для достижения целей процессов разработки ПО. Он может быть включен в План Сертификации ПО. Этот план должен содержать:

a. Стандарты: Идентификация Стандартов на Разработку Требований к ПО, Стандартов на Проектирование ПО и Стандартов на Кодирование ПО для данного проекта. Кроме того, ссылки на стандарты, применявшиеся при создании ранее разработанного ПО, включая готовое коммерческое ПО, если они отличаются от используемых в данном проекте.

b. ЖЦ ПО: Описание процессов ЖЦ ПО, которые предполагается использовать при построении конкретных ЖЦ ПО в проекте, включая критерии перехода для процессов разработки ПО. Данное описание отличается от сводки, включаемой в План Сертификации ПО тем, что в нем содержатся подробности, необходимые для обеспечения правильного выполнения процессов ЖЦ ПО.

c. Среда разработки ПО: Описание выбранной среды разработки ПО с указанием аппаратуры и ПО, включая:

    1. Применяемые метод(ы) и инструменты разработки требований.
    2. Применяемые метод(ы) и инструменты проектирования.
    3. Используемые метод(ы) кодирования, язык(и) программирования, инструмент (инструменты) кодирования и, когда это применимо, опции и ограничения генераторов автокода.
    4. Используемые компиляторы, линкеры и загрузчики.
    5. Аппаратные платформы для используемых инструментов.

11.3. План Верификации ПО

План Верификации ПО представляет собой описание процедур верификации, которые должны использоваться, чтобы достичь целей процесса верификации ПО. Эти процедуры могут меняться в зависимости от уровня ПО, как это указано в таблицах Приложения А. Этот план должен содержать:

a. Организация: Распределение ответственности в рамках процесса верификации ПО и взаимодействие с другими процессами ЖЦ ПО.

b. Независимость: Описание методов, с помощью которых обеспечивается независимость верификации, когда это требуется.

c. Методы верификации: Описание методов верификации, которые предполагается использовать для каждого мероприятия процесса верификации ПО:

    1. Методы рассмотрения, включая проверочные перечни и другие средства.
    2. Методы анализа, включая трассируемость и анализ покрытия.
    3. Методы тестирования, включая метод выбора тестовых примеров, используемые тестовые процедуры и данные, которые должны быть получены.

d. Среда верификации: Описание оборудования для тестирования, инструментов тестирования и анализа, а также порядок применения этих инструментов и оборудования для тестирования. В разделе 4.4.3b представлены инструктивные материалы по описанию различий между целевым вычислителем и симулятором или эмулятором.

e. Критерии перехода: Критерии входа в процесс верификации ПО.

f. Аспекты, связанные с обособлением: Если применяется обособление, то указываются методы верификации целостности обособления.

g. Допущения относительно компилятора: Описание допущений, принятых Заявителем относительно корректности компилятора, линкера или загрузчика (см. раздел 4.4.2).

h. Метод повторной верификации: Для модификации ПО – описание методов определения, анализа и верификации частей ПО, затронутых изменениями, а также измененных частей Исполняемого Объектного Кода.

i. Ранее разработанное ПО: Если методы определения соответствия исходной базовой версии ранее разработанного ПО не соответствуют настоящему документу в части процесса верификации, то приводится описание методов для достижения целей настоящего документа.

j. Многоверсионное разнородное ПО: Если применяется многоверсионное разнородное ПО, то приводится описание соответствующих мероприятий процесса верификации ПО (см. раздел 12.3.2).

11.4. План Управления Конфигурацией ПО

В Плане Управления Конфигурацией ПО устанавливаются методы, которые предполагается использовать на протяжении ЖЦ ПО для достижения целей процесса УКПО. Этот план должен содержать:

a. Среда УКПО: Описание среды, которую предполагается использовать для УКПО, в том числе описание процедур, инструментов, методов, стандартов, организационной ответственности и взаимодействия.

b. Мероприятия: Описание мероприятий процесса УКПО, проводимых в течение ЖЦ ПО:

   1. Идентификация конфигурации: Единицы конфигурации, подлежащие идентификации, когда они будут идентифицироваться, методы идентификации для данных ЖЦ ПО (например, правила присвоения идентификационных номеров), а также взаимосвязь идентификации ПО и идентификации системы или оборудования.
   2. Базовые версии и трассируемость: Средства установления базовых версий, какие базовые версии будут установлены, когда эти базовые версии будут установлены, средства управления библиотеками ПО, а также трассируемость единиц конфигурации и базовых версий.
   3. Регистрация проблем: Содержание и идентификация Сообщений о Проблемах для программного продукта и процессов ЖЦ ПО, когда они должны быть составлены, порядок закрытия Сообщений о Проблемах, а также взаимосвязь с мероприятием управления изменениями.
   4. Управление изменениями: Единицы конфигурации и базовые версии, подлежащие управлению, когда они будут контролироваться, мероприятия по управлению проблемами и изменениями, с помощью которых осуществляется управление единицами конфигурации и базовыми версиями, средства управления до сертификации, средства управления после сертификации, методы сохранения целостности базовых версий и единиц конфигурации.
   5. Рассмотрение изменений: Метод обработки входящих и исходящих связей между процессами ЖЦ ПО; методы оценки и назначения приоритетов проблемам, утверждения изменений, порядок разрешения проблем или реализации изменений; а также взаимосвязь этих методов с мероприятиями по регистрации проблем и управлению изменениями.

6. Учет состояния конфигурации: Данные, которые предполагается регистрировать для обеспечения получения информации о состоянии конфигурации; определение того, где эти данные будут храниться, как они могут быть получены для составления отчетов, и когда они становятся доступными.
7. Хранение, воспроизведение и выпуск: Средства контроля целостности, метод и полномочия на выпуск, а также сохранение данных.
8. Контроль загрузки ПО: Описание мер предосторожности при загрузке ПО и записей загрузки ПО.
9. Контроль среды ЖЦ ПО: Средства управления инструментами, используемыми для разработки, сборки, верификации и загрузки ПО с учетом разделов с 11.4.b.1 по 11.4.b.7.
   Сюда включается также контроль инструментов, подлежащих квалификации.
10. Средства управления данными ЖЦ ПО: Средства управления, соответствующие 1-й и 2-й категориям контроля данных (КК1 и КК2).

c. Критерии перехода: Критерии перехода для входа в процесс УКПО.

d. Данные УКПО: Определение данных ЖЦ ПО, создаваемых процессом УКПО, включая Записи УКПО, Указатель Конфигурации ПО и Указатель Конфигурации Среды ЖЦ ПО.

e. Контроль поставщика: Методы распространения требований процесса УКПО на поставщиков.

11.5. План Гарантии Качества ПО

План Гарантии Качества ПО устанавливает методы, которые предполагается использовать для достижения целей процесса гарантии качества ПО. План ГКПО может включать описание мероприятий в части улучшения процесса, метрик и прогрессивных методов управления. Этот план должен содержать:

a. Среда: Описание среды ГКПО, включая описание сферы действия, распределение ответственности и взаимодействие, стандарты, процедуры, инструменты и методы.

b. Полномочия: Положение о полномочиях ГКПО, ответственности и независимости, включая утверждение ГКПО программных продуктов.

c. Мероприятия: Мероприятия ГКПО, которые должны выполняться для каждого процесса ЖЦ ПО и в течение всего ЖЦ ПО, включая:

   1. Методы ГКПО, например, рассмотрения, аудиты, составление отчетов, инспекции и мониторинг процессов ЖЦ ПО;

   2. Мероприятия, связанные с регистрацией и отслеживанием проблем и корректирующими действиями;

   3. Описание мероприятия рассмотрения соответствия ПО.

d. Критерии перехода: Критерии перехода для входа в процесс ГКПО.

e. Синхронизация: Временная привязка мероприятий процесса ГКПО к мероприятиям процессов ЖЦ ПО.

f. Записи ГКПО: Определение записей, производимых процессом ГКПО.

g. Контроль поставщика. Описание средств обеспечения того, что процессы поставщиков и выходные данные поставщиков соответствуют планам и стандартам.

11.6. Стандарты на Разработку Требований к ПО

Стандарты на Разработку Требований к ПО определяют методы, правила и инструменты, которые предполагается использовать для разработки требований высокого уровня. Эти стандарты должны содержать:

a. Методы, которые предполагается использовать для разработки требований к ПО, такие, как методы структурирования.

b. Нотации, которые предполагается использовать для формулирования требований, такие, как диаграммы потоков данных, языки формальных спецификаций.

c. Ограничения на использование инструментов для разработки требований.

d. Метод, который предполагается использовать для передачи производных требований в процессы системы.

11.7. Стандарты на Проектирование ПО

Стандарты на Проектирование ПО определяют методы, правила и инструменты, которые предполагается использовать для разработки архитектуры ПО и требований низкого уровня. Эти стандарты должны содержать:

a. Используемый метод(ы) описания проекта.

b. Используемые правила именования.

c. Условия, накладываемые на разрешенные методы проектирования, например диспетчеризация, использование прерываний, использование управляемых событиями архитектур, динамическое управление задачами, использование повторного входа, глобальные данные, обработка исключений, а также обоснование их использования.

d. Ограничения на использование инструментов проектирования.

e. Ограничения на проект, например исключение рекурсии, динамических объектов, альтернативных имен данных и компактных выражений.

f. Ограничения на сложность, например, на максимальный уровень вложенности вызовов или условных структур, на использование безусловных переходов, количество точек входа-выхода в компоненты программы.

11.8. Стандарты на Кодирование ПО

Стандарты на Кодирование ПО определяют языки программирования, методы, правила и инструменты, которые предполагается использовать для кодирования ПО. Эти стандарты должны содержать:

a. Язык(и) программирования и (или) его(их) подмножества, которые предполагается использовать. Для языка программирования должна быть указана ссылка на документы, которые однозначно определяют синтаксис, поведение конструкций языка, поведение данных и побочные эффекты языка. Это может потребовать введения ограничений на использование некоторых особенностей языка.

b. Стандарты на форму представления Исходного Кода, например, ограничение длины строк, использование отступов и пустых строк, а также стандарты на документирование Исходного Кода, например, указание имени автора, истории изменений, входных и выходных данных, изменяемых глобальных данных.

c. Правила именования компонентов, подпрограмм, переменных и констант.

d. Условия и ограничения, накладываемые на принятые соглашения о приемах кодирования, такие как степень связности между компонентами программы, сложность логических или численных выражений, а также обоснование их применения.

e. Ограничения на применение инструментов кодирования.

11.9. Требования к ПО

Требования к ПО представляют собой определение требований высокого уровня, включая производные требования. Эти данные должны включать в себя:

a. Описание того, как требования к системе, отнесенные к ПО, будут реализованы в ПО, причем особое внимание уделяется требованиям в части обеспечения безопасности и возможным отказным состояниям.

b. Функциональные и эксплуатационные требования для каждого режима работы.

c. Рабочие характеристики, например, точность и правильность.

d. Временные требования и ограничения.

e. Ограничения на размер памяти.

f. Интерфейсы аппаратуры и ПО, например, протоколы, форматы, частоты входных и выходных сигналов.

g. Требования к обнаружению отказов и мониторингу безопасности.

h. Требования к обособлению, отнесенные к ПО, как обособленные компоненты взаимодействуют друг с другом, а также уровень (уровни) ПО каждого обособленного компонента.

11.10. Описание Проекта ПО

В Описании Проекта ПО определяются архитектура ПО и требования низкого уровня, удовлетворяющие требованиям высокого уровня. Эти данные должны содержать:

a. Подробное описание того, каким образом ПО удовлетворяет предъявляемым к нему требованиям высокого уровня, включая алгоритмы, структуры данных, а также то, как требования к ПО распределяются по процессорам и задачам.

b. Описание архитектуры ПО, определяющей структуру ПО для выполнения заданных требований.

c. Описание входных и выходных данных, например, словарь данных, как внутренних, так и внешних для архитектуры ПО.

d. Поток данных и поток управления для проекта.

e. Ограничения на ресурсы, стратегия управления каждым ресурсом и его ограничениями, запасы и методы измерения этих запасов, например, в отношении времени и памяти.

f. Процедуры планирования и механизмы межпроцессорного и межзадачного взаимодействия, включая жесткое временное упорядочение, приоритетное планирование, рандеву языка Ада и прерывания.

g. Методы проектирования и подробные сведения относительно их применения, например, загрузка ПО, модифицируемое пользователем ПО или многоверсионное разнородное ПО.

h. Методы обособления и средства предотвращения нарушения обособления.

i. Описание компонентов ПО, как новых, так и разработанных ранее, и, если они разработаны ранее, то ссылка на базовые версии, из которых они получены.

j. Производные требования, появившиеся в процессе проектирования ПО.

k. Если в системе имеется отключенный код, то дается описание средств, гарантирующих, что этот код не будет активизирован в целевом вычислителе.

l. Обоснование тех проектных решений, которые трассируемы на требования обеспечения безопасности системы.

11.11. Исходный Код

Эти данные представляют собой код, записанный на исходном языке (языках) программирования. Исходный Код вместе с данными компиляции, линковки и загрузки используется в процессе интеграции для того, чтобы создать интегрированную систему или оборудование. Для каждого компонента Исходного Кода эти данные должны включать идентификацию ПО, в том числе, имя и дату изменения и (или) версию, в зависимости от того, какое из этих двух понятий применимо.

11.12. Исполняемый Объектный Код

Исполняемый Объектный Код представляет собой форму кода, пригодную для непосредственного использования процессором целевого вычислителя, и, таким образом, является тем ПО, которое загружается в аппаратуру или в систему.

11.13. Примеры и Процедуры Верификации ПО

Примеры и Процедуры Верификации ПО подробно описывают, как реализуются мероприятия процесса верификации ПО. Эти данные должны включать в себя следующие описания:

a. Процедуры рассмотрений и анализов. Рамки и глубина используемых методов рассмотрения или анализа, дополняющие описание в Плане Верификации ПО.

b. Тестовые примеры. Назначение каждого тестового примера, набор входных данных, условия, ожидаемые результаты для удовлетворения требуемых критериев покрытия, а также критерии «проходит - не проходит».

c. Тестовые процедуры. Пошаговые инструкции по подготовке и выполнению каждого тестового примера, оценке результатов тестирования, а также используемая среда тестирования.

11.14. Результаты Верификации ПО

Результаты Верификации ПО создаются в ходе мероприятий процесса верификации. Результаты Верификации ПО должны:

a. Для каждого рассмотрения, анализа и теста указывать все процедуры и их результаты «проходит – не проходит», а также общий результат «проходит – не проходит».

b. Идентифицировать единицу конфигурации или версию ПО, которая проходила рассмотрение, анализ или тестирование.

c. Включать результаты тестов, рассмотрений и анализов, включая анализы покрытия и трассируемости.

Любые обнаруженные отклонения должны быть зарегистрированы и отслежены процессом регистрации проблем.

Кроме того, свидетельства, представленные в поддержку оценки процессами системы информации, предоставляемой процессами ПО (см. разделы 2.2.1.f и 2.2.1.g), должны считаться Результатами Верификации ПО.

11.15. Указатель Конфигурации Среды ЖЦ ПО

Указатель Конфигурации Среды ЖЦ ПО определяет конфигурацию среды ЖЦ ПО. Указатель составляется для поддержки воспроизведения аппаратуры и ПО среды ЖЦ при повторном получении ПО, повторной верификации или при модификации ПО и должен:

a. Идентифицировать аппаратуру среды ЖЦ ПО и ее операционную систему.

b. Идентифицировать инструменты, используемые во время разработки ПО. Примерами являются компиляторы, линкеры и загрузчики, а также средства обеспечения целостности данных, такие как инструменты, рассчитывающие и встраивающие контрольные суммы или проверки с использованием циклического избыточного кода, а также все генераторы автокода с указанием используемых опций.

c. Идентифицировать среду тестирования, используемую для верификации программного продукта, например, инструменты тестирования и анализа ПО.

d. Идентифицировать квалифицированные инструменты и соответствующие данные по их квалификации.

Примечание: Эти данные могут быть включены в Указатель Конфигурации ПО.

11.16. Указатель Конфигурации ПО

Указатель Конфигурации ПО определяет конфигурацию программного продукта. Должны быть представлены конкретные идентификаторы конфигурации и идентификаторы версии.

Примечание: Указатель Конфигурации ПО может содержать одну единицу данных или набор (иерархию) единиц данных. Указатель Конфигурации ПО может содержать перечисленные ниже единицы или ссылаться на другой Указатель Конфигурации или на другие идентифицированные данные, которые указывают отдельные единицы и их версии.

Указатель Конфигурации ПО должен идентифицировать:

a. Программный продукт.

b. Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются.

c. Каждый компонент Исходного Кода.

d. Ранее разработанное ПО, входящее в состав программного продукта, если таковое используется.

e. Данные ЖЦ ПО.

f. Носители для хранения и выпуска.

g. Инструкции по построению Исполняемого Объектного Кода и Файлов Компонентов Параметрических Данных, если таковые имеются, включая, например, инструкции и данные для компиляции и линковки, а также процедуры, используемые для воспроизведения ПО при его повторной генерации, тестировании или модификации.

h. Ссылку на Указатель Конфигурации Среды ЖЦ ПО (см. раздел 11.15), если он составлен отдельно.

i. Проверки целостности данных для Исполняемого Объектного Кода, если они используются.

j. Процедуры, методы и инструменты выполнения модификаций ПО, модифицируемого пользователем, если таковое имеется.

k. Процедуры и методы для загрузки ПО в целевую аппаратуру.

Примечание: Указатель Конфигурации ПО может быть создан для одной версии программного продукта или может быть расширен так, чтобы содержать данные для нескольких альтернативных или последовательных версий программного продукта.

11.17. Сообщения о Проблемах

Сообщения о Проблемах являются средством идентификации и регистрации анализа фактов ненормального поведения программного продукта, несоответствия процесса планам и стандартам создания ПО, недостатков в данных ЖЦ ПО. Сообщения о Проблемах должны содержать:

a. Идентификацию единицы конфигурации ПО и (или) мероприятия процесса ЖЦ ПО, где обнаружена проблема.

b. Идентификацию единицы (единиц) конфигурации, которая будет изменена, или описание процесса, в который будут внесены изменения.

c. Описание проблемы, позволяющее понять ее суть и разрешить ее. Описание проблемы должно содержать достаточно подробную информацию, способствующую оценке потенциального влияния проблемы на безопасность или функциональность.

d. Описание корректирующих действий, предпринятых для разрешения зарегистрированной проблемы.

11.18. Записи Управления Конфигурацией ПО

Результаты мероприятий процесса УКПО регистрируются в Записях УКПО. Примерами являются списки идентификаторов конфигураций, записи библиотек базовых версий или библиотек программ, отчеты об истории изменений, записи хранения и выпуска. Приведенные примеры не означают, что записи этих конкретных типов обязательно должны создаваться.

Примечание: Вследствие интегрального характера процесса УКПО его выходные данные часто включаются в другие документы ЖЦ ПО как составная часть.

11.19. Записи Гарантии Качества ПО

Результаты мероприятий процесса ГКПО регистрируются в Записях ГКПО. Они могут включать в себя отчеты о рассмотрениях или аудитах ГКПО, протоколы совещаний, записи об утвержденных отклонениях от процессов или записи рассмотрений соответствия ПО.

11.20. Итоговое Заключение о ПО

Итоговое Заключение о ПО представляет собой основную единицу данных для демонстрации соответствия Плану Сертификации ПО. Это заключение должно включать в себя:

a. Обзорное описание системы. В данном разделе дается обзорное описание системы, включающее описание ее функций и их распределения между аппаратурой и ПО, описание архитектуры, используемого процессора (процессоров), интерфейсов аппаратуры и ПО, а также функций обеспечения безопасности. Кроме того, в данном разделе указываются все отличия от обзорного описания системы, приведенного в Плане Сертификации ПО.

b. Обзорное описание ПО. В данном разделе кратко описываются функции ПО с уделением особого внимания используемым концепциям обеспечения безопасности и обособления, а также описываются отличия от обзорного описания ПО, приведенного в Плане Сертификации ПО.

c. Аспекты, связанные с сертификацией. В данном разделе повторяются аспекты сертификации, описанные в Плане Сертификации ПО, с указанием всех отличий от него.

d. ЖЦ ПО. В данном разделе дается обобщенное описание действительно выполненного (-ых) ЖЦ ПО и объясняются отличия от ЖЦ ПО и его процессов, предложенных в Плане Сертификации ПО.

e. Данные ЖЦ ПО. В данном разделе описываются все отклонения от предложений, сделанных в Плане Сертификации ПО для производимых данных ЖЦ ПО, связи этих данных между собой и с другими данными, определяющими систему, а также средства, которыми эти данные были сделаны доступными для сертифицирующего органа. Настоящий раздел в прямой форме, посредством идентификаторов конфигурации и версии, ссылается на соответствующий Указатель Конфигурации ПО и указатель Конфигурации Среды ЖЦ ПО. Подробная информация, касающаяся идентификаторов конфигурации и конкретных версий данных ЖЦ ПО, представлена в Указателе Конфигурации ПО.

f. Дополнительные аспекты. В данном разделе обобщаются любые особые аспекты, которые могут привлечь внимание сертифицирующего органа. В нем объясняются любые отклонения от предложений, содержащихся в Плане Сертификации ПО в части таких аспектов. Должны быть указаны ссылки на единицы данных, применимые к этим вопросам, такие как протоколы обсуждения или особые условия.

g. Контроль поставщика: В данном разделе описывается, как процессы и выходные данные поставщика соответствуют планам и стандартам.

h. Идентификация ПО. В этом разделе указывается идентификация конфигурации ПО в виде идентификационного номера и версии.

i. Характеристики ПО. В данном разделе указывается размер Исполняемого Объектного Кода, временные характеристики, включая наихудшее время исполнения, запасы памяти, ограничения ресурсов и средства, использованные для измерения каждой характеристики.

j. История изменений. В данном разделе, если применимо, дается сводка изменений ПО, уделяя особое внимание тем из них, которые появились в результате отказов, влияющих на безопасность, а также указываются любые изменения и улучшения процессов ЖЦ ПО, имевшие место после предыдущей сертификации.

k. Состояние ПО. В этом разделе приводится сводка Сообщений о Проблемах, по которым на момент сертификации не приняты решения. Сводка Сообщений о Проблемах включает в себя описание каждой проблемы и всех связанных с ней ошибок, ограничений функциональности, эксплуатационных ограничений, потенциального неблагоприятного влияния на безопасность вместе с обоснованием, позволяющим оставить Сообщение о Проблеме открытым, и подробным описанием выполненных или требуемых компенсационных мер.

l. Заявление о соответствии. В этот раздел включено заявление о соответствии настоящему документу, а также сводка методов, используемых для демонстрации соответствия критериям, указанным в планах создания ПО. Кроме того, в этом разделе рассматриваются дополнительные правила, установленные сертифицирующим органом, а также любые отклонения от планов и стандартов создания ПО и от настоящего документа, не учтенные где либо в ином месте Итогового Заключения о ПО.

11.21. Данные Трассировки

Данные Трассировки устанавливают связи между содержимым единиц данных ЖЦ. Должны быть предоставлены Данные Трассировки, демонстрирующие двунаправленные связи между:

a. Требованиями к системе, отнесенными к ПО, и требованиями высокого уровня.

b. Требованиями высокого уровня и требованиями низкого уровня.

c. Требованиями низкого уровня и Исходным Кодом.

d. Требованиями к ПО и тестовыми примерами.

e. Тестовыми примерами и тестовыми процедурами.

f. Тестовыми процедурами и результатами тестов.

11.22. Файл Компонента Параметрических Данных

Файл Компонента Параметрических Данных состоит из данных в форме, пригодной для прямого использования процессором целевого вычислителя.

Данные ЖЦ ПО должны создаваться для каждого экземпляра Компонента Параметрических Данных. Если эти данные комплектуются отдельно, то они должны включать в себя ссылку на Итоговое Заключение о ПО для соответствующего Исполняемого Объектного Кода.

12.0. ДОПОЛНИТЕЛЬНЫЕ АСПЕКТЫ

В предыдущих разделах настоящего документа приведены инструктивные материалы по выполнению сертификационных требований, согласно которым Заявитель представляет свидетельства того, что процессы ЖЦ ПО выполняются в соответствии с этими разделами. В этом разделе представлены инструктивные материалы, связанные с дополнительными аспектами, касающиеся случаев, когда цели и (или) мероприятия заменяют, изменяют или дополняют все или некоторые цели и (или) мероприятия, определенные ранее в настоящем документе. Использование дополнительных аспектов, равно как и планируемое влияние на выполнение инструктивных материалов предыдущих разделов, должны быть согласованы с сертифицирующим органом для каждого конкретного случая.

12.1. Использование ранее разработанного ПО

В данном подразделе приводятся инструктивные материалы, связанные с использованием ранее разработанного ПО, включая: оценку модификаций; влияние изменений условий установки на воздушное судно, условий применения или среды разработки; доработку базовой версии; аспекты, касающиеся процессов УКПО и ГКПО. О намерении использовать ранее разработанное ПО должно быть заявлено в Плане Сертификации ПО. Необходимо также оценить влияние нерешенных Сообщений о Проблемах, связанных с ранее разработанным ПО.

12.1.1. Модификации ранее разработанного ПО

Эти инструктивные материалы рассматривают модификацию ранее разработанного ПО, у которого выходные данные ранее выполненных процессов ЖЦ соответствуют требованиям настоящего документа. Вносимые модификации могут быть следствием изменения требований, обнаружения ошибок и (или) усовершенствований ПО.

Мероприятия включают в себя:

a. Пересмотренные выходные данные процесса оценки безопасности системы должны быть рассмотрены с учетом предлагаемых модификаций.

b. Если пересматривается уровень ПО, то необходимо учитывать инструктивные материалы раздела 12.1.4.

c. Должны быть проанализированы как влияние изменения требований к ПО, так и влияние изменения архитектуры, в том числе влияние изменения требований к ПО на другие требования и связность между несколькими программными компонентами, в результате чего работами по повторной верификации может быть охвачена большая область, чем измененная.

d. Должна быть определена область, на которую влияет изменение. Это может быть сделано с помощью анализа потоков данных и управления, временного анализа, анализа трассируемости или их комбинации.

e. Области, на которые влияют изменения, должны пройти повторную верификацию в соответствии с рекомендациями раздела 6.

12.1.2. Изменение установки на воздушном судне

Бортовые системы или оборудование, имеющие в своем составе ПО, и ранее «одобренные» по определенному уровню ПО и на соответствие конкретному сертификационному базису, могут использоваться при изменившихся условиях установки на воздушном судне (или на другом воздушном судне). Мероприятия следующие:

a. Процесс оценки безопасности системы оценивает новые условия установки на воздушном судне и определяет уровень ПО и сертификационный базис. Если последние при новой установке не изменяются, то дополнительные мероприятия не требуются.

b. Если при изменившихся условиях установки требуется изменение выполняемых функций, то необходимо следовать инструктивному материалу раздела 12.1.1.

c. Если мероприятия, выполненные в ходе предыдущей разработки, не породили выходных данных, требуемых для подтверждения достижения целей обеспечения безопасности системы для новых условий установки, то необходимо следовать инструктивному материалу раздела 12.1.4.

12.1.3. Изменение условий применения или среды разработки ПО

Использование или модификация ранее разработанного ПО могут быть связаны с новой средой разработки, новым целевым процессором или другой аппаратурой, а также интеграцией с другим ПО, отличным от использованного при первоначальном применении.

Новая среда разработки может увеличить или уменьшить объем мероприятий, выполняемых в рамках ЖЦ ПО. Новые условия применения могут потребовать дополнительных мероприятий помимо мероприятий процессов ЖЦ ПО, вызванных модификацией.

Изменения условий применения или среды разработки должны быть определены, проанализированы и повторно верифицированы.

Мероприятия включают:

a. Если новая среда разработки использует инструменты, то может потребоваться применение инструктивных материалов раздела 12.2.

b. Строгость оценки изменения условий применения должна учитывать синтаксическую и семантическую сложность языка программирования. Например, оценка настраиваемых модулей языка Ада будет более строгой, если в новых условиях применения будут по-другому использоваться настраиваемые параметры. Для объектно-ориентированных языков строгость будет выше, если унаследованные объекты отличаются в новых условиях применения.

c. Если используется другой генератор автокода или другой набор опций генератора автокода, то могут измениться как генерируемый Исходный Код, так и объектный код. Должно быть проанализировано влияние любых изменений.

d. Если используется другой компилятор или другой набор опций компилятора, приводящий к созданию другого объектного кода, то результаты предыдущих мероприятий процесса верификации, использовавших объектный код, могут оказаться недействительными и они не должны использоваться для новых условий применения. В этом случае полученные ранее результаты тестирования могут перестать удовлетворять критериям структурного покрытия для новых условий применения. Аналогично, могут оказаться неверными допущения об оптимизации, выполняемой компилятором.

e. Если используется другой процессор, то выполняется анализ влияния изменений, чтобы определить:

   1. Новые компоненты ПО и те компоненты, которые потребуют модификации в результате замены процессора, включая любую модификацию для интеграции аппаратуры и ПО.

   2. Результаты ранее выполненных мероприятий процесса верификации ПО, связанные с интерфейсом ПО и аппаратуры, которые можно использовать для новых условий применения.

   3. Имеющиеся тесты интеграции аппаратуры и ПО, которые должны быть выполнены для новых условий применения. Ожидается, что всегда будет минимальный набор таких тестов.

   4. Дополнительные тесты и рассмотрения интеграции аппаратуры и ПО, которые могут потребоваться.

f. Если меняется отличный от процессора элемент аппаратуры, а проект ПО изолирует взаимодействующие модули от остальных модулей, то тогда должен быть выполнен анализ влияния изменения, чтобы:

   1. Определить модули и интерфейсы ПО, которые являются новыми или будут модифицироваться, чтобы обеспечить использование изменившегося компонента аппаратуры.

   2. Определить требуемый объем повторной верификации.

g. В тех случаях, когда ранее разработанное ПО используется с другим ПО, должна быть проведена верификация интерфейсов ПО. Для определения требуемого объема повторной верификации может быть использован анализ влияния изменений.

12.1.4. Доработка базовой версии

Приведенные ниже инструктивные материалы относятся к ПО, данные ЖЦ которого для предыдущего применения признаны недостаточными, либо они не достигают целей настоящего документа из-за требований, связанных с новыми условиями применения. Данные инструктивные материалы предназначены для облегчения достижения целей настоящего документа, когда они применяются к:

- Готовому коммерческому ПО.
- Бортовому ПО, разработанному в соответствии с другими инструктивными материалами.
- Бортовому ПО, разработанному до выпуска данного документа.
- ПО, разработанному ранее в соответствии с настоящим документом как ПО более низкого уровня.

Мероприятия по доработке базовой версии включают в себя:

a. Цели настоящего документа должны быть достигнуты при использовании тех данных ЖЦ ПО предыдущей разработки, которые достигают целей в новых условиях применения.

b. Аспекты сертификации, связанные с использованием ПО, должны основываться на отказных состояниях и уровне (уровнях) ПО, как это определено процессом оценки безопасности системы. Сопоставление с отказными состояниями для прежних условий применения определит те области, которые, возможно, потребуют доработки.

c. Данные ЖЦ ПО из предыдущей разработки должны быть оценены, чтобы подтвердить, что цели процесса верификации ПО для установленного уровня ПО достигнуты для новых условий применения с нужным уровнем строгости и независимости.

d. Для создания данных ЖЦ ПО, которые отсутствуют или не соответствуют целям настоящего документа, может быть использован метод обратного проектирования. Кроме получения программного продукта может потребоваться выполнение дополнительных мероприятий для достижения целей процесса верификации ПО.

e. Если для достижения целей настоящего документа при доработке базовой версии планируется использовать сведения об истории эксплуатации изделия, то необходимо учитывать рекомендации, изложенные в разделе 12.3.4.

f. В Плане Сертификации ПО Заявитель должен описать стратегию достижения соответствия настоящему документу.

12.1.5. Аспекты, связанные с управлением конфигурацией ПО

Если используется ранее разработанное ПО, то мероприятия процесса УКПО для новых условий применения, в дополнение к мероприятиям раздела 7, должны включать:

a. Обеспечение трассируемости программного продукта и данных ЖЦ ПО от прежних условий применения к новым условиям применения.

b. Обеспечение управления изменениями, которое позволяет регистрировать проблемы, принимать решения по ним и отслеживать изменения в компонентах ПО, использующихся более чем в одних условиях применения.

12.1.6. Аспекты, связанные с гарантией качества ПО

Если используется ранее разработанное ПО, то мероприятия процесса ГКПО, в дополнение к мероприятиям раздела 8, должны включать:

a. Обеспечение гарантии того, что в новых условиях применения компоненты ПО удовлетворяют или превосходят критерии ЖЦ для установленного уровня ПО.

b. Обеспечение гарантии того, что изменения в процессах ЖЦ ПО указаны в планах создания ПО.

12.2. Квалификация инструмента

12.2.1. Определение необходимости квалификации инструмента

Квалификация инструмента требуется в тех случаях, когда процессы, указанные в настоящем документе, исключаются, упрощаются или автоматизируются за счет применения инструмента без верификации его выходных результатов согласно рекомендациям раздела 6.

Процесс квалификации инструмента нужен, чтобы подтвердить, что данный инструмент обеспечивает уровень доверия, по крайней мере, эквивалентный тому, который обеспечивается исключаемым, упрощаемым или автоматизируемым процессом (процессами).

Процесс квалификации инструмента может применяться к одному инструменту, их набору или к одной или нескольким функциям инструмента. Для инструмента с несколькими функциями, если можно продемонстрировать их защиту, то квалификация требуется только для тех функций, которые используются для исключения, упрощения или автоматизации процессов ЖЦ ПО, и чьи выходные данные не верифицируются. Защита представляет собой использование механизма, гарантирующего, что одна функция инструмента не может неблагоприятно влиять на другую функцию.

Инструмент квалифицируется только для использования при разработке конкретной системы, когда намерение его использовать объявлено в Плане Сертификации ПО этой системы. Если инструмент, ранее квалифицированный для одной системы, предлагается использовать при разработке другой системы, то он должен быть повторно квалифицирован в контексте этой другой системы.

12.2.2. Определение уровня квалификации инструмента

Если требуется квалификация инструмента, то должно быть оценено влияние его использования в процессах ЖЦ ПО, чтобы определить уровень квалификации инструмента (TQL). Для определения влияния инструмента необходимо использовать приведенные ниже критерии:

a. Критерий 1: Инструмент, выходные данные которого являются частью бортового ПО, и поэтому он может быть источником ошибок.

b. Критерий 2: Инструмент, который автоматизирует процесс (процессы) верификации ПО, и поэтому может не обнаружить ошибку, и выходные данные которого используются для обоснования исключения или упрощения:

   1. Процесса (процессов) верификации, отличных от тех, которые автоматизируются данным инструментом, или

   2. Процесса (процессов) разработки, которые могут влиять на бортовое ПО.

c. Критерий 3: Инструмент, который, в пределах предполагаемого его использования, может не обнаружить ошибку.

Если инструмент исключает, упрощает или автоматизирует процессы, указанные в настоящем документе, и его выходные данные не верифицируются так, как указано в разделе 6, то его TQL определяется по Таблице 12-1. На основании использования инструмента и его потенциального влияния на процессы ЖЦ ПО определяются пять уровней квалификации, с TQL-1 по TQL-5. Уровень TQL-1 является наиболее строгим, а уровень TQL-5 – наименее строгим. Оценивая влияние конкретного инструмента, необходимо последовательно рассматривать критерии с 1 по 3. Уровень квалификации инструмента необходимо согласовать с сертифицирующим органом как можно раньше.

Таблица 12-1. Определение уровня квалификации инструмента

Уровень ПО	Критерии		
	1	2	3
A	TQL-1	TQL-4	TQL-5
B	TQL-2	TQL-4	TQL-5
C	TQL-3	TQL-5	TQL-5
D	TQL-4	TQL-5	TQL-5

12.2.3. Процесс квалификации инструмента

Цели, мероприятия, инструктивные материалы и данные ЖЦ, необходимые для каждого TQL, описываются в документе Р-330 «Руководство по квалификации программных инструментов».

12.3. Альтернативные методы

Некоторые методы не рассматривались в предыдущих разделах настоящего документа из-за их недостаточной зрелости в момент написания документа или из-за ограниченной применимости для бортового ПО. Настоящий документ не ставит своей целью ограничивать использование каких-либо существующих или будущих методов. Любой альтернативный метод из обсуждаемых в этом разделе может использоваться для достижения одной или более целей настоящего документа. Кроме того, альтернативные методы могут использоваться в поддержку друг друга.

Альтернативные методы нельзя рассматривать в отрыве от комплекса процессов разработки ПО. Усилия, затрачиваемые на получение сертификационного зачета альтернативного метода, зависят от уровня ПО и от влияния, которое оказывает этот альтернативный метод на процессы ЖЦ ПО. Инструктивные материалы по использованию альтернативных методов включают в себя:

a. Должно быть продемонстрировано, что альтернативный метод достигает целей настоящего документа или соответствующего дополнения.

b. Заявитель должен указать в Плане Сертификации ПО и согласовать с сертифицирующим органом:

1. Влияние предлагаемого метода на процессы разработки ПО.
2. Влияние предлагаемого метода на данные ЖЦ ПО.
3. Обоснование использования альтернативного метода, которое демонстрирует достижение целей обеспечения безопасности системы. Одним из способов обоснования является обоснование гарантии, в котором приведены подробные доводы в пользу связи свидетельств с заявлениями о соответствии целям обеспечения безопасности системы.

c. Обоснование должно быть подкреплено планами создания ПО, процессами, ожидаемыми результатами и свидетельствами, подтверждающими применение метода.

12.3.1. Тестирование по методу полного перебора

Существуют ситуации, когда компонент ПО бортовой системы или оборудования прост и изолирован настолько, что можно определить конечный полный набор входных и выходных данных. В таких случаях можно продемонстрировать, что тестирование по методу полного перебора входных данных может заменить одно или несколько мероприятий процесса верификации ПО, представленных в Разделе 6.

Мероприятия для данного альтернативного метода включают в себя:

a. Определение полной совокупности допустимых входных и выходных данных ПО.

b. Проведение анализа, подтверждающего изолированность входных данных ПО.

c. Выработка обоснования тестовых примеров и процедур с полным перебором входных данных.

d. Разработка тестовых примеров, процедур и получение результатов тестов.

12.3.2. Аспекты, связанные с верификацией многоверсионного разнородного ПО

Приведенные ниже инструктивные материалы относятся к процессу верификации ПО, применяемому к многоверсионному разнородному ПО. Если в связи с использованием многоверсионного разнородного ПО внесены изменения в процесс верификации, то необходимо предоставить свидетельства того, что цели процесса верификации ПО достигнуты, и что эквивалентный уровень обнаружения ошибок достигается для каждой версии ПО.

Несколько разнородных версий ПО создается с помощью комбинаций следующих методов:

- Исходный Код реализуется на двух или более различных языках программирования.
- Объектный код генерируется двумя или более различными компиляторами.
- Каждая версия Исполняемого Объектного Кода выполняется на своем процессоре (при этом процессоры должны быть разнородными) или на одном процессоре, но с помощью средств, обеспечивающих обособление версий ПО.
- Требования к ПО, проект ПО и (или) Исходный Код разрабатываются двумя или более группами разработчиков, взаимодействие которых координируется.
- Требования к ПО, проект ПО и (или) Исходный Код разрабатываются с использованием двух или более сред разработки ПО, и (или) каждая версия верифицируется с использованием отдельных сред тестирования.
- Исполняемый Объектный Код линкуется и загружается двумя или более различными линкерами и двумя или более различными загрузчиками.
- Требования к ПО, проект ПО и (или) Исходный Код разрабатываются в соответствии с двумя или более различными Стандартами на Разработку Требований к ПО, Стандартами на Проектирование ПО и (или) Стандартами на Кодирование ПО соответственно.

Если используется несколько версий ПО, то методы верификации могут отличаться от тех, которые используются для верификации однородного ПО. Такие методы будут применяться к мероприятиям разработки ПО, выполняющимся параллельно, например, разными группами разработчиков. Процесс верификации ПО зависит от сочетания архитектур аппаратуры и ПО, поскольку они влияют на разнородность версий ПО. Дополнительными целями процесса верификации ПО, которые при этом необходимо достичь, является демонстрация того, что:

a. Удовлетворены требования к совместимости между версиями, включая совместимость при нормальном и ненормальном поведении, а также при переходах между состояниями.

b. Достигается эквивалентное обнаружение ошибок.

С сертифицирующим органом могут быть согласованы и другие изменения в процессе верификации ПО, если эти изменения обосновываются доказательством того, что достигается эквивалентное покрытие целей процесса верификации ПО.

12.3.2.1. Независимость многоверсионного разнородного ПО

Если версии многоверсионного разнородного ПО разрабатываются независимо с использованием некоторого единого метода, то появляется возможность выявить в процессе разработки определенные классы ошибок, таких, что верификация каждой версии ПО будет эквивалентна независимой верификации процессов разработки ПО. Мероприятия включают:

a. Заявитель должен продемонстрировать, что разные группы специалистов с ограниченным взаимодействием разработали требования к ПО, проект ПО и Исходной Код для каждой версии ПО.

b. Должны быть выполнены независимые анализы полноты тестового покрытия, как и для случая с одной версией.

Примечание: Раздел 12.3.2.1 рассматривает только вопрос независимости. Понижение уровней ПО не рассматривается и не предполагается.

12.3.2.2. Верификация ПО, реализуемого на нескольких процессорах

В случае, когда версии разнородного ПО выполняются на разных типах процессоров, верификация некоторых аспектов совместимости процессора и кода (см. раздел 6.4.3.a) может быть заменена верификацией, которая подтверждает, что процессоры разных типов формируют правильные выходные данные. Эта верификация состоит из интеграционных тестов, в которых выходные данные разных процессоров перекрестно сравниваются между собой в тестовых примерах, основанных на требованиях. Заявитель должен выполнить следующие мероприятия:

a. Продемонстрировать, что достигается эквивалентное обнаружение ошибок.

b. Продемонстрировать, что каждый процессор был спроектирован отдельным разработчиком.

c. Продемонстрировать, что выходные данные разных версий ПО эквивалентны.

12.3.2.3. Верификация многоверсионного исходного кода

Для систем или оборудования, использующих многоверсионное разнородное ПО в инструктивный материал по анализу структурного покрытия (см. 6.4.4.2) могут быть внесены изменения. Для анализа структурного покрытия соответствующее мероприятие Уровня А (см. 6.4.4.2.b) по оценке дополнительного объектного кода (созданного компилятором, линкером или иными средствами), не трассируемого напрямую на Исходный Код, не требуется, при условии, что Заявитель выполнит следующие мероприятия:

a. Продемонстрирует, что версии ПО закодированы на разных языках программирования.

b. Продемонстрирует, что каждый используемый компилятор создан независимым разработчиком.

12.3.2.4. Квалификация инструментов для многоверсионного разнородного ПО

Если используется многоверсионное разнородное ПО, то процесс квалификации инструментов может быть изменен, если имеются свидетельства того, что инструменты, используемые в процессе разработки ПО, являются разнородными. Это зависит от демонстрации эквивалентного мероприятия процесса верификации ПО при разработке версий разнородного ПО с помощью разнородных инструментов. Заявителю необходимо выполнить следующие мероприятия:

a. Продемонстрировать, что каждый инструмент был получен от отдельного разработчика.

b. Продемонстрировать, что инструменты имеют различную архитектуру.

12.3.2.5. Применение нескольких симуляторов при верификации

Если для верификации многоверсионного разнородного ПО используются отдельные, разнородные симуляторы, то может быть изменен подход к квалификации симуляторов. Это зависит от демонстрации эквивалентного мероприятия процесса верификации ПО при симулировании нескольких версий ПО с использованием нескольких симуляторов. За исключением случаев, когда удается обосновать, что не нужна разнородность нескольких симуляторов, Заявителю необходимо выполнить следующие мероприятия:

a. Предоставить свидетельства того, что каждый симулятор разработан отдельной группой.

b. Предоставить свидетельства того, что каждый симулятор разработан с использованием различных требований, архитектуры и языка программирования.

c. Предоставить свидетельства того, что симуляторы выполняются на разных процессорах.

Примечание: Если в многопроцессорной системе, использующей несколько разнородных версий ПО, установлены одинаковые процессоры, могут возникнуть трудности в демонстрации разнородности симуляторов из-за использования информации, получаемой из общего источника, каковым, например, является разработчик процессора.

12.3.3. Модели надежности ПО

Опубликовано много методов расчета надежности ПО, основанных на метриках разработки, таких как структура ПО, частота обнаружения дефектов и т.д. Настоящий документ не содержит инструктивные материалы по таким методам, поскольку на момент его написания эти методы не обеспечивали результатов, на которые можно положиться.

12.3.4. История эксплуатации изделия

Если на основе истории эксплуатации изделия, содержащего данное ПО, можно продемонстрировать, что обеспечивается эквивалентный уровень безопасности ПО, то это может быть учтено при получении сертификационного зачета. Приемлемость этого метода зависит от:

- Управления конфигурацией ПО.
- Эффективности мероприятий регистрации проблем.
- Стабильности и зрелости ПО.
- Релевантности среды истории эксплуатации изделия.
- Продолжительности истории эксплуатации изделия.
- Частоты появления ошибок в истории эксплуатации изделия.
- Влияния модификаций.

Использование данных истории эксплуатации для получения сертификационного зачета основывается на достаточности, релевантности и типах проблем, возникающих в период регистрации истории эксплуатации.

Процессы системы, включая процесс оценки безопасности системы, должны определять, оценивать и предоставлять соответствующему сертифицирующему органу способ и условия использования и результаты истории эксплуатации ПО. Ниже представлены инструктивные материалы по определению применимости истории эксплуатации и необходимой продолжительности истории эксплуатации.

12.3.4.1. Релевантность истории эксплуатации

Для установления релевантности истории эксплуатации применяются следующие критерии:

a. Тип истории эксплуатации: История эксплуатации, которую планируется использовать, должна быть определена и должна получить одобрение сертифицирующего органа. Данные истории эксплуатации должны быть представлены с использованием единиц измерения, релевантных условиям эксплуатации системы. Например, часы налета являются адекватной единицей измерения для ПО, используемого в течение полета непрерывно, такого как система управления полетом; количество обращений является адекватной единицей измерения для ПО, которое исполняется по запросу, такого как ПО управления шасси.

b. Известная конфигурация: Заявитель должен продемонстрировать, что ПО и свидетельства, используемые для подтверждения соответствия целям обеспечения безопасности системы, были под управлением конфигурацией в течение истории эксплуатации изделия.

c. Процесс сбора данных о времени функционирования: Заявитель должен продемонстрировать, что средства сбора и расчета летных часов (для ПО, используемого в течение полета непрерывно) или количества обращений (для ПО, исполняемого по запросу) являются достаточно точными и полными. Летные часы или количество обращений должны подсчитываться с учетом изменения любых факторов, которые важны для применения по назначению, включая, но не ограничиваясь следующим:

    1. Конфигурация ПО и системы.
    2. Рабочий режим или состояние.
    3. Среда эксплуатации.

d. Изменения ПО: Изменения конфигурации в течение истории эксплуатации изделия должны быть идентифицированы. Нужно провести анализ, чтобы определить, не привели ли изменения в ПО к изменению применимости данных по истории эксплуатации за период, предшествующий изменениям.

e. Использование и среда: Чтобы продемонстрировать релевантность истории эксплуатации изделия, должно быть проанализировано предлагаемое использование ПО.

    1. Должно быть гарантировано, что планируемые к использованию возможности ПО выполняются во всех режимах работы.
    2. Должен быть проведен анализ для гарантии того, что проверены релевантные перестановки входных данных.
    3. Среда эксплуатации, в которой были собраны данные истории эксплуатации, должна быть оценена для подтверждения релевантности среде в предполагаемом применении. Если среды эксплуатации существующего и предлагаемого применений отличаются, то должна быть проведена дополнительная верификация для подтверждения соответствия целям обеспечения безопасности системы в целевой среде.
    4. Если планируется получение зачёта в части совместимости с аппаратной средой, то должна быть рассмотрена преемственность между аппаратной средой, на которой получена история эксплуатации, и предполагаемой к применению аппаратной средой. Должно также быть оценено влияние любых модификаций аппаратуры в течение периода истории эксплуатации.

f. Отключенный код: Необходимо выполнить анализ, чтобы продемонстрировать, что любой код, который был отключен во время сбора данных истории эксплуатации, не активируется в новой среде. Если обнаруживается, что ранее отключенный код активируется в новой среде, необходимо выполнить дополнительную верификацию.

12.3.4.2. Достаточность собранной истории эксплуатации

Необходимое количество данных истории эксплуатации определяется:

a. Целями обеспечения безопасности системы в отношении ПО и уровнем ПО.

b. Любыми отличиями среды с историей эксплуатации и среды эксплуатации системы.

c. Целями разделов 4–9, отраженными в истории эксплуатации.

d. Свидетельствами, в дополнение к истории эксплуатации, относящимися к этим целям.

12.3.4.3. Сбор, регистрация и анализ проблем, обнаруженных в течение истории эксплуатации

a. Процесс регистрации проблем: Заявитель должен показать, что регистрация проблем в период истории эксплуатации изделия гарантирует наличие репрезентативных данных и что в течение периода истории эксплуатации сведения о проблемах поступали, регистрировались и доступны для извлечения.

1. Состав данных, подлежащих сбору, должен быть согласован с сертифицирующим органом, и эти данные, в дополнение к перечню, приведенному в разделе 11.17, должны включать следующее:
    i. Действующая на момент возникновения проблемы конфигурация аппаратуры и ПО.
    ii. Среда эксплуатации, в которой возникла проблема. iii. Режим работы или состояние, в которых возникла проблема.
    iv. Любая специфическая для условий применения информация, необходимая для оценки проблемы.
    v. Классификация проблемы по степени серьезности, значимости для безопасности, а также, была ли проблема результатом изменения в конфигурации ПО с момента начала сбора данных истории эксплуатации.
    vi. Оценка того, была ли проблема:
        - Воспроизводимой.
        - Устранимой.
        - Связанной с другими ранее зарегистрированными проблемами, в том числе имеющей общую причину с ними.
2. Должны быть оценены хронологические тенденции Сообщений о Проблемах и объяснены любые тенденции к росту.
3. Должна также быть учтена полнота истории ошибок ПО. Это включает в себя:
    i. Возможность обнаружения неисправностей и ведения журнала неисправностей.
    ii. Средства регистрации проблем, имеющиеся у оператора.
    iii. Полнота записей регистрации проблем.
    iv. Средства определения Заявителем влияния любых открытых Сообщений о Проблемах на безопасность системы и определения того, будут ли проблемы, не влиявшие на безопасность системы в течение истории эксплуатации, влиять на безопасность системы в планируемом

применении. Проблемы, не влиявшие на безопасность системы в течение истории эксплуатации, но которые будут влиять на безопасность системы в планируемом применении, могут указывать на необходимость дополнительной верификации.

v. Средства определения Заявителем количества возникновений конкретной проблемы.

b. Проблемы, связанные с процессом. Проблемы, свидетельствующие о неудовлетворительном выполнении процесса ЖЦ, такие как ошибки проекта или кода, должны быть указаны отдельно от тех, которые находятся вне области применения настоящего документа, таких как ошибки требований к системе или аппаратуре.

c. Проблемы, связанные с безопасностью. Все проблемы, выявленные при эксплуатации, должны быть оценены, чтобы определить, какие из них влияют на безопасность, и подтвердить, что все связанные с безопасностью проблемы были решены.

12.3.4.4. Информация периода эксплуатации, которая должна быть включена в План Сертификации ПО

При необходимости получить сертификационный зачет за счет истории эксплуатации необходимо включить в План Сертификации ПО и согласовать с сертифицирующим органом следующее:

a. Обоснование для заявления релевантной истории эксплуатации, согласно разделу 12.3.4.1.

b. Необходимый объем истории эксплуатации, вместе с обоснованием. Это включает пункты раздела 12.3.4.2, любые правила отбора данных, используемых для оценки, и измеряемые параметры (если есть). Данные предоставляются с использованием единиц изменения, релевантных условиям эксплуатации системы.

c. Обоснование расчета полного релевантного периода истории эксплуатации, включая такие факторы, как режимы работы, количество независимо работающих копий ПО, установленных и эксплуатируемых, а также определение понятий «нормальная эксплуатация» и «время нормальной эксплуатации».

Примечание: *Если частота возникновения ошибок выше, чем указанная в плане, такие ошибки необходимо проанализировать и результаты анализа рассмотреть вместе с сертифицирующим органом. Может потребоваться увеличение периода истории эксплуатации или история эксплуатации может оказаться неприменимой в качестве альтернативного метода определения соответствия.*

d. Определение того, что считалось ошибкой, и обоснование такого определения с учетом пунктов раздела 12.3.4.3a.

e. Предложенные приемлемые частоты возникновения ошибок и обоснование для периода истории эксплуатации изделия в отношении обеспечения безопасности системы и предложенных частот возникновения ошибок с учётом пунктов разделов 12.3.4.3b и 12.3.4.3c.

f. Определение критериев для проблем, делающих недействительной историю эксплуатации в свете раздела 12.3.4.3 или по иным причинам.

g. Критерии ошибок, которые будут исправляться, как они будут исправляться и верифицироваться, а также обоснование для тех ошибок, которые исправляться не будут.

h. Цели разделов 4–9, которые достигаются за счет использования истории эксплуатации.

ПРИЛОЖЕНИЕ А
ЦЕЛИ И ВЫХОДНЫЕ ДАННЫЕ ПРОЦЕССОВ В ЗАВИСИМОСТИ ОТ УРОВНЯ ПО

Ссылки в данных таблицах указывают на те разделы в тексте, в которых определяются конкретные цели, связанные с ними мероприятия и выходные данные.

Таблицы содержат инструктивный материал для:

a. Целей процесса, применимых к каждому уровню ПО. Для Уровня ПО “E” см. раздел 2.3.3.

b. Независимости мероприятий процессов ЖЦ ПО, проводимых для достижения целей этого процесса, по уровням ПО.

c. Категорий контроля данных, порождаемых мероприятиями процессов ЖЦ ПО, по уровням ПО (см. раздел 7.3).

Эти таблицы не должны быть использованы в качестве проверочных перечней. Эти таблицы не отражают все аспекты соответствия настоящему документу. Для полного понимания смысла инструктивных материалов необходимо изучить документ полностью.

Приведенная ниже расшифровка условных обозначений применима к колонкам “Применимо к Уровню ПО“ и “Категория Контроля для Уровня ПО“:

●	Цель должна быть достигнута с помощью независимого мероприятия.
O	Цель должна быть достигнута.
Пустая ячейка	Цель достигается по усмотрению Заявителя.
1	Данные достигают целей категории контроля 1 (КК1).
2	Данные достигают целей категории контроля 2 (КК2).

Таблица А-1. Процесс планирования ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Определены мероприятия процессов ЖЦ ПО	4.1.a	4.2.a 4.2.c 4.2.d 4.2.e 4.2.g 4.2.i 4.2.l 4.3.c	O	O	O	O	План Сертификации ПО План Разработки ПО План Верификации ПО План УКПО План ГКПО	11.1 11.2 11.3 11.4 11.5	1 1 1 1 1	1 1 1 1 1	1 2 2 2 2	1 2 2 2 2
2	Определены ЖЦ ПО, включая взаимосвязи между процессами, порядок их выполнения, механизмы обратной связи и критерии перехода	4.1.b	4.2.i 4.3.b	O	O	O		План Сертификации ПО План Разработки ПО План Верификации ПО План УКПО План ГКПО	11.1 11.2 11.3 11.4 11.5	1 1 1 1 1	1 1 1 1 1	1 2 2 2 2	
3	Выбрана и определена среда ЖЦ ПО	4.1.c	4.4.1 4.4.2.a 4.4.2.b 4.4.2.c 4.4.3	O	O	O		План Сертификации ПО План Разработки ПО План Верификации ПО План УКПО План ГКПО	11.1 11.2 11.3 11.4 11.5	1 1 1 1 1	1 1 1 1 1	1 2 2 2 2	
4	Рассмотрены дополнительные аспекты	4.1.d	4.2.f 4.2.h 4.2.i 4.2.j 4.2.k	O	O	O	O	План Сертификации ПО План Разработки ПО План Верификации ПО План УКПО План ГКПО	11.1 11.2 11.3 11.4 11.5	1 1 1 1 1	1 1 1 1 1	1 2 2 2 2	1 2 2 2 2
5	Определены стандарты на разработку ПО	4.1.e	4.2.b 4.2.g 4.5	O	O	O		Стандарты на Разработку Требований к ПО Стандарты на Проектирование ПО Стандарты на Кодирование ПО	11.6 11.7 11.8	1 1 1	1 1 1	2 2 2	
6	Планы создания ПО соответствуют настоящему документу	4.1.f	4.3.a 4.6	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
7	Разработка и пересмотр планов создания ПО скоординированы	4.1.g	4.2.g 4.6	O	O	O		Результаты Верификации ПО	11.14	2	2	2	

Таблица А-2. Процессы разработки ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Разработаны требования высокого уровня	5.1.1.a	5.1.2.a 5.1.2.b 5.1.2.c 5.1.2.d 5.1.2.e 5.1.2.f 5.1.2.g 5.1.2.j 5.5.a	O	O	O	O	Требования к ПО Данные Трассировки	11.9 11.21	1 1	1 1	1 1	1 1
2	Производные требования высокого уровня определены и переданы в процессы системы, включая процесс оценки безопасности системы	5.1.1.b	5.1.2.h 5.1.2.i	O	O	O	O	Требования к ПО	11.9	1	1	1	1
3	Разработана архитектура ПО	5.2.1.a	5.2.2.a 5.2.2.d	O	O	O	O	Описание Проекта ПО	11.10	1	1	1	2
4	Разработаны требования низкого уровня	5.2.1.a	5.2.2.a 5.2.2.e 5.2.2.f 5.2.2.g 5.2.3.a 5.2.3.b 5.2.4.a 5.2.4.b 5.2.4.c 5.5.b	O	O	O		Описание Проекта ПО Данные Трассировки	11.10 11.21	1 1	1 1	1 1	
5	Производные требования низкого уровня определены и переданы в процессы системы, включая процесс оценки безопасности системы	5.2.1.b	5.2.2.b 5.2.2.c	O	O	O		Описание Проекта ПО	11.10	1	1	1	
6	Разработан Исходный Код	5.3.1.a	5.3.2.a 5.3.2.b 5.3.2.c 5.3.2.d 5.5.c	O	O	O		Исходный Код Данные Трассировки	11.11 11.21	1 1	1 1	1 1	
7	Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных (если есть) созданы и загружены в целевой вычислитель	5.4.1.a	5.4.2.a 5.4.2.b 5.4.2.c 5.4.2.d 5.4.2.e 5.4.2.f	O	O	O	O	Исполняемый Объектный Код Файл Компонента Параметрических Данных	11.12 11.22	1 1	1 1	1 1	1

Таблица A-3. Верификация выходных данных процесса разработки требований к ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Требования высокого уровня соответствуют требованиям к системе	6.3.1.a	6.3.1	●	●	O	O	Результаты Верификации ПО	11.14	2	2	2	2
2	Требования высокого уровня точны и непротиворечивы	6.3.1.b	6.3.1	●	●	O	O	Результаты Верификации ПО	11.14	2	2	2	2
3	Требования высокого уровня совместимы с целевым вычислителем	6.3.1.c	6.3.1	O	O			Результаты Верификации ПО	11.14	2	2		
4	Требования высокого уровня верифицируемы	6.3.1.d	6.3.1	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
5	Требования высокого уровня соответствуют стандартам	6.3.1.e	6.3.1	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
6	Требования высокого уровня трассируемы на требования к системе	6.3.1.f	6.3.1	O	O	O	O	Результаты Верификации ПО	11.14	2	2	2	2
7	Алгоритмы точны	6.3.1.g	6.3.1	●	●	O		Результаты Верификации ПО	11.14	2	2	2	

Таблица А-4. Верификация выходных данных процесса проектирования ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Требования низкого уровня соответствуют требованиям высокого уровня	6.3.2.a	6.3.2	●	●	○		Результаты Верификации ПО	11.14	2	2	2	
2	Требования низкого уровня точны и непротиворечивы	6.3.2.b	6.3.2	●	●	○		Результаты Верификации ПО	11.14	2	2	2	
3	Требования низкого уровня совместимы с целевым вычислителем	6.3.2.c	6.3.2	○	○			Результаты Верификации ПО	11.14	2	2		
4	Требования низкого уровня верифицируемы	6.3.2.d	6.3.2	○	○			Результаты Верификации ПО	11.14	2	2		
5	Требования низкого уровня соответствуют стандартам	6.3.2.e	6.3.2	○	○	○		Результаты Верификации ПО	11.14	2	2	2	
6	Требования низкого уровня трассируемы на требования высокого уровня	6.3.2.f	6.3.2	○	○	○		Результаты Верификации ПО	11.14	2	2	2	
7	Алгоритмы точны	6.3.2.g	6.3.2	●	●	○		Результаты Верификации ПО	11.14	2	2	2	
8	Архитектура ПО совместима с требованиями высокого уровня	6.3.3.a	6.3.3	●	○	○		Результаты Верификации ПО	11.14	2	2	2	
9	Архитектура ПО непротиворечива	6.3.3.b	6.3.3	●	○	○		Результаты Верификации ПО	11.14	2	2	2	
10	Архитектура ПО совместима с целевым вычислителем	6.3.3.c	6.3.3	○	○			Результаты Верификации ПО	11.14	2	2		
11	Архитектура ПО верифицируема	6.3.3.d	6.3.3	○	○			Результаты Верификации ПО	11.14	2	2		
12	Архитектура ПО соответствует стандартам	6.3.3.e	6.3.3	○	○	○		Результаты Верификации ПО	11.14	2	2	2	
13	Целостность обособления ПО подтверждается	6.3.3.f	6.3.3	●	○	○	○	Результаты Верификации ПО	11.14	2	2	2	

Таблица A-5. Верификация выходных данных процессов кодирования и интеграции ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Исходный Код удовлетворяет требованиям низкого уровня	6.3.4.a	6.3.4	●	●	O		Результаты Верификации ПО	11.14	2	2	2	
2	Исходный Код соответствует архитектуре ПО	6.3.4.b	6.3.4	●	O	O		Результаты Верификации ПО	11.14	2	2	2	
3	Исходный Код верифицируем	6.3.4.c	6.3.4	O	O			Результаты Верификации ПО	11.14	2	2		
4	Исходный Код соответствует стандартам	6.3.4.d	6.3.4	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
5	Исходный Код трассируем на требования низкого уровня	6.3.4.e	6.3.4	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
6	Исходный Код точен и непротиворечив	6.3.4.f	6.3.4	●	O	O		Результаты Верификации ПО	11.14	2	2	2	
7	Результаты процесса интеграции ПО полны и правильны	6.3.5.a	6.3.5	O	O	O		Результаты Верификации ПО	11.14	2	2	2	
8	Файл Компонента Параметрических Данных правильный и полный	6.6.a	6.6	●	●	O	O	Примеры и Процедуры Верификации ПО Результаты Верификации ПО	11.13 11.14	1 2	1 2	2 2	2 2
9	Верификация Файла Компонента Параметрических Данных проведена	6.6.b	6.6	●	●	O		Результаты Верификации ПО	11.14	2	2	2	

Таблица A-6. Верификация выходных данных процесса интеграции (тестирование)

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Исполняемый Объектный Код удовлетворяет требованиям высокого уровня	6.4.a	6.4.2 6.4.2.1 6.4.3 6.5	O	O	O	O	Примеры и Процедуры Верификации ПО Результаты Верификации ПО Данные Трассировки	11.13 11.14 11.21	1 2 1	1 2 1	2 2 2	2 2 2
2	Исполняемый Объектный Код является робастным по отношению к требованиям высокого уровня	6.4.b	6.4.2 6.4.2.2 6.4.3 6.5	O	O	O	O	Примеры и Процедуры Верификации ПО Результаты Верификации ПО Данные Трассировки	11.13 11.14 11.21	1 2 1	1 2 1	2 2 2	2 2 2
3	Исполняемый Объектный Код удовлетворяет требованиям низкого уровня	6.4.c	6.4.2 6.4.2.1 6.4.3 6.5	●	●	O		Примеры и Процедуры Верификации ПО Результаты Верификации ПО Данные Трассировки	11.13 11.14 11.21	1 2 1	1 2 1	2 2 2	
4	Исполняемый Объектный Код является робастным по отношению к требованиям низкого уровня	6.4.d	6.4.2 6.4.2.2 6.4.3 6.5	●	O	O		Примеры и Процедуры Верификации ПО Результаты Верификации ПО Данные Трассировки	11.13 11.14 11.21	1 2 1	1 2 1	2 2 2	
5	Исполняемый Объектный Код совместим с целевым вычислителем	6.4.e	6.4.1.a 6.4.3.a	O	O	O	O	Примеры и Процедуры Верификации ПО Результаты Верификации ПО	11.13 11.14	1 2	1 2	2 2	2 2

Таблица А-7. Верификация выходных данных процесса верификации ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Тестовые процедуры правильны	6.4.5.b	6.4.5	●	O	O		Результаты Верификации ПО	11.14	2	2	2	
2	Результаты тестов правильны, расхождения объяснены	6.4.5.c	6.4.5	●	O	O		Результаты Верификации ПО	11.14	2	2	2	
3	Достигнуто покрытие тестами требований высокого уровня	6.4.4.a	6.4.4.1	●	O	O	O	Результаты Верификации ПО	11.14	2	2	2	2
4	Достигнуто покрытие тестами требований низкого уровня	6.4.4.b	6.4.4.1	●	O	O		Результаты Верификации ПО	11.14	2	2	2	
5	Достигнуто покрытие тестами структуры ПО – MC/DC	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3	●				Результаты Верификации ПО	11.14	2			
6	Достигнуто покрытие тестами структуры ПО – покрытие решений	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3	●	●			Результаты Верификации ПО	11.14	2	2		
7	Достигнуто покрытие тестами структуры ПО – покрытие операторов	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3	●	●	O		Результаты Верификации ПО	11.14	2	2	2	
8	Достигнуто покрытие тестами структуры ПО – связность по управлению и данным	6.4.4.d	6.4.4.2.c 6.4.4.2.d 6.4.4.3	●	●	O		Результаты Верификации ПО	11.14	2	2	2	
9	Проведена верификация дополнительного кода, не трассируемого на исходный код	6.4.4.c	6.4.4.2.b	●				Результаты Верификации ПО	11.14	2			

Таблица А-8. Процесс управления конфигурацией ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Единицы конфигурации идентифицированы	7.1.a	7.2.1	O	O	O	O	Записи УКПО	11.18	2	2	2	2
2	Базовые версии и трассируемость установлены	7.1.b	7.2.2	O	O	O	O	Указатель Конфигурации ПО Записи УКПО	11.16 11.18	1 2	1 2	1 2	1 2
3	Организованы регистрация проблем, управление изменениями, рассмотрение изменений и учет состояния конфигурации	7.1.c 7.1.d 7.1.e 7.1.f	7.2.3 7.2.4 7.2.5 7.2.6	O	O	O	O	Сообщения о Проблемах Записи УКПО	11.17 11.18	2 2	2 2	2 2	2 2
4	Организованы хранение, воспроизведение и выпуск	7.1.g	7.2.7	O	O	O	O	Записи УКПО	11.18	2	2	2	2
5	Организован контроль загрузки ПО	7.1.h	7.4	O	O	O	O	Записи УКПО	11.18	2	2	2	2
6	Организован контроль среды ЖЦ ПО	7.1.i	7.5	O	O	O	O	Указатель Конфигурации Среды ЖЦ ПО Записи УКПО	11.15 11.18	1 2	1 2	1 2	2 2

Таблица А-9. Процесс гарантии качества ПО

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Получена гарантия того, что планы и стандарты создания ПО разработаны и рассмотрены на предмет соответствия друг другу и настоящему документу	8.1.a	8.2.b 8.2.h 8.2.i	●	●	●		Записи ГКПО	11.19	2	2	2	
2	Получена гарантия того, что процессы ЖЦ ПО соответствуют утвержденным планам создания ПО	8.1.b	8.2.a 8.2.c 8.2.d 8.2.f 8.2.h 8.2.i	●	●	●	●	Записи ГКПО	11.19	2	2	2	2
3	Получена гарантия того, что процессы ЖЦ ПО соответствуют утвержденным стандартам	8.1.b	8.2.a 8.2.c 8.2.d 8.2.f 8.2.h 8.2.i	●	●	●		Записи ГКПО	11.19	2	2	2	
4	Получена гарантия того, что критерии перехода для процессов ЖЦ ПО удовлетворяются	8.1.c	8.2.e 8.2.h 8.2.i	●	●	●		Записи ГКПО	11.19	2	2	2	
5	Получена гарантия того, что проводится рассмотрение соответствия ПО	8.1.d	8.2.g 8.2.h 8.3	●	●	●	●	Записи ГКПО	11.19	2	2	2	2

Таблица А-10. Процесс взаимодействия Заявителя с сертифицирующим органом

	Цель		Мероприятие	Применимо к уровню ПО				Выходные данные		Категория контроля для уровня ПО			
№	Описание цели	Пункт	Пункт	A	B	C	D	Наименование	Пункт	A	B	C	D
1	Установлены связь и взаимопонимание между Заявителем и сертифицирующим органом	9.a	9.1.b 9.1.c	O	O	O	O	План Сертификации ПО	11.1	1	1	1	1
2	Методы определения соответствия предложены, План сертификации ПО согласован	9.b	9.1.a 9.1.b 9.1.c	O	O	O	O	План Сертификации ПО	11.1	1	1	1	1
3	Доказательство соответствия приведено	9.c	9.2.a 9.2.b 9.2.c	O	O	O	O	Итоговое Заключение о ПО Указатель Конфигурации ПО	11.20 11.16	1 1	1 1	1 1	1 1

ПРИЛОЖЕНИЕ В
СОКРАЩЕНИЯ И СЛОВАРЬ ТЕРМИНОВ

Сокращение (англ.)	Значение (англ.)	Значение (рус.)	Сокращение (рус.)
ARP	Aerospace Recommended Practice	Рекомендованные методы аэрокосмической промышленности	
CC1	Control Category 1	Категория контроля 1	КК1
CC2	Control Category 2	Категория контроля 2	КК2
CRC	Cyclic Redundancy Check	Проверка с использованием циклического избыточного кода	
DO	Document	Документ	
EASA	European Aviation Safety Agency	Европейское агентство по авиационной безопасности	
EUROCAE	European Organization for Civil Aviation Equipment	Европейская организация по оборудованию для гражданской авиации	
FAA	Federal Aviation Administration	Федеральное управление гражданской авиации (США)	
MC/DC	Modified Condition/Decision Coverage	Модифицированное покрытие условий и решений	
PSAC	Plan for Software Aspects of Certification	План Сертификации ПО	
RTCA	RTCA, Inc.	Компания RTCA	
SCI	Software Configuration Index	Указатель Конфигурации ПО	
SCM	Software Configuration Management	Управление конфигурацией ПО	УКПО
SDP	Software Development Plan	План Разработки ПО	
SECI	Software Life Cycle Environment Configuration Index	Указатель Конфигурации Среды ЖЦ ПО	
SQA	Software Quality Assurance	Гарантия качества ПО	ГКПО
SVP	Software Verification Plan	План Верификации ПО	
SW	Software	Программное обеспечение	ПО
TQL	Tool Qualification Level	Уровень квалификации инструмента	
	Life Cycle	Жизненный цикл	ЖЦ
	Aircraft	Воздушное судно	ВС

СЛОВАРЬ ТЕРМИНОВ

Приведенные ниже определения даны для терминов, используемых в тексте этого документа. Если термин не определен в данном приложении, возможно, что его определение дано в тексте настоящего документа.

Алгоритм (Algorithm) – Конечный набор точно определенных правил, указывающих последовательность действий для решения конкретной задачи.

Альтернативный метод (Alternative method) – Другой подход к достижению одной или нескольких целей настоящего документа.

Анализ покрытия (Coverage analysis) – Процесс определения того, в какой степени предложенные мероприятия процесса верификации ПО достигают его целей.

Анализ структурного покрытия (Structure coverage analysis) – Оценка структуры кода, включая интерфейсы, отработавшего во время тестирования на основе требований.

Архитектура ПО (Software architecture) – Структура ПО, выбранная для реализации требований к ПО.

Архитектура системы (System architecture) – Структура аппаратуры и программного обеспечения, выбранная для реализации требований к системе.

Аудит (Audit) – Независимая проверка процессов ЖЦ ПО и их выходных данных для подтверждения требуемых свойств. Аэронавигационные данные (Aeronautical data) – Данные, используемые для таких аэронавигационных применений, как навигация, планирование полета, симуляция полета, предупреждение столкновения с землей и для других целей. База данных (Database) – Набор данных, часть или целое другого набора данных, состоящий, по крайней мере, из одного файла, достаточный для определенного применения или для определенной системы обработки данных. Базовая версия (Baseline) – Утвержденная зарегистрированная конфигурация одной или нескольких единиц конфигурации, которая используется в качестве базы для дальнейшей разработки и которая изменяется только через процедуры управления изменениями.

Библиотека ПО (Software library) – Находящееся под контролем хранилище, в котором содержатся ПО и связанные с ним данные и документы, и предназначенное для поддержки разработки, применения или модификации ПО. Примерами являются: библиотека разработки ПО, главная библиотека, библиотека программ и хранилище ПО.

Бортовой (Airborne) – Определение, используемое для обозначения ПО, оборудования или систем на борту воздушного судна. Валидация (Validation) – Процесс установления того, что требования являются корректными и полными. При валидации системы в процессах ЖЦ системы могут использоваться требования к ПО и производные требования.

Верификация (Verification) – Оценка выходных данных процесса для подтверждения их корректности и соответствия входным данным и стандартам, определенным для этого процесса.

Встроенный идентификатор (Embedded identifier) – Атрибуты идентификации ПО, такие как дата создания, идентификационный номер, контрольная сумма или циклический избыточный код (CRC), используемые для проверки целостности линковки, или идентификатор версии, включаемые в целевой исполняемый объектный код.

Выпуск (Release) – Действие, формально открывающее доступ к воспроизводимой единице конфигурации и разрешающее ее использование.

Гарантия (Assurance) – Запланированные и систематические действия, необходимые для достижения соответствующего уровня доверия и получения свидетельств того, что изделие или процесс удовлетворяет определенным требованиям.

Гарантия разработки ПО (Software assurance) – Запланированные и систематические действия, необходимые для достижения уровня доверия и получения свидетельств того, что программный продукт или процесс удовлетворяет предъявленным требованиям.

Генератор автокода (Autocode generator) – Инструмент кодирования, который автоматически создает Исходный Код или объектный код по требованиям низкого уровня.

Готовое коммерческое ПО (Commercial-Off-The-Shelf (COTS) software) – Имеющиеся на рынке программные приложения, продаваемые поставщиками по общедоступным каталогам. Готовое коммерческое ПО не модифицируется в соответствии с требованиями заказчика и не усовершенствуется под конкретный заказ. ПО, созданное по контракту для конкретного применения, не является готовым коммерческим ПО.

Данные истории эксплуатации (Service history data) – Данные, собираемые в течение периода эксплуатации. Данные трассировки (Trace data) – Данные, свидетельствующие о трассируемости данных процессов разработки и верификации ПО. Данные трассировки не обязательно представляют собой специально созданные конкретные объекты. Данные трассировки могут показывать связи, например, посредством соглашения об именах, либо с помощью ссылок или указателей, как встраиваемых в данные ЖЦ ПО, так и внешних по отношению к ним.

Дополнение (Supplement) – Инструктивные материалы, используемые совместно с настоящим документом, рассматривающие уникальные особенности конкретного подхода, метода или приема. Дополнение добавляет, исключает или иным образом вносит изменения в цели, мероприятия, пояснительный текст и данные ЖЦ ПО, представленные в настоящем документе.

Единица конфигурации (Configuration item) – (1) Один или более аппаратных или программных компонентов, которые в целях управления конфигурацией рассматриваются как единое целое. (2) Данные ЖЦ ПО, которые в целях управления конфигурацией рассматриваются как единое целое.

Единичный сбой (Single event upset) - Случайное изменение бита данных, которое может произойти в аппаратуре. Жизненный цикл ПО (Software life cycle) – (1) Упорядоченная совокупность процессов, определенная организацией как достаточная и адекватная для создания программного продукта. (2) Период времени от момента принятия решения о создании или модификации программного продукта до момента снятия его с эксплуатации.

Задача (Task) – Основная единица работы с точки зрения управляющей программы.

«Заплата» (Patch) – Модификация Исполняемого Объектного Кода, в ходе которой пропускается один или несколько запланированных шагов перекомпиляции, переассемблирования или перелинковки. Изменение встроенных идентификаторов не считается «заплатой».

Заявитель (Applicant) – Лицо или организация, запрашивающие одобрение от сертифицирующего органа.

Идентификационный номер (Part number) – Набор чисел, букв или иных символов, используемый для идентификации единицы конфигурации.

Идентификация конфигурации (Configuration identification) – (1) Процесс обозначения единиц конфигурации в системе и регистрации их характеристик. (2) Утвержденная документация, которая определяет единицу конфигурации.

Изменение ПО (Software change) – Изменение по сравнению с базовой версией в Исходном Коде, объектном коде, Исполняемом Объектном Коде или в соответствующей документации.

Инструмент (Software tool) – Программа, используемая для поддержки разработки, тестирования, анализа, создания или модификации другой программы или ее документации. Примерами таких инструментов являются инструмент автоматизированного проектирования, компилятор, инструменты тестирования и инструменты для внесения изменений.

Инструментальный компьютер (Host computer) – Компьютер, используемый для разработки ПО.

Интегральный процесс (Integral process) – Процесс, который поддерживает процессы разработки ПО и другие интегральные процессы и который, по этой причине, выполняется в течение всего ЖЦ ПО.

Интегральными процессами являются: процесс верификации ПО, процесс гарантии качества ПО, процесс управления конфигурацией ПО и процесс взаимодействия Заявителя с сертифицирующим органом.

Интеграция аппаратуры и ПО (Hardware/software integration) – Процесс встраивания ПО в целевой вычислитель.

Интеграция ПО (Software integration) – Процесс объединения компонентов кода.

Исполняемый Объектный Код (Executable Object Code) – Форма кода, которая может быть напрямую использована процессором целевого вычислителя и является, следовательно, скомпилированным, ассемблированным и слинкованным двоичным образом, который загружается в целевой вычислитель. История эксплуатации изделия (Product service history) – Непрерывный период времени, в течение которого ПО эксплуатируется в рамках известной среды и в течение которого регистрируются последовательные отказы. Исходный Код (Source Code) – Текст компьютерной программы на языке Ассемблера и (или) языках программирования высокого уровня в форме, пригодной для машинного считывания ассемблером или компилятором. Категория контроля (Control category) – Набор процедур управления конфигурацией, установленный для данных ЖЦ ПО. Две категории, КК1 и КК2, определяют процессы и мероприятия управления конфигурацией ПО, применяемые для управления данными ЖЦ ПО.

Квалификация инструмента (Tool qualification) – Процесс, необходимый для получения сертификационного зачета для программного инструмента применительно к конкретной бортовой системе.

Класс эквивалентности (Equivalence class) – Такое подмножество значений входных данных программы, что тестирование, выполненное для репрезентативного значения из этого класса, эквивалентно тестированию с другими значениями из этого класса.

Код (Code) – Представление конкретных данных или конкретной компьютерной программы в символьной форме, такой как Исходный Код, объектный код или машинный код.

Компактные выражения (Compacted expressions) – Представления Исходного Кода, в которых в одном выражении объединена совокупность нескольких конструкций языка программирования.

Компилятор (Compiler) – Программа, транслирующая операторы Исходного Кода, записанные на языке высокого уровня, таком как FORTRAN или Pascal, в объектный код.

Компонент (Component) – Автономная часть, комбинация частей, узлов или модулей, выполняющая отдельную функцию системы.

Компонент параметрических данных (Parameter data item) – Набор данных, которые, будучи представленными в форме Файла Компонента Параметрических Данных, влияют на работу ПО без

изменения Исполняемого Объектного Кода и которые управляются как отдельная единица конфигурации. Примерами являются базы данных и таблицы конфигураций.

Критерии перехода (Transition criteria) – Минимальные условия, определенные процессом планирования ПО, которые следует удовлетворить, чтобы перейти к какому-либо процессу.

Логическая операция (Boolean operator) – Операция с логическими операндами, которая дает логический результат.

Логическое выражение (Boolean expression) – Выражение, результатом вычисления которого может быть либо значение «Истина» («True»), либо значение «Ложь» («False»).

Мероприятие (Activity) – Задачи, являющиеся средствами достижениями цели.

Мертвый код (Dead code) – Исполняемый Объектный Код или данные, которые вследствие ошибки разработки ПО не могут быть выполнены (код) или использованы (данные) при любой нормальной конфигурации среды целевого вычислителя. Мертвый код не трассируется на системные требования и на требования к ПО. Приведенные ниже исключения необходимы для реализации проекта или требований, но часто ошибочно классифицируются как мертвый код: встроенные идентификаторы, структуры безопасного программирования для улучшения робастности, а также отключенный код, такой как неиспользуемые функции библиотеки. Методы определения соответствия (Means of compliance) – Методы, которые планирует использовать Заявитель для удовлетворения требований, установленных в сертификационном базисе для воздушного судна, двигателя или воздушного винта. Примерами являются спецификации, чертежи, анализы, расчеты, испытания, тестирование, моделирование, инспекции, и квалификация комплектующего изделия на внешние воздействия. Если это применимо, используется рекомендательный материал, выпущенный сертифицирующим органом.

Многоверсионное разнородное ПО (Multiple-version dissimilar software) – Два или более программных компонента, которые удовлетворяют одним и тем же функциональным требованиям, но специально созданы отличающимися друг от друга. Примерами такого подхода являются использование отдельных организаций-разработчиков или использование различных методов разработки. За счет применения многоверсионного разнородного ПО можно свести к минимуму ошибки общего режима.

Модифицированное покрытие условий и решений (Modified condition/decision coverage) – Каждая точка входа и выхода в программе пройдена хотя бы раз, каждое условие в каждом решении в программе приняло все возможные значения хотя бы раз, каждое решение в программе приняло все возможные значения хотя бы один раз, а также показано, что каждое условие в решении независимо влияет на исход данного решения. Независимое влияние условия на исход решения демонстрируется посредством (1) изменения только этого условия при неизменных всех других возможных условиях или (2) изменения только этого условия при неизменных всех других возможных условиях, которые могут влиять на исход. Модифицируемое пользователем ПО (User-modified software) – ПО, которое можно модифицировать без согласования с сертифицирующим органом, самолетостроителем или поставщиком оборудования, если модификация происходит в пределах, установленных в исходном проекте сертификации.

Мониторинг (Monitoring) – Действие по освидетельствованию или инспекционной проверке выбранных тестов, инспекций или других мероприятий, а также записей этих мероприятий, для того чтобы гарантировать, что мероприятие находится под контролем и что зарегистрированные результаты соответствуют ожидаемым. Мониторинг обычно связан с мероприятиями, выполняемыми в течение длительного периода времени, когда 100-процентное освидетельствование считается невозможным или ненужным. Мониторинг позволяет получить официальное удостоверение того, что заявленное мероприятие выполнено так, как планировалось.

Мониторинг безопасности (Safety monitoring) – Метод защиты от определенных отказных состояний с помощью непосредственного мониторинга функции на предмет наличия отказов, которые могут привести к отказному состоянию. Независимость (Independence) – Разделение ответственности, гарантирующее выполнение объективной оценки. (1) Для мероприятий процесса верификации ПО независимость достигается тогда, когда верификационное мероприятие выполняется лицом или лицами, не участвовавшими в разработке верифицируемой единицы, а также инструментом (или инструментами), обеспечивающими эквивалентные человеку верификационные действия. (2) Для процесса гарантии качества независимость также включает полномочия для обеспечения выполнения корректирующих действий.

Неисправность (Fault) – Проявление ошибки в ПО. Неисправность, если она возникает, может явиться причиной отказа.

Ненормальное поведение (Anomalous behavior) – Поведение, которое не соответствует заданным требованиям. Неограниченный рекурсивный алгоритм (Unbounded recursive algorithm) – Алгоритм, который запускает сам себя непосредственно (саморекурсия) или неявно (взаимная рекурсия) и не имеет механизма, ограничивающего количество таких запусков.

Носители (Media) – Устройства или материалы, которые являются средствами переноса или хранения ПО, например, программируемые постоянные запоминающие устройства, магнитные ленты или диски, бумага.

Обособление (Partitioning) – Метод обеспечения изоляции компонентов ПО для локализации и (или) изоляции отказов.

Обособление ПО (Software partitioning) – Процесс разделения, обычно с выраженной целью изоляции одной или более частей ПО для того, чтобы предотвратить специфическое нежелательное взаимодействие или перекрестную связность.

Обратная разработка (Reverse engineering) – Процесс разработки данных ПО более высокого уровня из имеющихся данных более низкого уровня. Примерами являются разработка Исходного Кода из объектного кода или из Исполняемого Объектного Кода, разработка требований высокого уровня на основании требований низкого уровня.

Объектный код (Object code) – Низкоуровневое представление компьютерной программы в форме, которая обычно непригодна для непосредственного использования в вычислителе, но которая дополнительно к командам процессора включает, в том числе, информацию для размещения.

Одобрение (Approval) – Действие или факт формального признания или официального разрешения. Опыт эксплуатации (Service experience) - Интервалы времени, в течение которых ПО эксплуатируется в рамках известной релевантной и контролируемой среды и в течение которых регистрируются последовательные отказы.

Отказ (Failure) – Неспособность системы или ее компонента выполнять требуемую функцию в пределах заданных ограничений. Отказ может быть результатом проявления неисправности.

Отказное состояние (Failure condition) – Состояние, оказывающее непосредственное или косвенное влияние на ВС и находящихся на борту лиц, вызванное или усугублённое одним или несколькими отказами или сбоями, с учётом фазы полёта, соответствующих неблагоприятных условий эксплуатации или среды, а также внешних воздействий. Примечание: Под отказным состоянием понимается состояние системы, характеризуемое конкретным нарушением ее функций независимо от причин, вызывающих это состояние. Отказное состояние определяется на уровне каждой системы через последствия, оказываемые им на функционирование этой системы. Оно также характеризуется влиянием на другие системы и на самолет в целом (РЦ 25.1309).

Отключенный код (Deactivated code) – Исполняемый Объектный Код (или данные), трассируемый на требования и, в соответствие с проектом, либо (a) не предназначенный для исполнения (код) или использования (данные), например, часть ранее разработанного компонента ПО, такая как неиспользуемый унаследованный код, неиспользуемые функции библиотеки или код, предназначенный для последующего наращивания, либо (b) исполняемый (код) или используемый (данные) только в конкретных конфигурациях среды целевого вычислителя, например, код, который включается выбором контакта в аппаратуре или программируемой опцией ПО. Приведенные ниже примеры часто ошибочно классифицируются как отключенный код, хотя их следует рассматривать как необходимые для реализации проекта или требований: структуры безопасного программирования, используемые для улучшения робастности, включая вставляемый компилятором код для проверок диапазонов и индексов массивов, подпрограмм обработки ошибок и исключений, проверки границ и допустимости, средства управления очередями, отметки времени.

Ошибка (Error) – Применительно к ПО – ошибка в требованиях, проекте или коде.

Повторная верификация (Reverification) - Оценка результатов процесса модификации, например, коррекции ошибок или введения новой или дополнительной функциональности для подтверждения правильности по отношению к входным данным и согласованности со стандартами этого процесса.

Покрытие операторов (Statement coverage) - Каждый оператор программы был вызван хотя бы один раз. **Примечание**: *Что считать оператором, определяется языком программирования.*

Покрытие решений (Decision coverage) – Каждая точка входа и выхода в программе пройдена хотя бы раз, а также каждое решение в программе приняло все возможные значения хотя бы один раз.

Посторонний код (Extraneous code) – Код (или данные), которые не трассируемы на требования к системе или к ПО. Примером постороннего кода является унаследованный код, который ошибочно был сохранен, хотя требования к нему и его тестовые примеры были удалены. Другим примером постороннего кода является мертвый код.

Прерывание (Interrupt) – Приостановка задачи (такой как выполнение компьютерной программы), вызванная внешним по отношению к этой задаче событием и выполняемая таким образом, чтобы сохранить возможность возобновления выполнения задачи.

Программное обеспечение (Software) – Программы вычислителя и, возможно, связанные с ними документация и данные, имеющие отношение к работе вычислительной системы.

Программный продукт (Software product) – Набор программ вычислителя, соответствующие документация и данные, предназначенные для поставки пользователю. В контексте настоящего документа этот термин относится к ПО, предназначенному для использования в бортовом оборудовании, и к соответствующим данным ЖЦ ПО.

Производные требования (Derived requirements) – Требования, создаваемые процессами разработки ПО, которые (a) не трассируются непосредственно на требования более высокого уровня, и (или) (b) описывают особенности поведения, которые не определяются требованиями к системе или требованиями к ПО более высокого уровня.

Процесс (Process) – Совокупность мероприятий, выполняемых в ЖЦ ПО для получения определенных выходных данных или результата.

Процесс взаимодействия с сертифицирующим органом (Certification liaison process) – Процесс, обеспечивающий связь, взаимопонимание и достижение соглашения между Заявителем и сертифицирующим органом. Процесс оценки безопасности системы (System safety assessment process) – Постоянная, систематическая, комплексная оценка системы для демонстрации того, что требования по обеспечению

безопасности, удовлетворены. Основными мероприятиями в этом процессе являются оценка функциональных опасностей, предварительная оценка безопасности системы и оценка безопасности системы. Строгость мероприятий будет зависеть от критичности, сложности, новизны и соответствующего опыта эксплуатации рассматриваемой системы.

Разрешенный источник (Approved source) – Место, из которого следует воспроизводить данные ЖЦ ПО, указанное в Указателе Конфигурации ПО. Разрешенный источник может быть хранилищем системы управления конфигурацией ПО, электронным архивом или организацией, отличной от организации разработчика. Ранее разработанное ПО (Previously developed software) – ПО, уже разработанное для применения. Это охватывает широкий диапазон ПО, от готового коммерческого ПО до ПО, разработанного в соответствии с предыдущими или текущими инструктивными материалами.

Рассмотрение соответствия ПО (Software conformity review) – Рассмотрение, обычно выполняемое в конце цикла разработки ПО, с целью гарантии того, что процессы ЖЦ ПО выполнены, данные ЖЦ ПО являются полными, а Исполняемый Объектный Код находится под контролем и может быть сгенерирован повторно. Решение (Decision) – Логическое выражение, составленное из одного или более условий, связанных логическими операциями. Если некоторое условие появляется в решении более чем один раз, каждое его появление считается отдельным условием.

Робастность (Robustness) – Пределы, в которых ПО способно продолжать функционировать корректно, несмотря на ненормальные входные данные и условия.

Связность по данным (Data coupling) – Зависимость компонента ПО от данных, не находящихся под исключительным контролем данного компонента.

Связность по управлению (Control coupling) – Способ или степень влияния, оказываемого одним компонентом программы на работу другого компонента.

Сертификация (Certification) – Юридическое признание сертифицирующим органом того, что изделие, учреждение, организация или отдельный субъект удовлетворяют необходимым требованиям. Сертификация включает техническую проверку изделия, учреждения, организации или субъекта и формальное признание соответствия применяемым требованиям посредством оформления сертификата, лицензии, одобрения, или других документов, как это регламентировано внутригосударственными законами и процедурами. В частности сертификация изделия включает: (a) процесс оценки конструкции изделия, с целью гарантировать соответствие ряду требований, применимых к этому типу изделия для того, чтобы продемонстрировать допустимый уровень безопасности; (b) процесс оценки отдельного изделия для обеспечения соответствия сертифицированной типовой конструкции; (c) выдача сертификата, требующегося по внутригосударственным законам, для утверждения, что соответствие требованиям было обосновано в соответствии с приведенными выше пунктами (a) или (b).

Сертификационный зачет (Certification Credit) – Признание сертифицирующим органом того, что процесс, изделие или демонстрация удовлетворяют сертификационным требованиям.

Сертифицирующий орган (Certification authority) – Организация или лицо, несущие ответственность в соответствующем государстве за определенные требования. Вопросы, связанные с сертификацией типа воздушного судна, двигателя или воздушного винта или квалификацией оборудования, обычно решаются сертифицирующим органом; вопросы, связанные с продлением летной эксплуатации, могут рассматриваться органом контроля летной годности (имеющим различные названия).

Симулятор (Simulator) – Устройство, компьютерная программа или система, используемые при верификации ПО, которые принимают те же входные данные и формируют те же выходные данные, что и исходная система, используя для этого объектный код, полученный из оригинального объектного кода.

Система (System) – Совокупность аппаратных и программных компонентов, создаваемая для выполнения конкретной функции или набора функций.

Сквозная численная разрешающая способность (End-to-end numerical resolution) – Мера точности, получаемая в результате расчетов во всей интегрированной системе.

Словарь данных (Data dictionary) – Подробное описание данных, параметров, переменных и констант, используемых в системе. Стандарт (Standard) – Правило или основа для сравнения, используемое как инструктивные материалы по выполнению и оценке определенного мероприятия, а также содержимого определенной единицы данных. Стандарты на разработку ПО (Software development standards) – Стандарты, которые определяют правила и ограничения для процессов разработки ПО. Стандарты на разработку ПО включают в себя Стандарты на Разработку Требований к ПО, Стандарты на Проектирование ПО и Стандарты на Кодирование ПО.

Структура (Structure) – Определенное построение или взаимосвязь частей для образования единого целого.

Тестирование (Testing) – Процесс выполнения задания системой или ее компонентом для проверки соответствия заданным требованиям и для выявления ошибок.

Тестовый пример (Test case) – Набор тестовых входных данных, условий выполнения и ожидаемых результатов, разработанный с конкретной целью – выполнить тестирование конкретной ветви программы или проверить соответствие конкретному требованию.

Тестовая процедура (Test procedure) – Подробные инструкции по подготовке и выполнению заданного набора тестовых примеров, а также инструкции по оценке результатов их выполнения.

Типовая конструкция (Type design) - Для целей настоящего документа типовая конструкция состоит из: (1) чертежей и спецификаций, а также перечня чертежей и спецификаций, необходимых для определения конфигурации и конструктивных особенностей сертифицируемого изделия, для которых показано соответствие требованиям для данного изделия; (2) Любые другие данные, необходимые для того, чтобы дать возможность установить, путем сравнения, летную годность более поздних изделий того же самого типа. **Примечание**: *Использование в данном определении термина «изделие» относится к любому объекту, для которого сертифицирующий орган выдает сертификаты типа, например, воздушные суда, двигатели, винты.*

Трассируемость (Traceability) – Связь между объектами, например, между выходными данными процессов, между выходными данными и порождающим их процессом или между требованием и его реализацией.

Требования высокого уровня (High-level requirements) – Требования к ПО, разработанные на основе анализа требований к системе, требований к безопасности и архитектуры системы.

Требования к ПО (Software requirement) – Описание того, что должно порождать ПО при заданных входных данных и заданных ограничениях. Требования к ПО включают требования высокого и низкого уровней.

Требования низкого уровня (Low-level requirements) – Требования к ПО, разработанные на основе требований высокого уровня, производных требований и проектных ограничений, на основе которых можно непосредственно реализовать Исходный Код без привлечения дополнительной информации. Управление изменениями (Change control) – (1) Процесс регистрации, оценки, утверждения или отклонения и координации изменений в единицах конфигурации после формального введения их идентификации или в базовых версиях после их введения. (2) Систематическая оценка, координация, утверждение или отклонение, а также реализация утвержденных изменений конфигурации единицы конфигурации после формального введения идентификации этой конфигурации или базовых версий после их введения. **Примечание:** *в других документах может называться "Контролем конфигурации".*

Управление конфигурацией (Configuration management) – (1) Процесс (a) идентификации и определения единиц конфигурации системы, (b) контролирования выпуска и изменения этих единиц в течение ЖЦ ПО, (c) регистрации и составления отчетов о состоянии единиц конфигурации и Сообщений о Проблемах и (d) проверки полноты и корректности единиц конфигурации. (2) Порядок технического и административного руководства и надзора для (a) идентификации и регистрации функциональных и физических характеристик единиц конфигурации, (b) управления изменениями этих характеристик и (c) регистрации и составления отчетов о ходе процесса управления изменениями и о состоянии их реализации.

Управляющая программа (Control program) – Программа, предназначенная для диспетчеризации программ в вычислительной системе и для контроля их выполнения.

Уровень ПО (Software level) – Обозначение, присваиваемое компоненту ПО процессами системы. Уровень ПО устанавливает, насколько строго ПО должно соответствовать требованиям настоящего документа. **Примечание**: *Другие технические документы могут использовать различные термины для результата процесса оценки безопасности системы. Одним из примеров является термин «уровень гарантии разработки элемента» (IDAL – item development assurance level), который для ПО является синонимом термина «уровень ПО».*

Условие (Condition) – Логическое выражение, не содержащее логических операций за исключением унарной операции «Не» («NOT»).

Устойчивость к неисправностям (Fault tolerance) – Свойство системы сохранять непрерывное работоспособное состояние при ограниченном количестве неисправностей в аппаратуре или ПО.

Устройство памяти (Memory device) – Элемент аппаратуры, способный хранить машинно-читаемые компьютерные программы и относящиеся к ним данные. Примерами являются интегральные микросхемы, платы, содержащие интегральные микросхемы, память на магнитных сердечниках, диск, магнитная лента.
Учет состояния конфигурации (Configuration status accounting) – Регистрация и предоставление отчетности об информации, необходимой для эффективного управления конфигурацией, включая идентификацию утвержденной конфигурации, состояние предложенных изменений конфигурации и состояние реализации утвержденных изменений.

Файл Компонента Параметрических Данных (Parameter Data Item File) – Представление компонента параметрических данных, которое может напрямую использоваться процессором целевого вычислителя. Файл Компонента Параметрических Данных представляет собой реализацию компонента параметрических данных, содержащую определенные значения каждого элемента данных.

Формальные методы (Formal methods) – Описательная нотация и аналитические методы, используемые для составления, разработки и обоснования математических моделей поведения системы. Формальный метод представляет собой формальный анализ, выполняемый на формальной модели.

Целостность (Integrity) – Свойство системы или ее элемента, показывающее, что можно быть уверенным в том, что они работают корректно, когда требуется.

Цель (Objective) – Когда настоящий документ указывается в качестве метода определения соответствия нормам, цели представляют собой требования, которые должны быть выполнены для демонстрации соответствия. Эквивалентная безопасность (Equivalent safety) - Уровень безопасности, достигнутый с помощью альтернативного метода, для достижения как целей настоящего документа, так и целей обеспечения безопасности системы.

Эмулятор (Emulator) – Устройство, компьютерная программа или система, которые принимают те же входные данные и формируют те же выходные данные, что и заданная система, используя для этого тот же объектный код.